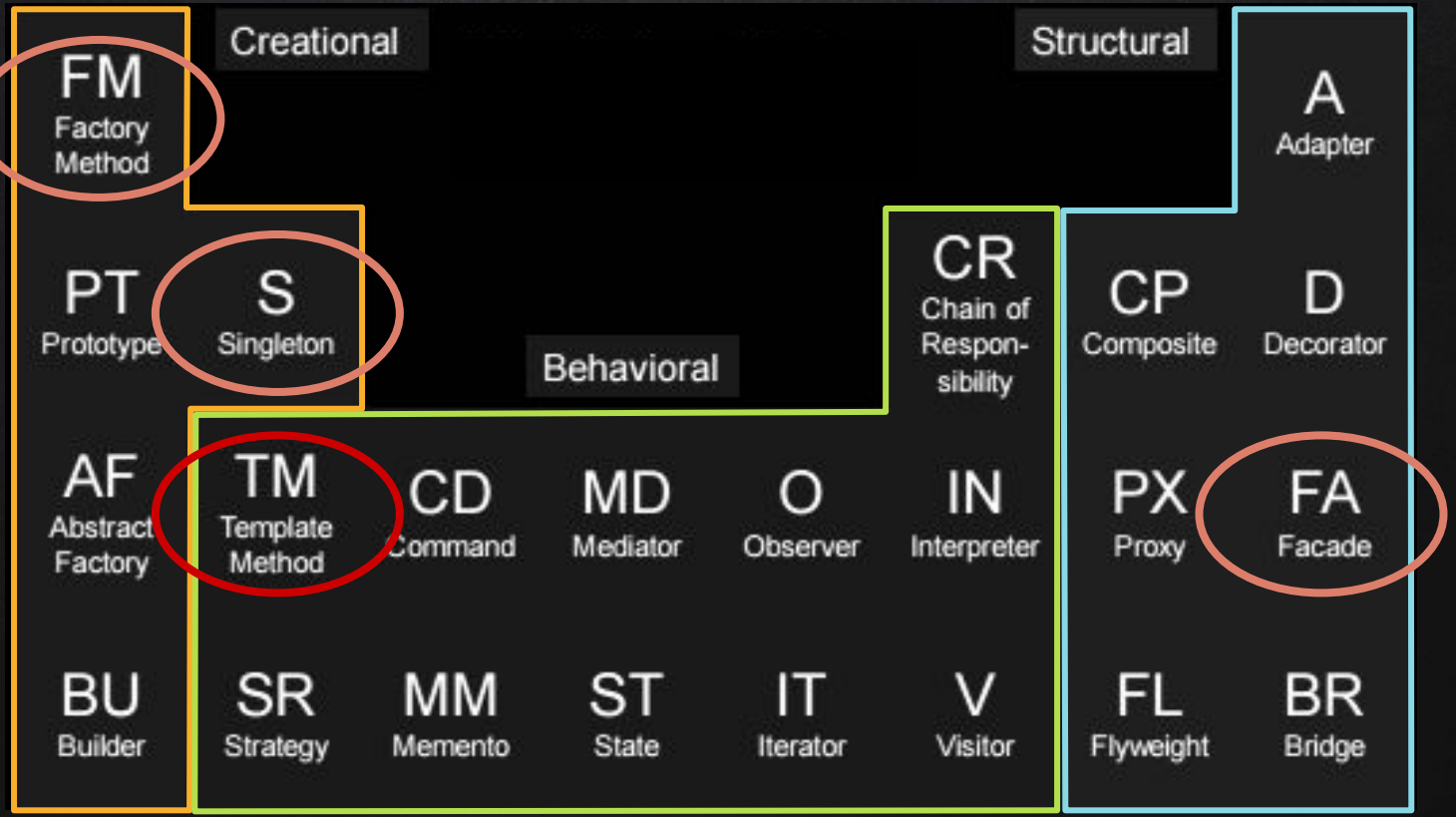


# Teórica 8
# Padrões de Desenho de Software

# Tipos de padrões



# Padrão Template Method (Behavioural)

Define as etapas de um algoritmo de forma geral, adiando a implementação de algumas etapas para subclasses. Isto é, preocupa-se com a atribuição de responsabilidades.

É mais adequado quando é possível generalizar o algoritmo entre dois algoritmos, implementados como classes, numa nova superclasse.

Devem pensar nisto como outra técnica usada quando percebem que têm duas classes separadas com funcionalidades e ordem de operações muito semelhantes.



# AS DUAS RECEITAS DE MASSA TÊM SEMELHANÇAS E DIFERENÇAS

Feito da mesma forma

- Ferver água
- Adicionar a massa
- Cozinhar a massa
- Escorrer e servir

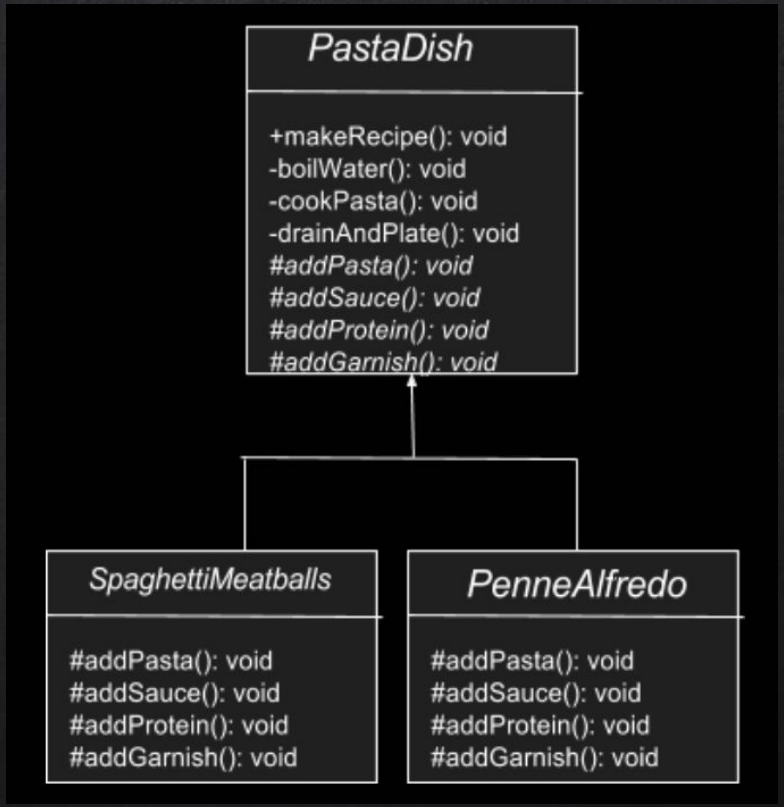
Diferentes formas de fazer

- Adicionar o molho
- Adicionar a proteína
- Adicionar a guarnição








PastaDish
+makeRecipe(): void
-boilWater(): void
-cookPasta(): void
-drainAndPlate(): void
#addPasta(): void
#addSauce(): void
#addProtein(): void
#addGarnish(): void
SpaghettiMeatballs
#addPasta(): void
#addSauce(): void
#addProtein(): void
#addGarnish(): void
PenneAlfredo
#addPasta(): void
#addSauce(): void
#addProtein(): void
#addGarnish(): void

# Superclasse do prato de massa

```
public abstract class PastaDish {
   final void makeRecipe() {
      boilWater();
      addPasta();
      cookPasta();
      drainAndPlate();
      addSauce();
      addProtein();
      addGarnish();
   }

   abstract void addPasta();
   abstract void addSauce();
   abstract void addProtein();
   abstract void addGarnish();

   private void boilWater() {
      System.out.println("Boiling water");
   }
   …
}
```

O método `makeRecipe()` é o template method, marcado como *final* – em Java, esta palavra-chave significa que o método declarado não pode ser substituído por subclasses.

Nenhuma subclasse de pratos específicos pode ter a sua própria versão de `makeRecipe()`, o que garante consistência nas etapas de preparação dos pratos e também reduz o código redundante.

# SUBCLASSES

```java
public class SpaghettiMeatballs extends PastaDish {
    public void addPasta() {
        System.out.println("Add spaghetti");
    }
    public void addProtein() {
        System.out.println("Add meatballs");
    }
    public void addSauce() {
        System.out.println("Add tomato sauce");
    }
    public void addGarnish() {
        System.out.println("Add Parmesan cheese");
    }
}
```

```java
public class PenneAlfredo extends PastaDish {
    public void addPasta() {
        System.out.println("Add penne");
    }
    public void addProtein() {
        System.out.println("Add chicken");
    }
    public void addSauce() {
        System.out.println("Add Alfredo sauce");
    }
    public void addGarnish() {
        System.out.println("Add parsley");
    }
}
```

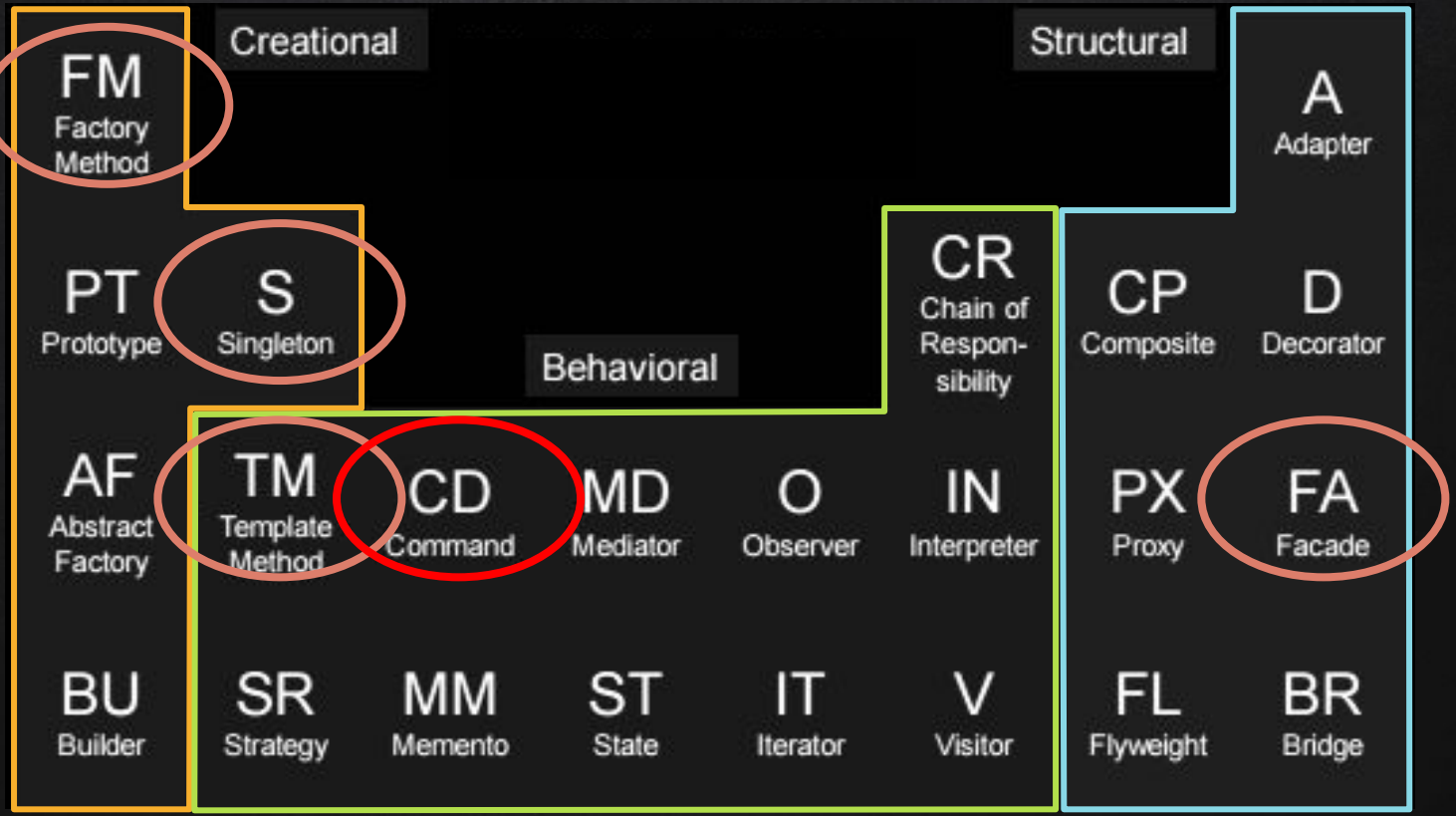
As subclasses estendem a classe `PastaDish` e implementam os métodos abstractos `addPasta()`, `addProtein()`, `addSauce()` e `addGarnish()`.

Estes métodos são invocados no método modelo `makeRecipe()` da superclasse `PastaDish()`. O *template method* é herdado por uma subclasse e comporta-se como esperado para fazer um prato com os ingredientes certos.

# Resumo – Padrão Template Method

- Útil se houver duas classes com funcionalidades semelhantes
- Duas classes com uma ordem de operações muito semelhante
- Aplicação prática de generalização e herança
- Ao escrever o código, poderá notar que duas classes distintas partilham semelhanças, como cada uma ter um método com um algoritmo muito semelhante
- Em vez de fazer alterações nos dois algoritmos nas duas classes, pode consolidar os algoritmos numa única classe
- Generaliza-se a partir de dois métodos separados num *template method* numa superclasse que passa a ser herdada pelas subclasses
- As alterações são feitas nas implementações das subclasses

# Tipos de Padrões

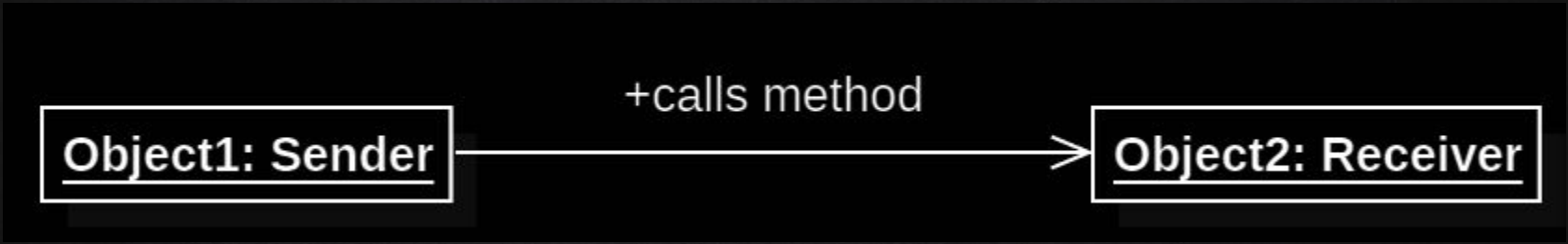


# Padrão Command

O padrão *command* encapsula um pedido como um objeto próprio.



Em geral, quando um objecto faz um pedido para que um segundo objecto execute uma acção, o primeiro objecto invocaria um método do segundo objecto e o segundo objecto completaria a tarefa.




Como podemos dissociar o remetente (cliente) do pedido do destinatário?

# Padrão Command




Um objecto remetente pode criar um objecto *command* para encapsular o pedido.

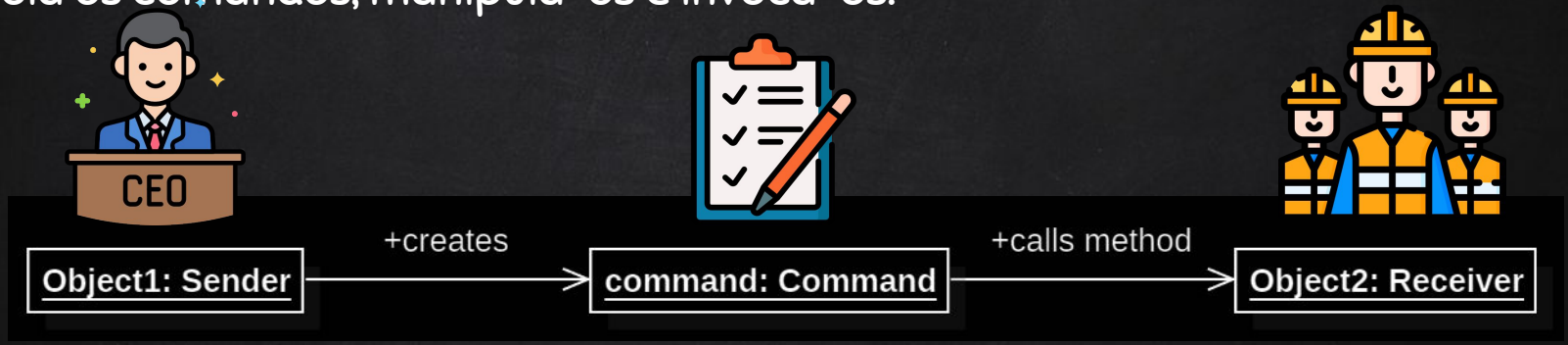
Mas o que faz com que o objecto *command* faça o que deve fazer e obtenha o objecto receptor específico para concluir a tarefa?

# Padrão Command



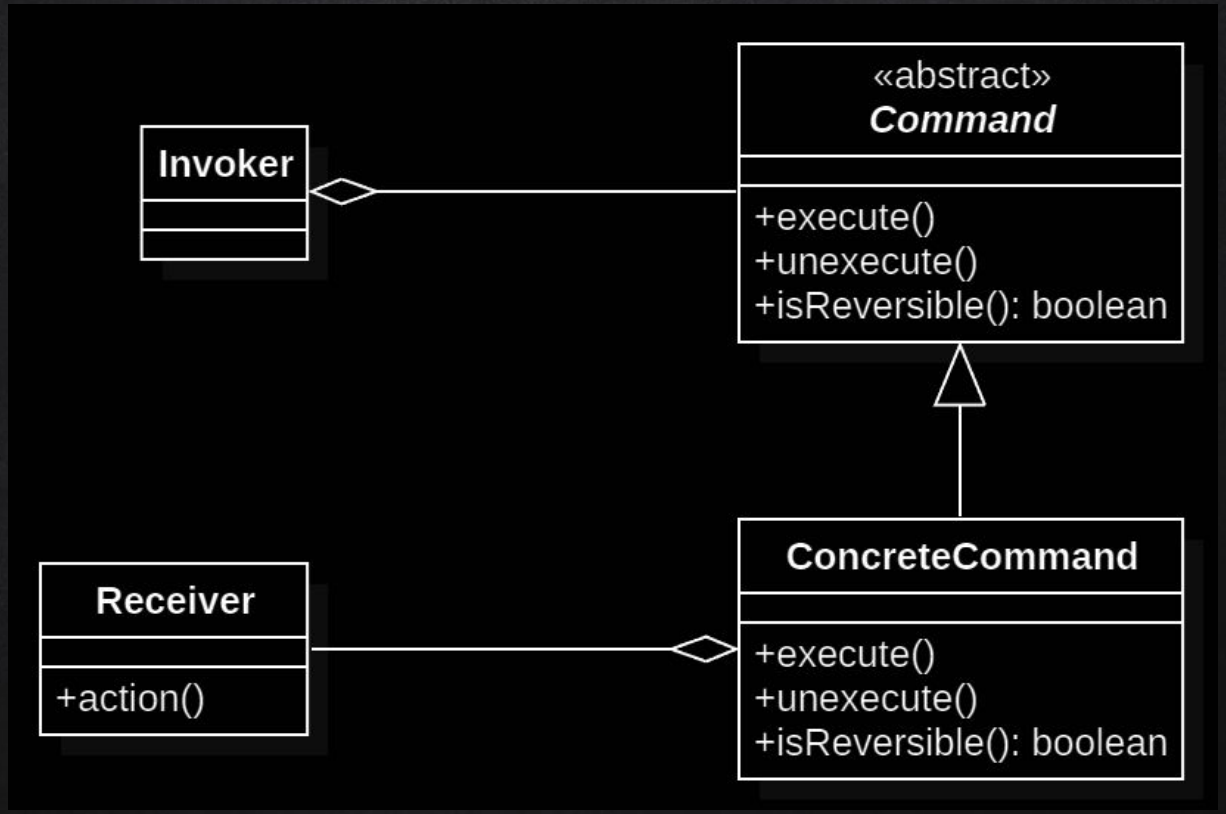
O objeto *invoker* invoca os objectos de comando para concluir qualquer tarefa que deva ser realizada.

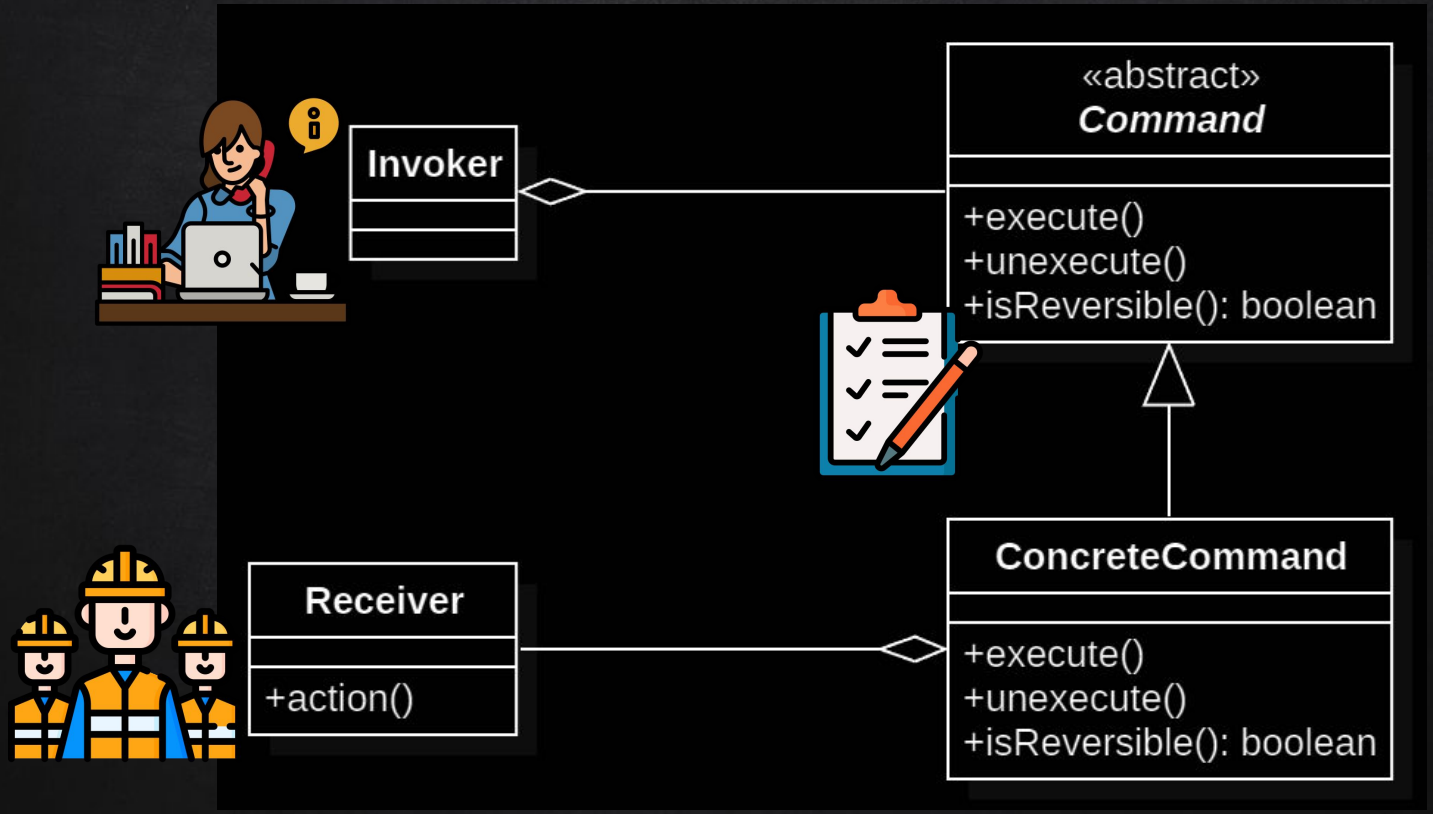
Um *command manager* também pode ser usado pelo *invoker* que, no fundo, controla os comandos, manipula-os e invoca-os.






invoker


«abstract»
Command
+execute()
+unexecute()
+isReversible(): boolean
Invoker
ConcreteCommand
+execute()
+unexecute()
+isReversible(): boolean
Receiver
+action()


«abstract»
Command
+execute()
+unexecute()
+isReversible(): boolean
Invoker
Receiver
+action()
ConcreteCommand
+execute()
+unexecute()
+isReversible(): boolean

# Onde implementar? Quais são os objectivos?

Armazenar e agendar diferentes pedidos (objectos de comando)

- armazenados em listas e manipulados antes de serem concluídos
- podem ser colocados numa fila para que diferentes comandos possam ser agendados para serem executados em momentos diferentes

Por exemplo, alarme numa aplicação de calendário:

- Um objecto de comando poderia ser criado para fazer tocar o alarme
- Esse comando poderia ser colocado numa fila
- O comando é executado quando o evento é programado para ocorrer



# Onde implementar? Quais são os objetivos?

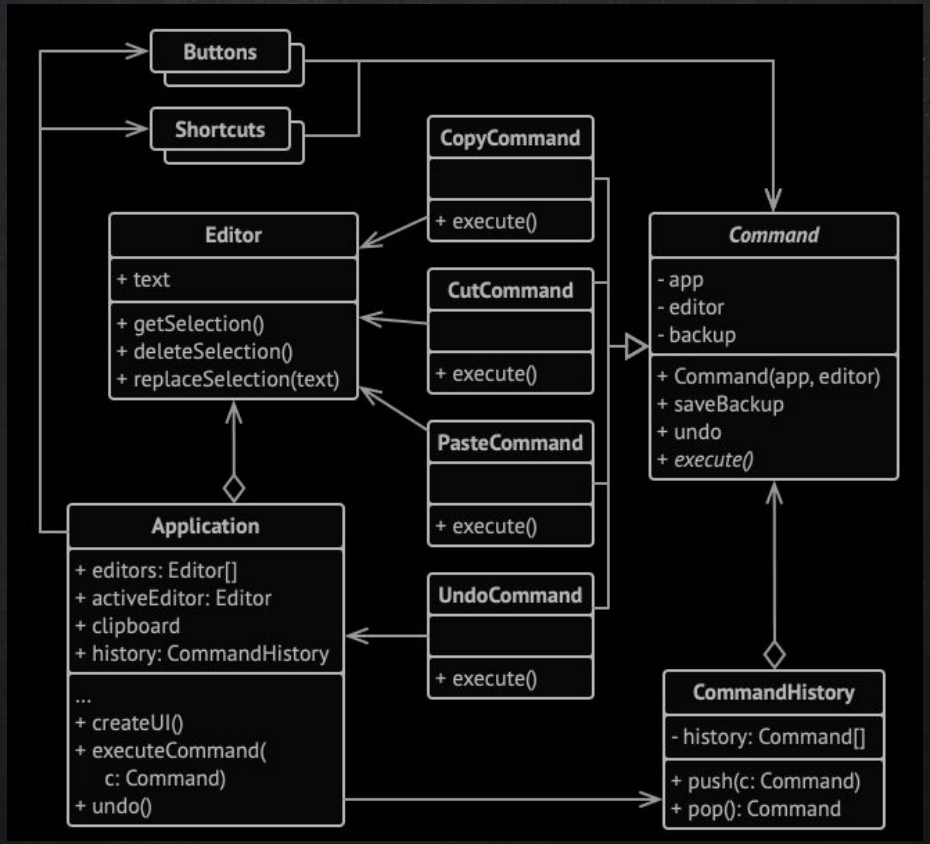
Permitir que os comandos sejam desfeitos ou refeitos



Por exemplo, podemos desfazer ou ou refazer alterações a um documento.
O software pode ter duas listas:

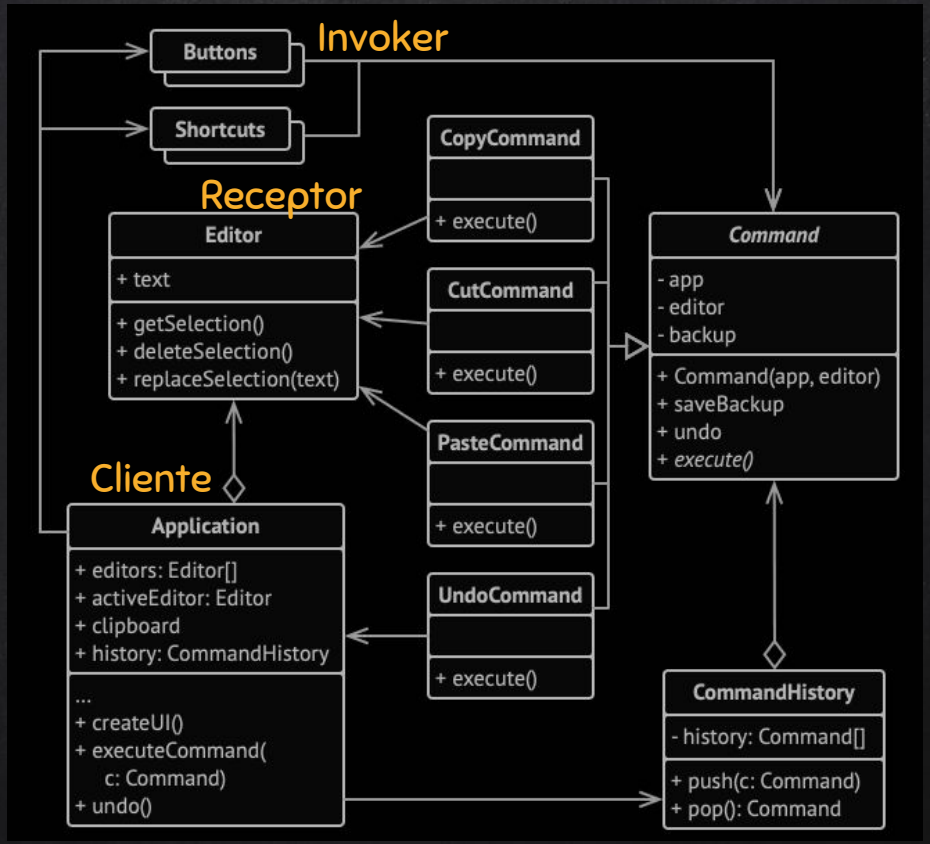
- history list – contém todos os comandos que foram executados
- redo list – usada para colocar comandos que foram desfeitos

# Exemplo do padrão Command num editor de texto




Exemplo retirado de https://refactoring.guru/

# Exemplo do padrão Command num editor de texto




Exemplo retirado de https://refactoring.guru/

# Exemplo

```
public class PasteCommand extends Command {
     private Document document;
     private int position;
     private String text;
     …
     public PasteCommand(Document document, int position, String text){
          this.document = document;
          this.position = position;
          this.text = =text;
     }
     public void execute() {
          document.insertText ( position, text );
     }
     public void unexecute() {
          document.deleteText ( position,
          text.length() );
     }
     public boolean isReversible() {
          return true;
     }
}
```

# Código do Invoker

```
//reference to the command manager
CommandManager commandManager =  CommandManager.getInstance();

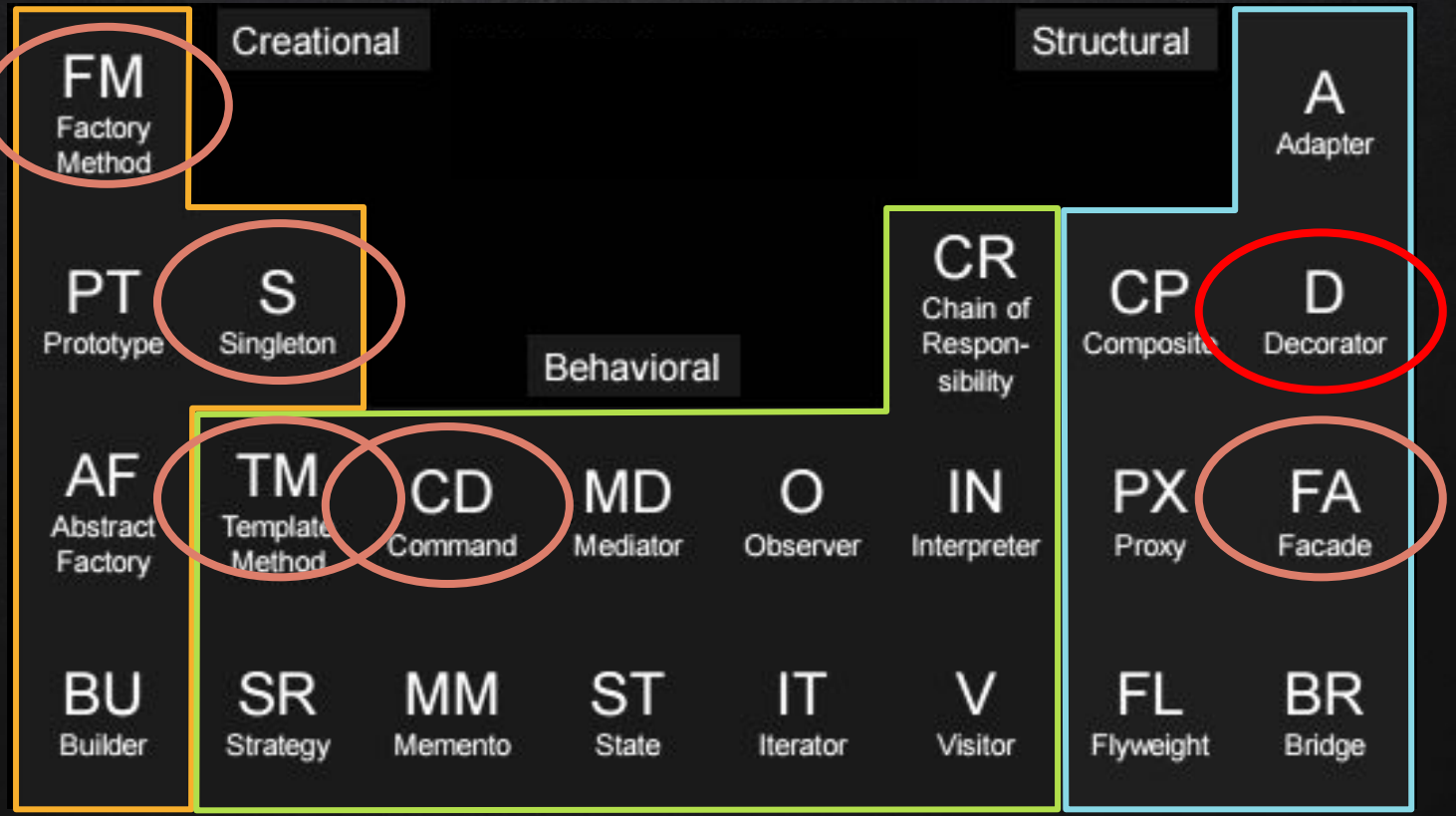
//create the command with appropriate information
Command command = new PasteCommand( aDocument, aPosition, AText );

//calls the command manager to execute the command
commandManager.invokeCommand ( command );
```

# Benefícios do padrão Command

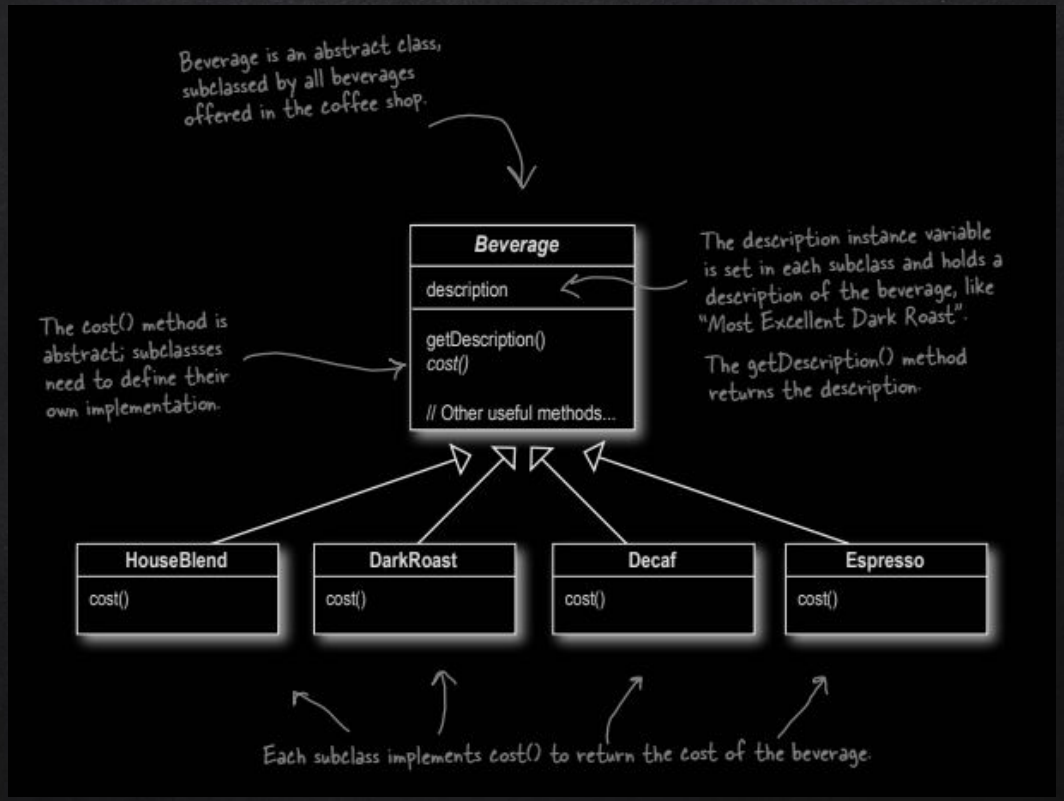
- Permite que os comandos sejam manipulados como objectos
- Podem ser adicionadas funcionalidades aos objectos de comando, por exemplo colocá-los em filas e adicionar funções de undo/redo
- Separa os objectos do programa de software
  - As classes não precisam de saber se existem outros objectos no sistema
- Permite que a lógica seja extraída das interfaces de utilizador
  - A interface do utilizador deve apenas obter informações de e para o utilizador
  - A lógica da aplicação não deve estar nas classes da interface do utilizador
  - O padrão *command* cria uma camada para onde os comandos vão
    - Sempre que um botão é clicado na interface, um comando é criado
    - É mais fácil adicionar alterações, como novos botões, à interface

# Tipos de padrões



# Vamos beber café

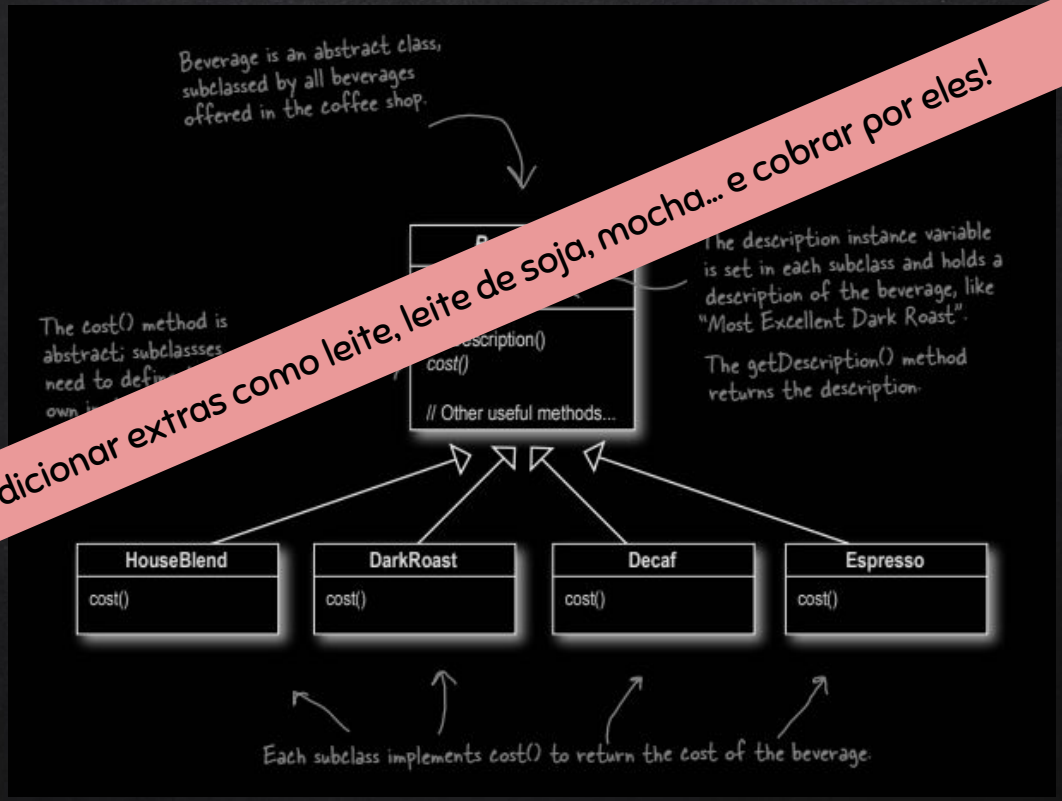





Exemplos retirados do livro "Head First" (ver referência do livro no final dos slides).

# Vamos beber café



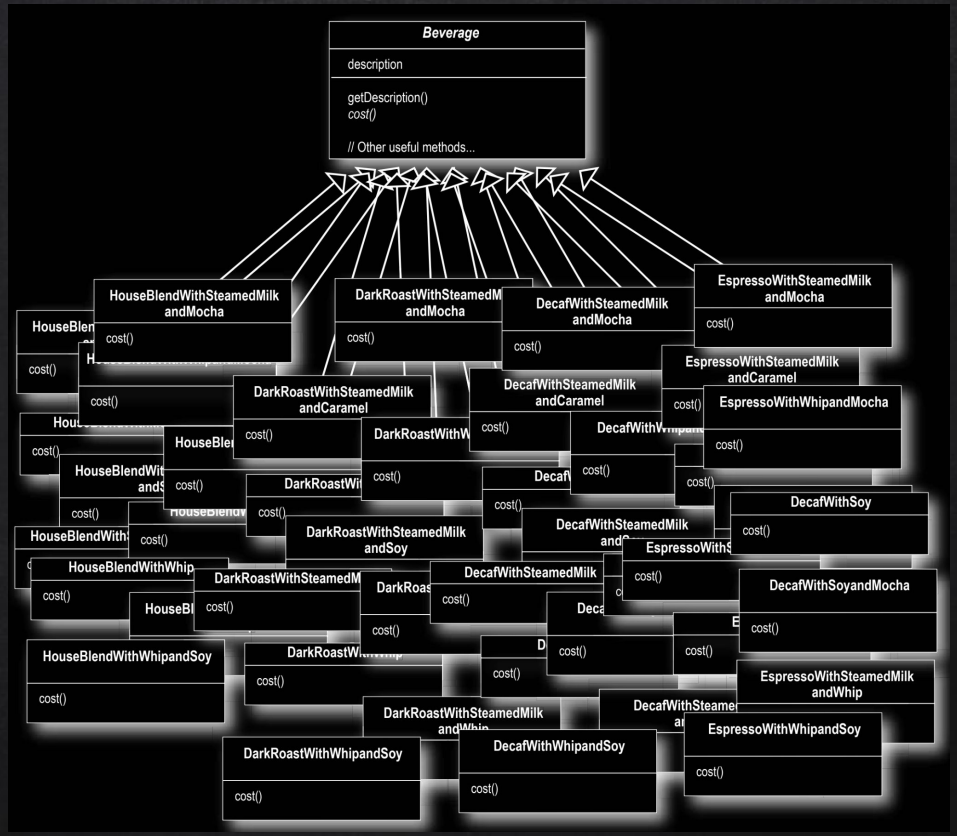



Agora queremos adicionar extras como leite, leite de soja, mocha... e cobrar por eles!

Exemplos retirados do livro "Head First" (ver referência do livro no final dos slides).

# Evolução do conceito

Solução possível?



# Isto tem problemas...

Foi preciso implementar, na fase de desenho, uma classe por combinação de extras, onde cada método calcula o custo do café com os extras.

- Por que não usar herança?
  - Ao herdar o comportamento por subclasse, o comportamento é definido estaticamente em tempo de compilação.
  - Todas as subclasses devem herdar o mesmo comportamento.

- Em vez disto, poderíamos ter o comportamento do objeto por composição
  - Pode ser feito dinamicamente em tempo de execução.
  - Podem adicionar-se novas funcionalidades escrevendo novo código em vez de alterar o código existente (menos bugs e efeitos colaterais no código pré-existente)

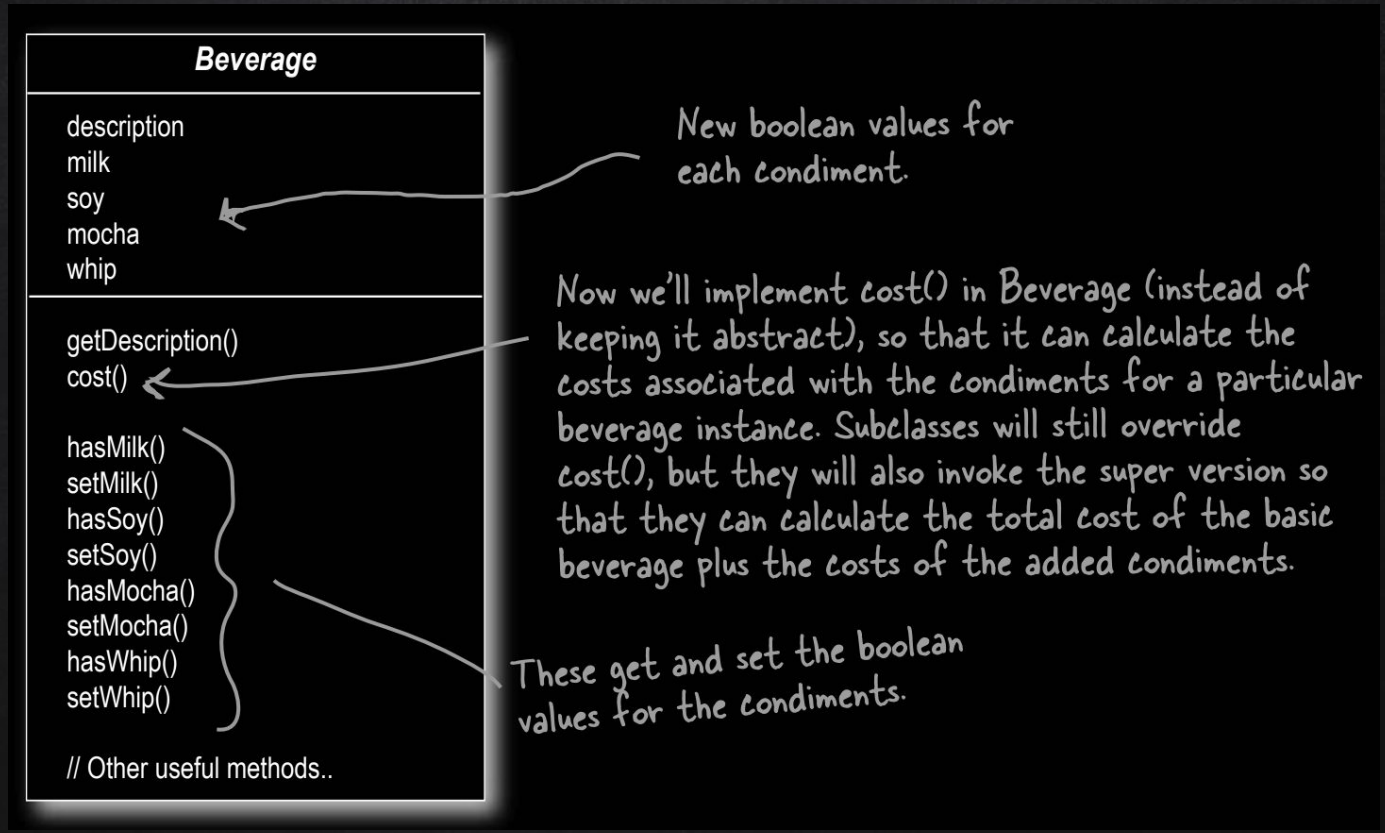
# Padrão Decorator (Structural)

Permite que comportamentos ou responsabilidades adicionais sejam dinamicamente anexados a um objeto, através do uso de agregação/composição para combinar comportamentos em tempo de execução.

Este padrão de desenho utiliza interfaces e herança, para que as classes estejam em conformidade com o tipo comum, cujas instâncias podem ser empilhadas de forma compatível. Isto cria uma combinação coerente de comportamentos.

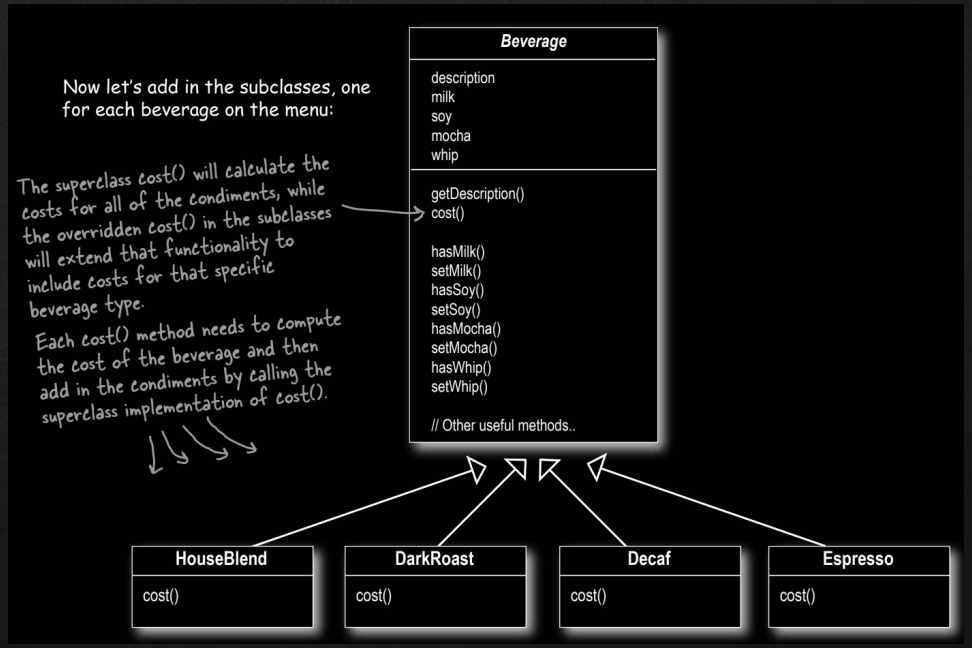


# Segunda tentativa...



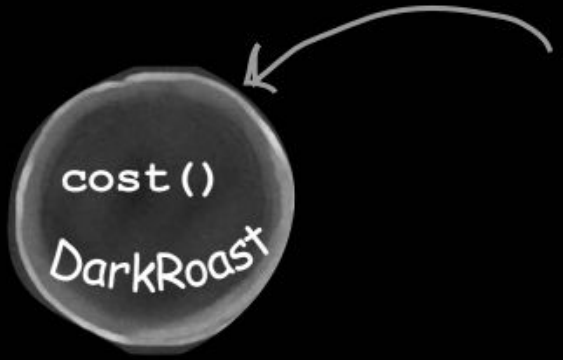
# Melhor. Mas será que está preparado para o futuro?

O Princípio Open-Closed:
As classes devem ser abertas para extensão, mas fechadas para modificação.



# Vamos criar um pedido de bebidas com um Decorator!

**We start with our DarkRoast object.**




Remember that DarkRoast inherits from Beverage and has a cost() method that computes the cost of the drink.

# Vamos criar um pedido de bebidas com um Decorator!

**The customer wants Mocha, so we create a Mocha object and wrap it around the DarkRoast.**

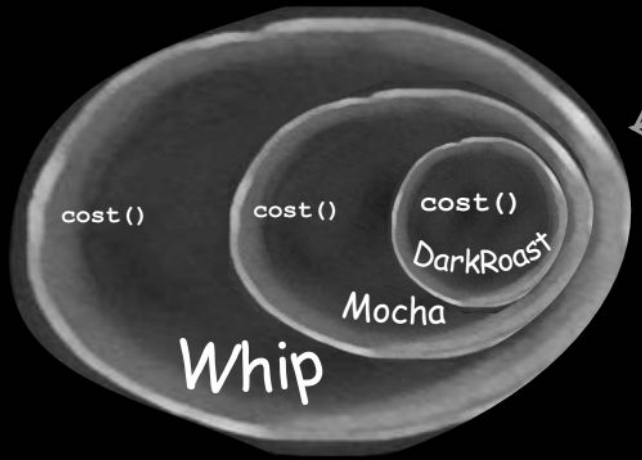



The Mocha object is a decorator. Its type mirrors the object it is decorating, in this case, a Beverage. (By "mirror", we mean it is the same type..)

So, Mocha has a cost() method too, and through polymorphism we can treat any Beverage wrapped in Mocha as a Beverage, too (because Mocha is a subtype of Beverage).

# Vamos criar um pedido de bebidas com um Decorator!

**The customer also wants Whip, so we create a Whip decorator and wrap Mocha with it.**




Whip is a decorator, so it also mirrors DarkRoast's type and includes a cost() method.

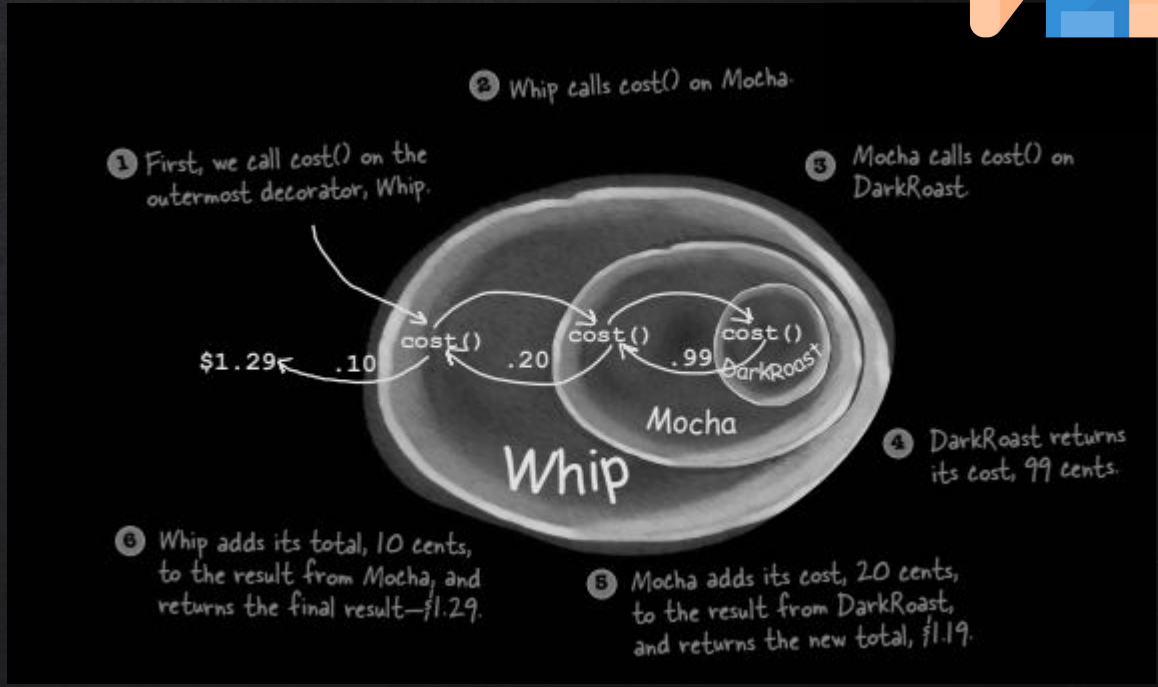
So, a DarkRoast wrapped in Mocha and Whip is still a Beverage and we can do anything with it we can do with a DarkRoast, including call its cost() method.

# Padrão Decorator (Structural)

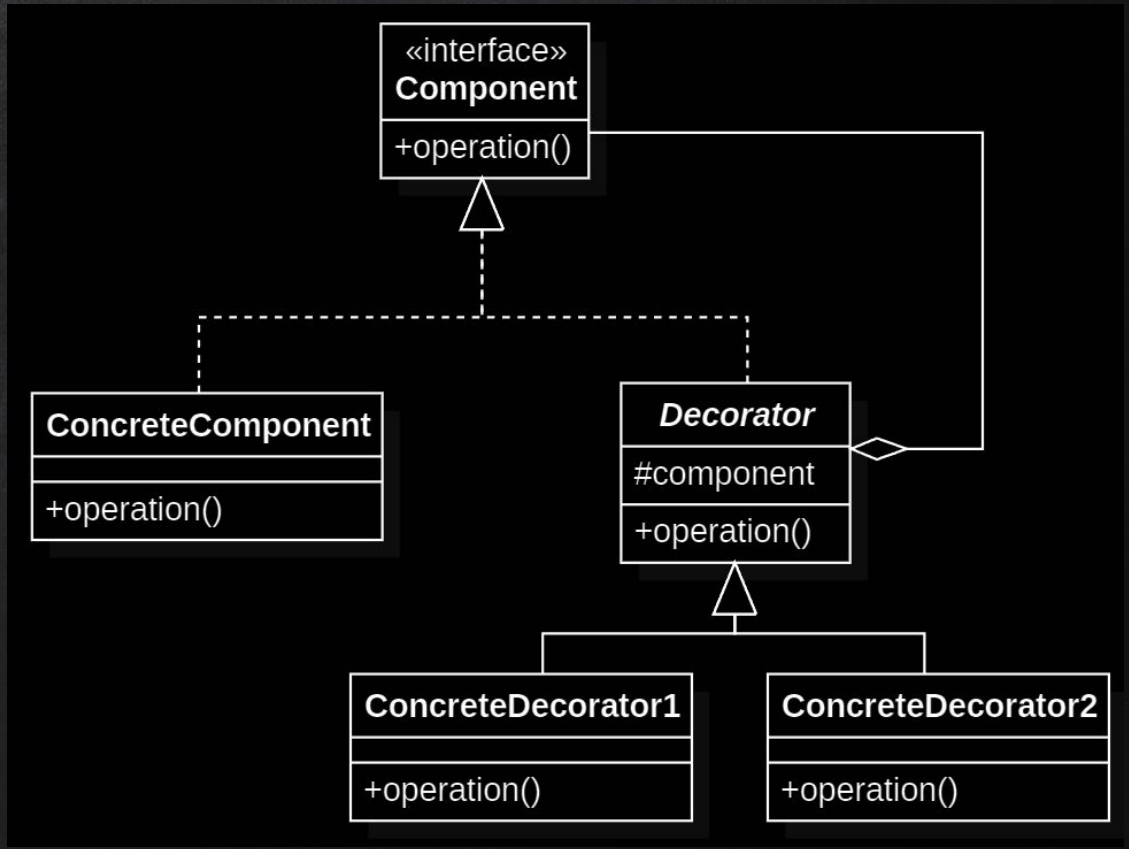


*Wrapping*:

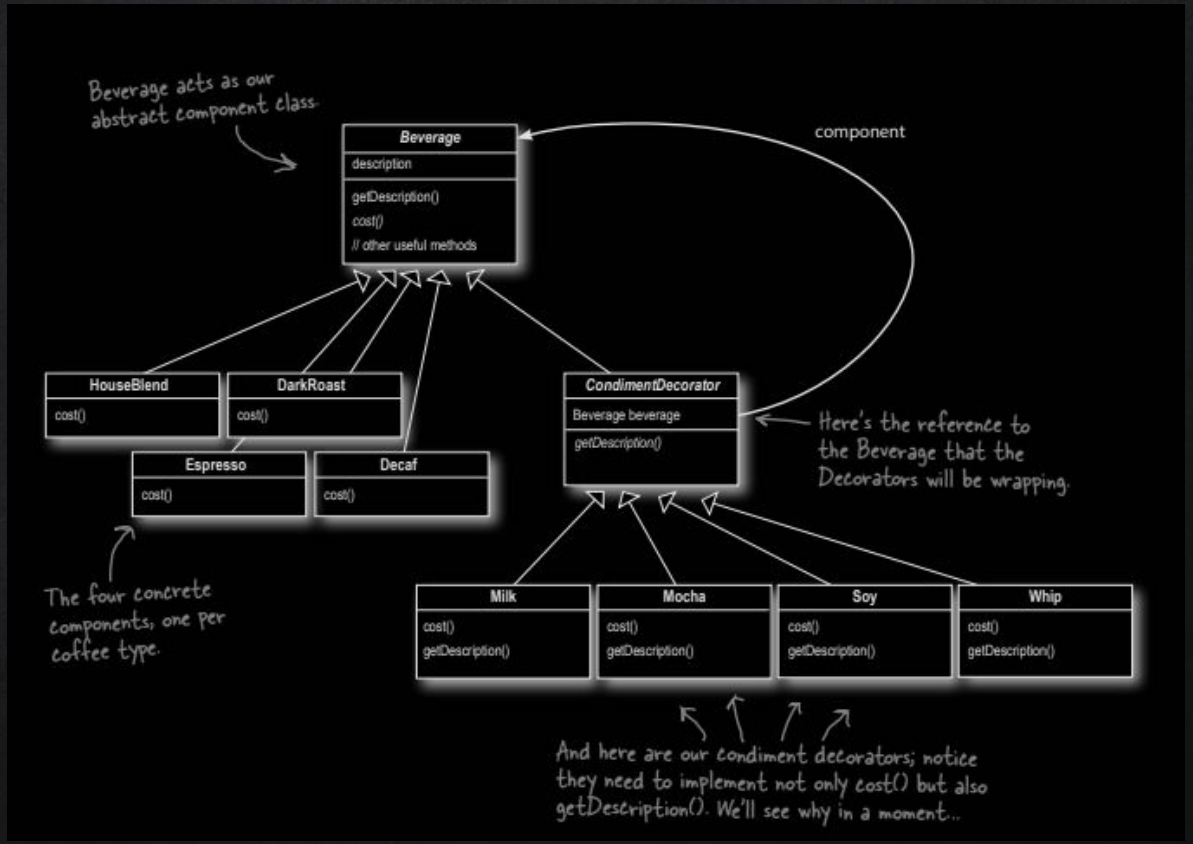
- Os *decoradores* têm o mesmo supertipo
- É possível usar um ou mais *decoradores* para fazer *wrap* de um objecto
- É possível passar um objecto decorado no lugar do original (objecto encapsulado)
- O decorador adiciona o seu próprio comportamento antes e depois de delegar ao objecto que decora o resto do trabalho
- Os objectos podem ser decorados a qualquer momento



- Interface `Component` que define o tipo comum para todas as classes.
- A classe cliente está à espera a mesma interface em todas as classes de componentes.
- A classe de `ConcreteComponent` implementa a interface `Component` e pode ser instanciada. A instância desta classe pode ser usada como um objeto base na pilha.
- *`Decorator`* é uma classe abstracta que implementa a interface `Component`.
  - Agrega/compõe outros tipos de componentes, o que permite "empilhar" componentes uns sobre os outros.
  - Também serve como superclasse abstracta de classes `ConcreteDecorator`, cada uma das quais oferecerá um incremento no comportamento.



# Voltando ao café



# Vamos implementá-lo!

```
public abstract class Beverage {
   String description = “Unknown Beverage”;

   public String getDescription() {
      return description;
   }
   public abstract double cost();
}
```

# Vamos implementá-lo!

```
public abstract class CondimentDecorator extends Beverage {
   public abstract String getDescription();
}
```

# Vamos implementá-lo!

```
public class Espresso extends Beverage {

    public Espresso() {
        description = “Espresso”;
    }

    public double cost() {
        return 1.99;
    }
}
```

# Vamos implementá-lo!

```
public class HouseBlend extends Beverage {
    public HouseBlend() {
        description = “House Blend Coffee”;
    }

    public double cost() {
        return .89;
    }
}
```

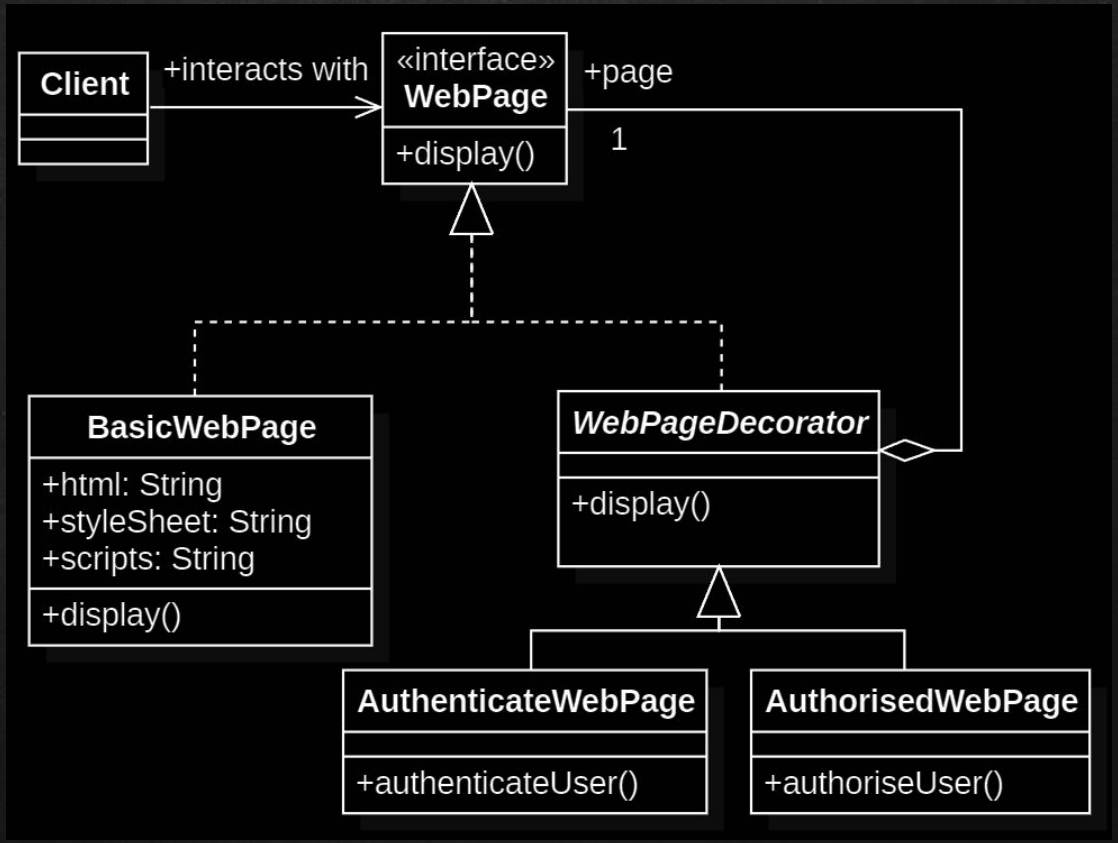
# Vamos implementá-lo!

```
public class Mocha extends CondimentDecorator {
   Beverage beverage;
   public Mocha(Beverage beverage) {
      this.beverage = beverage;
   }
   public String getDescription() {
      return beverage.getDescription() + “, Mocha”;
   }
   public double cost() {
      return .20 + beverage.cost();
   }
}
```

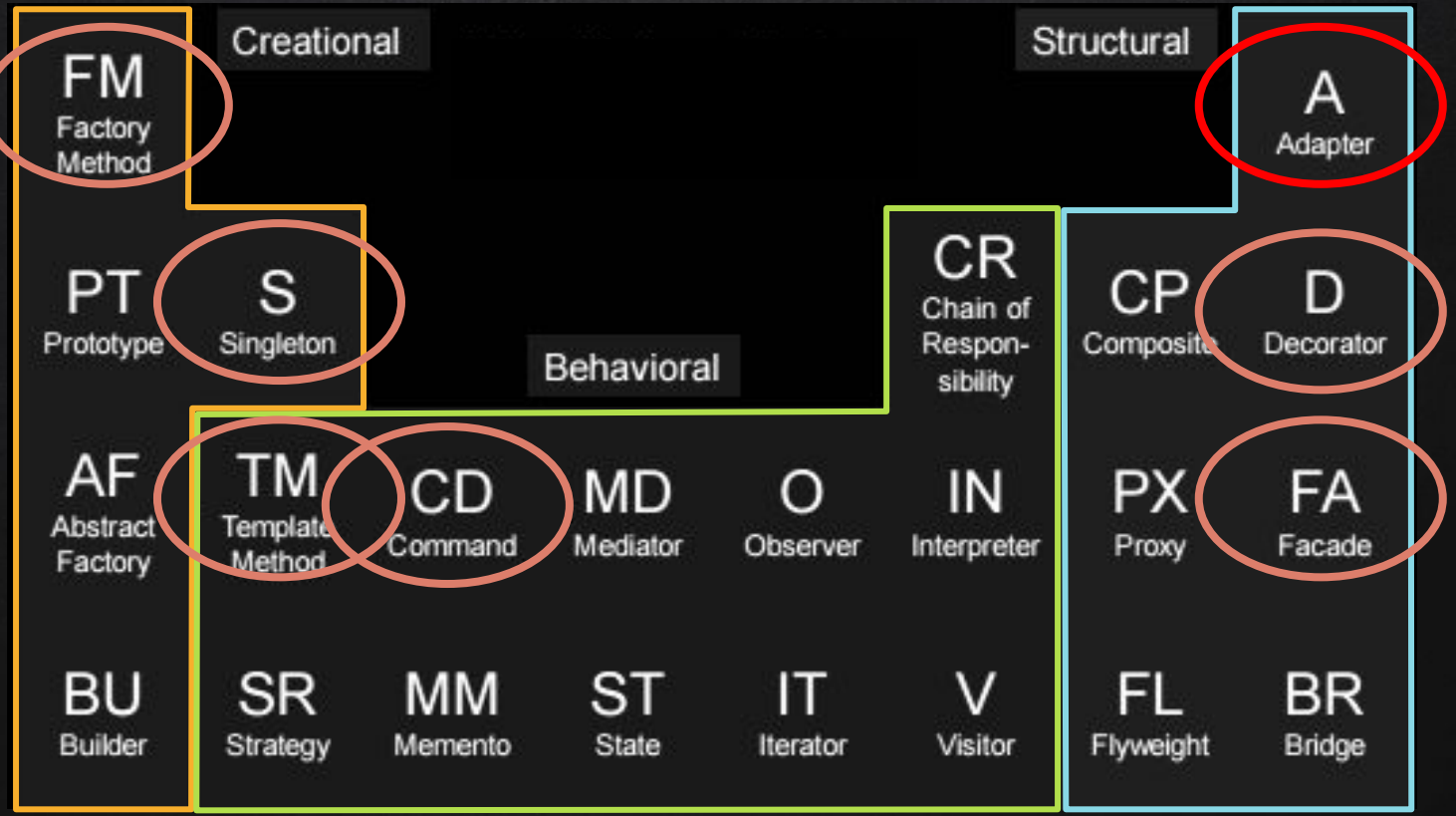
# Hora de testar o café! Ou melhor, prová-lo?

```
public class TestCoffee {
   public static void main(String args[]) {
      Beverage beverage = new Espresso();
      System.out.println(beverage.getDescription() + " $" + beverage.cost());
      Beverage beverage2 = new DarkRoast();
      beverage2 = new Mocha(beverage2);
      beverage2 = new Mocha(beverage2);
      beverage2 = new Whip(beverage2);
      System.out.println(beverage2.getDescription() + " $" + beverage2.cost());
      Beverage beverage3 = new HouseBlend();
      beverage3 = new Soy(beverage3);
      beverage3 = new Mocha(beverage3);
      beverage3 = new Whip(beverage3);
      System.out.println(beverage3.getDescription() + " $" + beverage3.cost());
   }
}
```

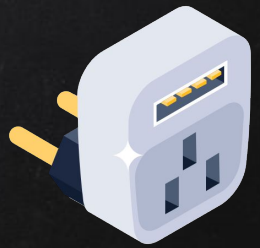
# Outro exemplo com páginas web



# Tipos de padrões



# Padrão Adapter (Structural)



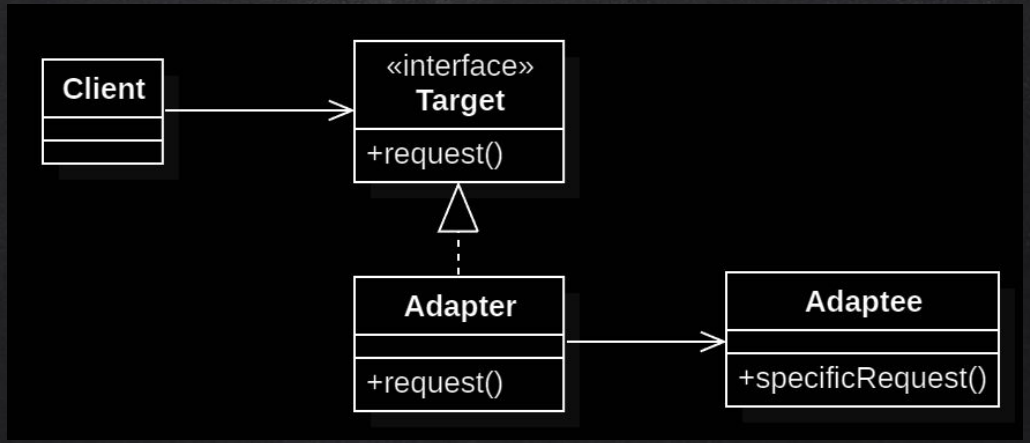
Facilita a comunicação entre dois sistemas existentes, oferecendo uma interface compatível.

Exemplo:

Um sistema de software existente que precisa de integrar uma nova biblioteca de classes de um fornecedor, mas o novo fornecedor projectou as suas interfaces de forma diferente do fornecedor anterior.

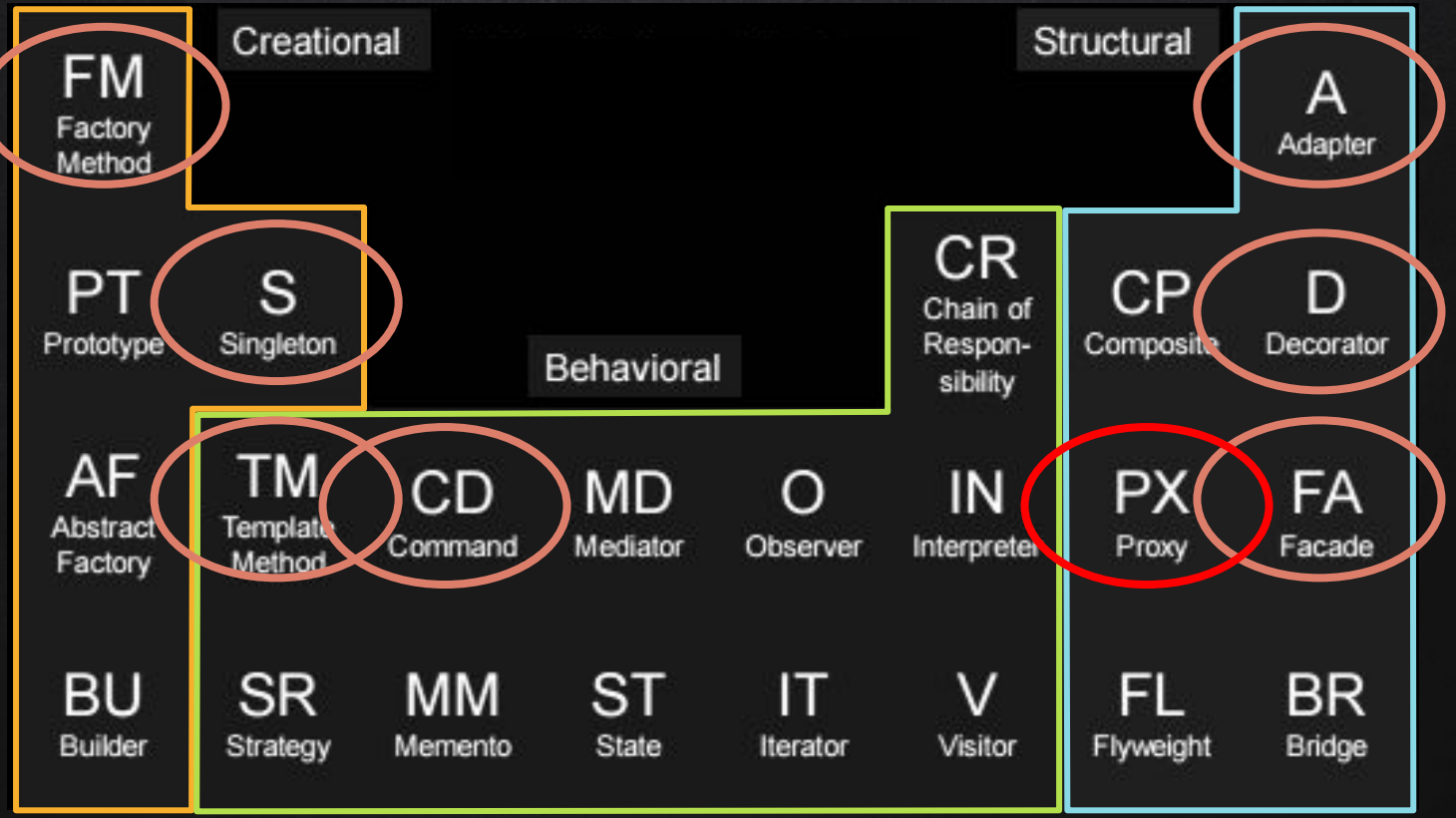
Não queremos resolver o problema alterando o código existente (e não podemos alterar o código do fornecedor)



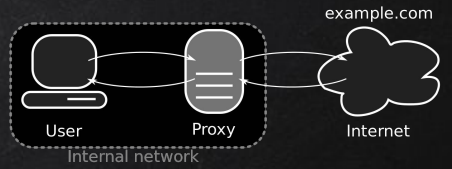



- Classe `Client` – a parte do sistema que pretende utilizar uma biblioteca de terceiros ou sistemas externos.
- Classe `Adaptee` – a biblioteca de terceiros ou sistema externo que será utilizada.
- Classe `Adapter` – classe que fica entre o cliente e o `Adapter`.
  - Está conforme com o que o cliente está à espera, implementando uma interface `Target`.
  - Traduz o pedido do cliente numa mensagem que o `Adaptee` compreende e devolve o pedido traduzido ao `Adaptee`.
  - É um tipo de classe *wrapper*.
- Interface de destino – É utilizada pelo cliente para enviar um pedido ao `Adapter`.

# Tipos de padrões



# Padrão Proxy (Structural)




Permite que uma classe proxy represente uma classe real.

Um proxy é algo que actua como uma versão simplificada do objecto original.

Um objecto proxy oferece as mesmas operações que o objecto original, mas pode delegar pedidos recebidos por si para o objecto original para serem executados. Quando o objecto original termina a execução devolve os resultados para o proxy que, por sua vez, os devolve ao cliente.

# Quando é que o padrão proxy é útil?

- Para actuar como um proxy virtual
  - Quando uma classe proxy é usada em vez de uma classe real que consome muitos recursos para ser instanciada.
  - Utiliza-se normalmente em imagens de páginas da web ou editores gráficos, pois uma imagem de alta definição pode ser extremamente grande para carregar.
- Para atuar como um proxy de proteção
  - Quando uma classe proxy é usada para controlar o acesso à classe real.
  - Por exemplo, um sistema usado por alunos e por professores pode limitar o acesso dependendo do tipo de utilizador.
- Para atuar como um proxy remoto
  - Quando uma classe proxy é local e a classe real existe remotamente.
  - Por exemplo, o Google Docs utiliza um proxy remoto: os browsers têm todos os objectos de que necessitam localmente mas estes objectos também existem num servidor remoto.

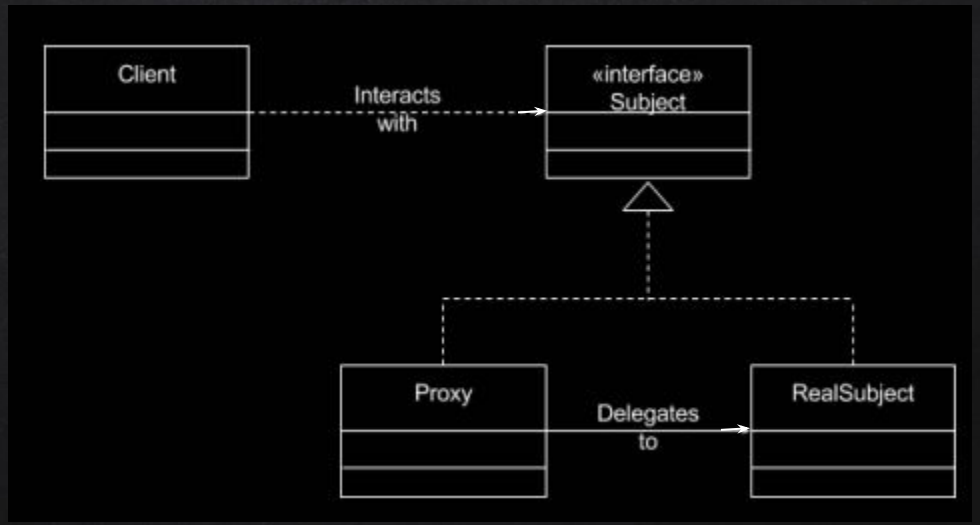
A classe proxy encapsula e pode delegar ou redirecionar pedidos para a classe original.

No entanto, nem todas as classes são delegadas, pois

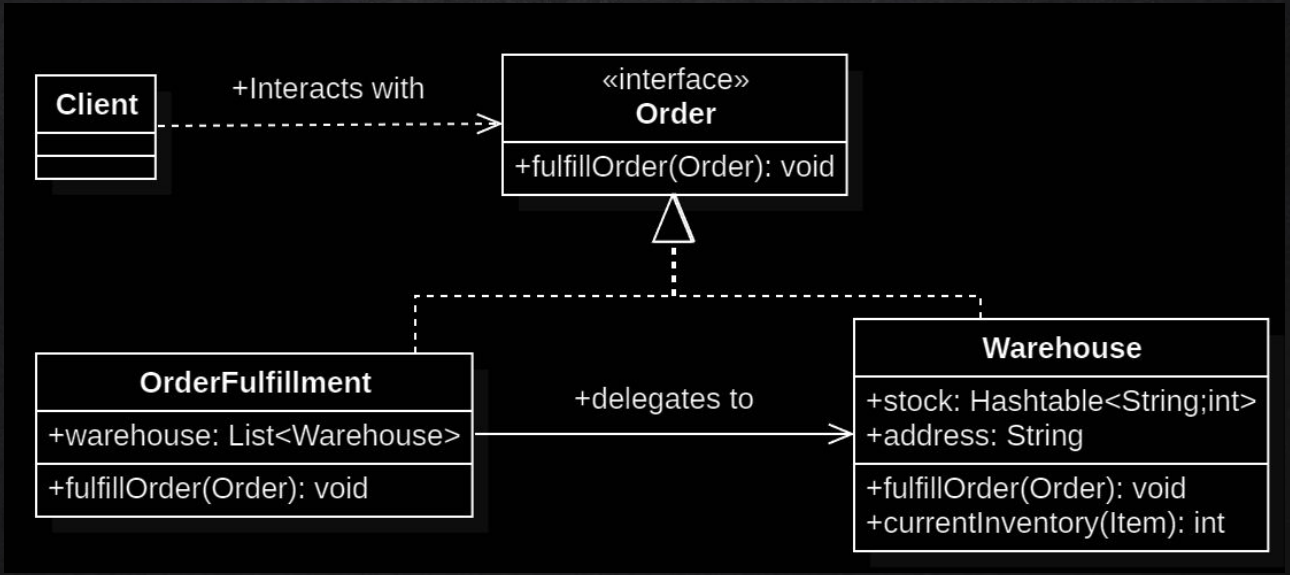
- a classe proxy pode lidar com algumas das suas responsabilidades mais leves.
- Apenas os pedidos substanciais são re-encaminhados para a classe original.

Por este motivo, a classe proxy deve oferecer os exactamente as mesmas operações.

Se ambas as classes implementarem a mesma interface estamos na presença de polimorfismo.

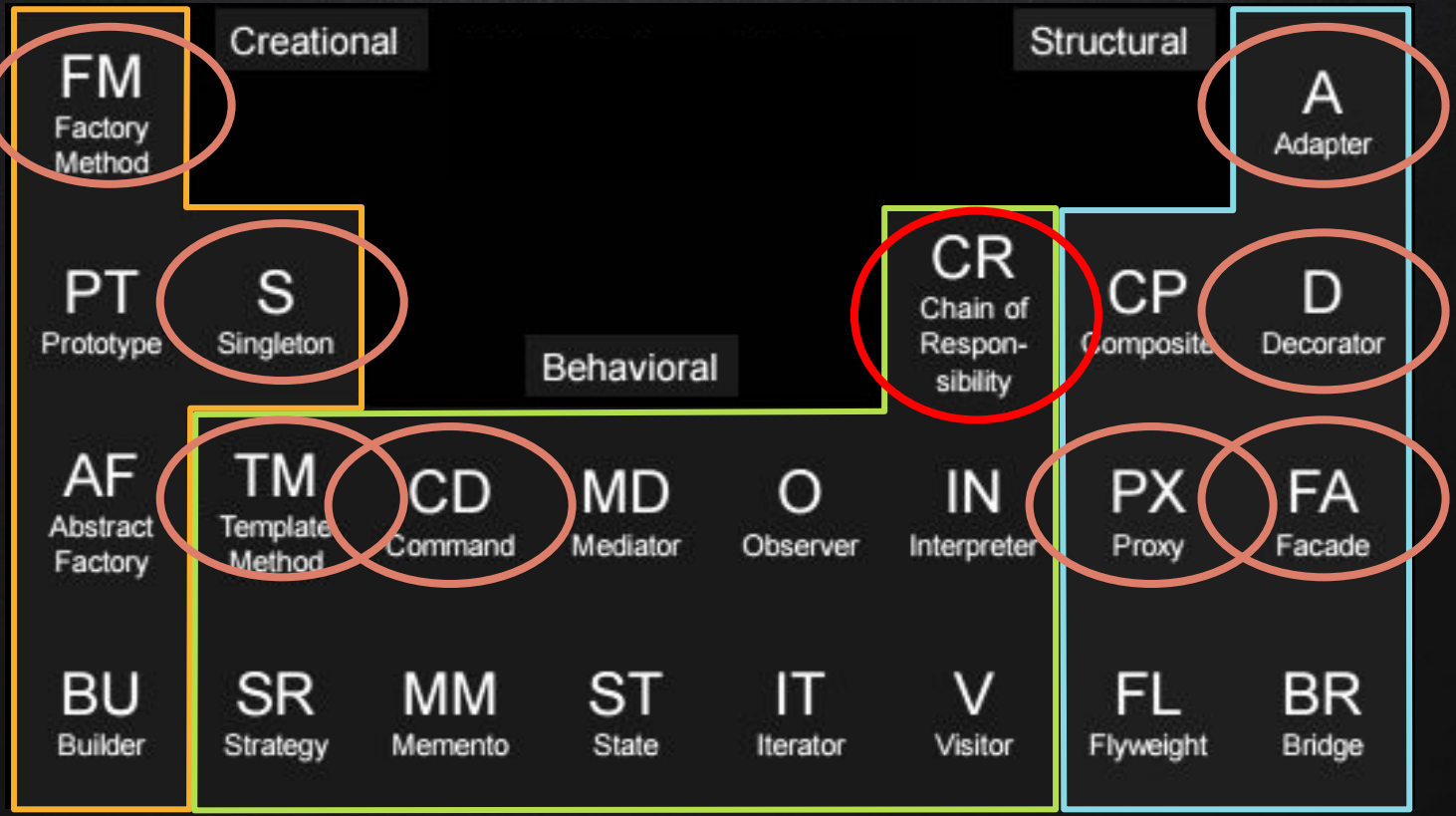


# Exemplo de padrão proxy: Supermercado com distribuição global e Armazém



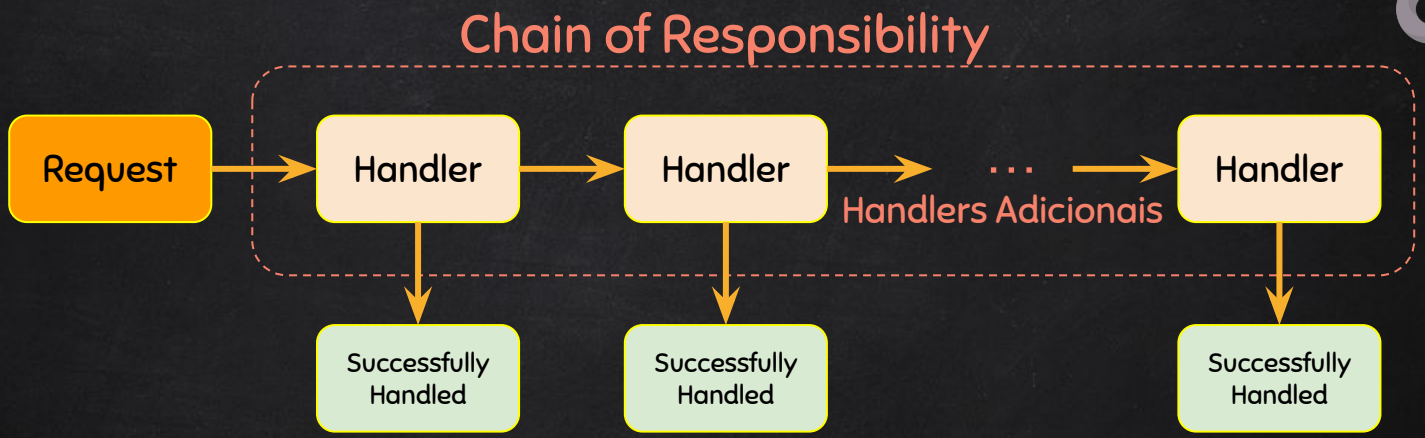

Neste cenário, é necessário determinar para qual armazém enviar as encomendas. Um sistema que encaminha as encomendas para um armazém adequado evita que os seus armazéns recebam encomendas que não podem satisfazer. Um proxy pode proteger o seu objeto original, o armazém, de receber encomendas se não tiverem stock suficiente para a satisfazer.

# Tipos de padrões



# Padrão Chain of Responsibility (Behavioural)

É uma cadeia de objetos responsáveis por processar pedidos.




Quando um cliente envia um pedido, o primeiro handler da cadeia vai tentar processá-lo. Se conseguir, o pedido fica concluído.

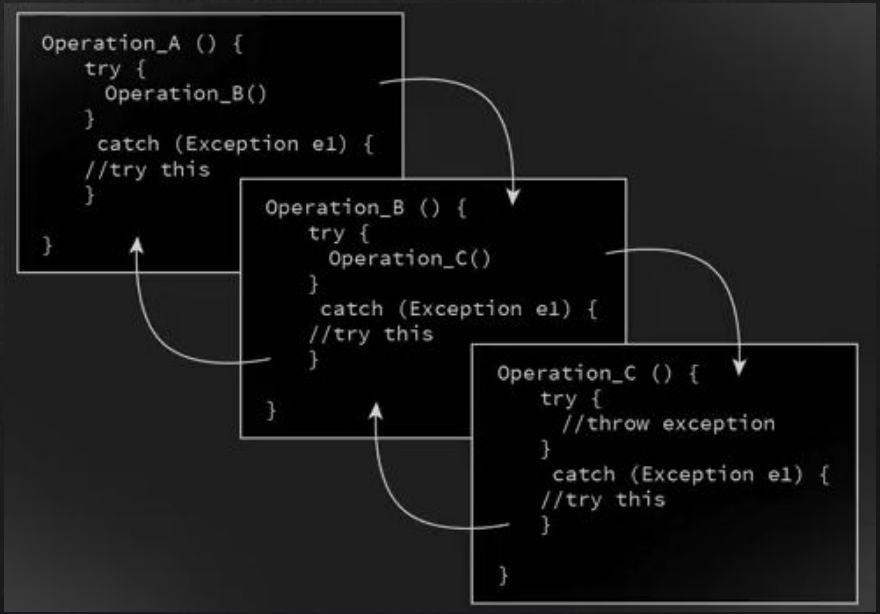
No entanto, se o handler não o processar, o pedido é re-encaminhado para o próximo handler da cadeia. O processo continua até que um handler consiga processar o pedido com sucesso.

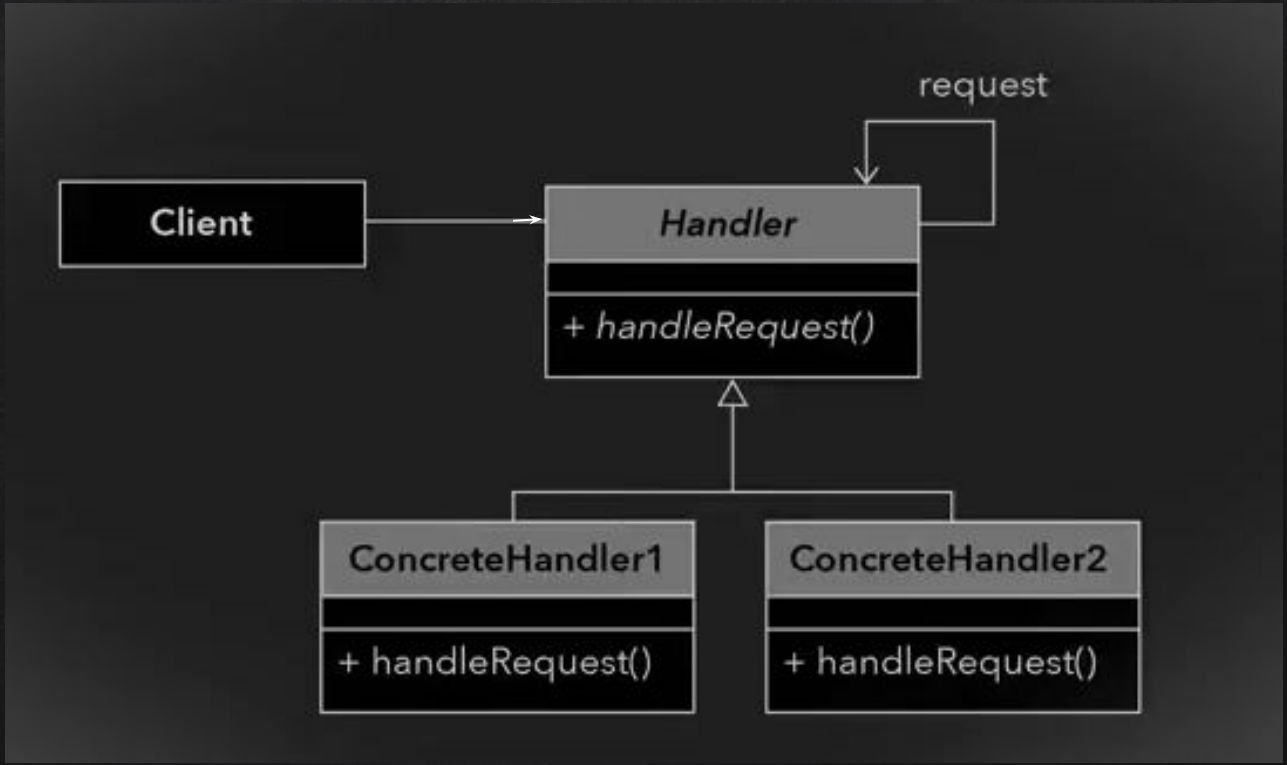
Se nenhum handler da cadeia for capaz de processar o pedido, este não é processado.

# Semelhante ao tratamento de excepções em Java

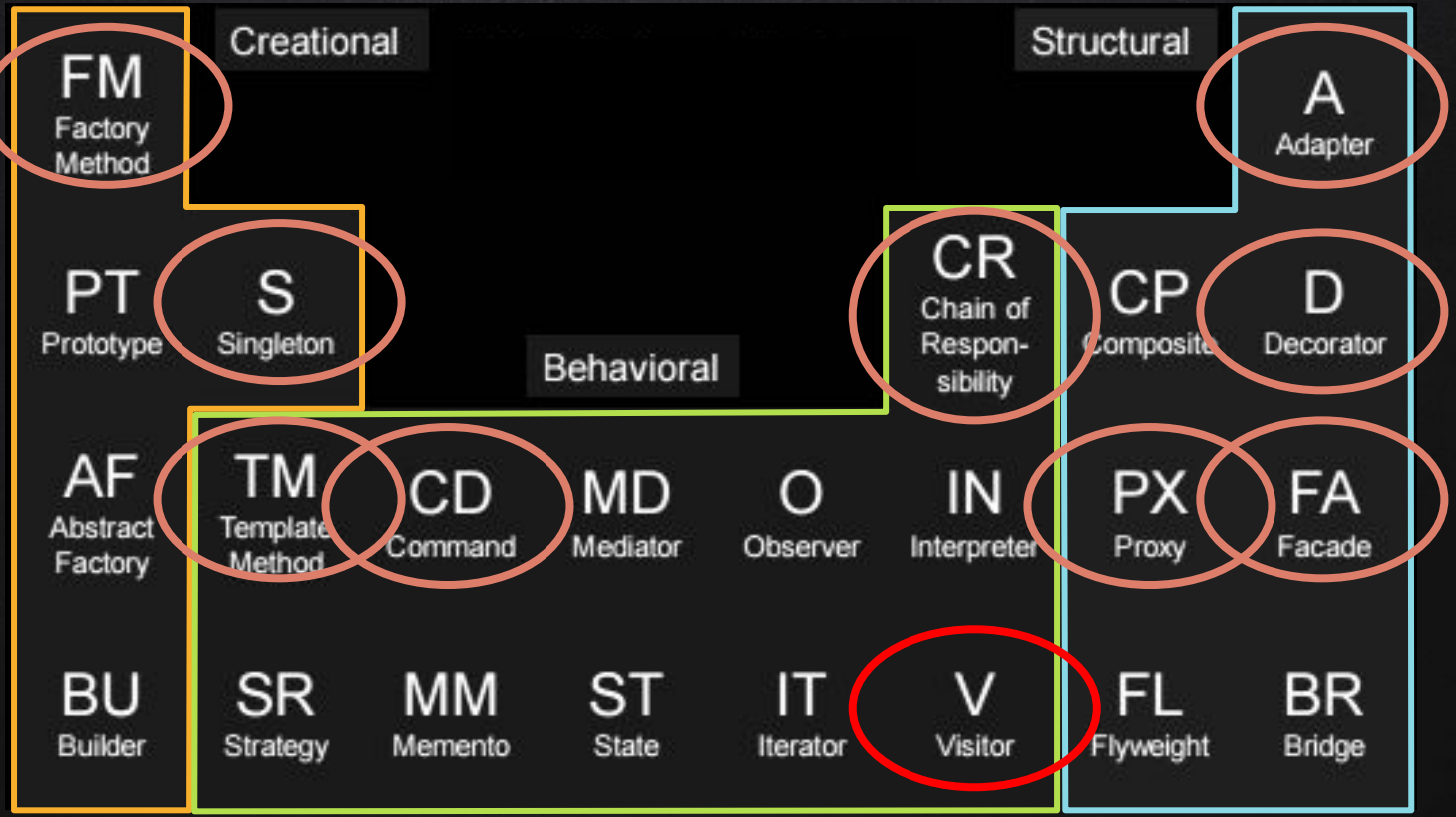
No tratamento de excepções, escreve-se uma série de blocos `try/catch` para garantir que as excepções sejam tratadas adequadamente.

Quando ocorre uma excepção, espera-se que um dos blocos `catch` seja capaz de a resolver.




request
Client
Handler
+ handleRequest()
ConcreteHandler1
+ handleRequest()
ConcreteHandler2
+ handleRequest()

# Tipos de padrões



# Padrão Visitor (Behavioural)



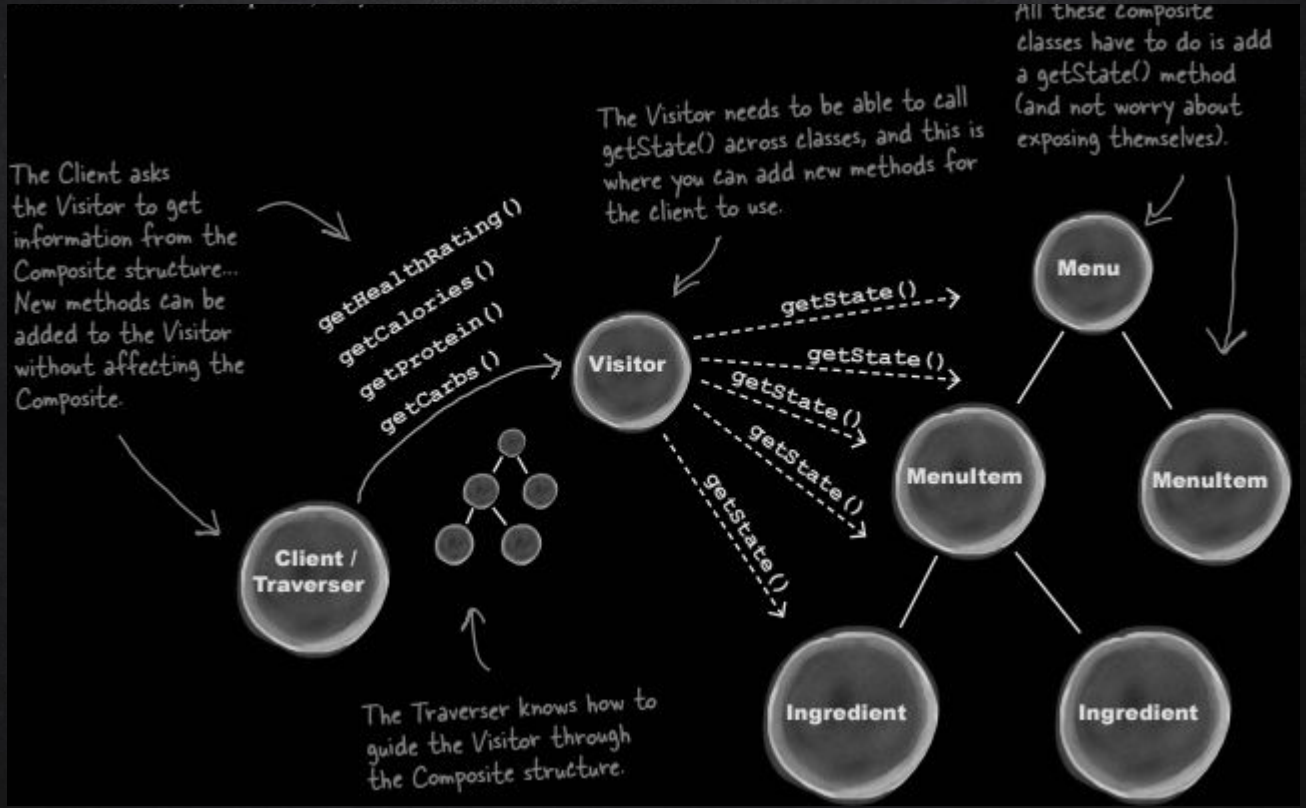

Quando queremos adicionar capacidades a uma composição de objectos e o encapsulamento não é importante.

Benefícios:

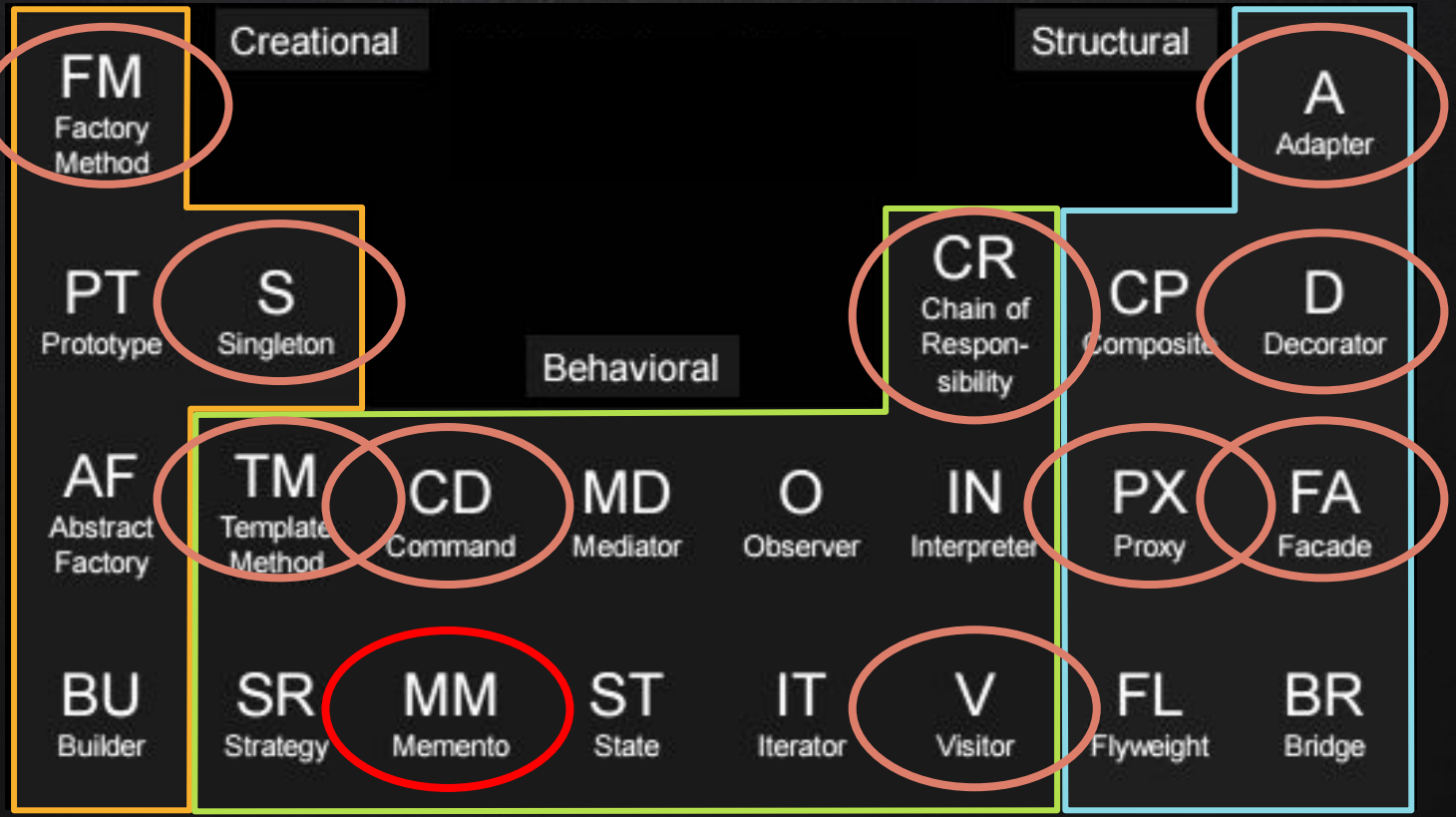
- Permite adicionar operações a uma estrutura composta sem alterar a estrutura.
- Adicionar novas operações é relativamente fácil.
- O código para operações realizadas pelo *Visitor* é centralizado

Desvantagens:

- A encapsulação das classes compostas é quebrada quando o *Visitor* é usado.
- Como é necessário percorrer a estrutura com uma função de travessia, alterações à estrutura composta são mais difíceis


All these Composite classes have to do is add a getState() method (and not worry about exposing themselves).
The Visitor needs to be able to call getState() across classes, and this is where you can add new methods for the client to use.
The Client asks the Visitor to get information from the Composite structure... New methods can be added to the Visitor without affecting the Composite.
getHealthRating()
getCalories()
getProtein()
getCarbs()
Visitor
getState()
getState()
getState()
getState()
getState()
Menu
MenuItem
MenuItem
Client / Traverser
Ingredient
Ingredient
The Traverser knows how to guide the Visitor through the Composite structure.

# Tipos de padrões



# Padrão Memento (Behavioural)




Quando é necessário que um objecto volte a um estado anterior; por exemplo, se o utilizador fizer um "undo".

Benefícios:

- Guardar o estado fora do objecto ajuda a manter a coesão.
- Mantém os dados do objecto encapsulados.
- A recuperação é fácil de implementar

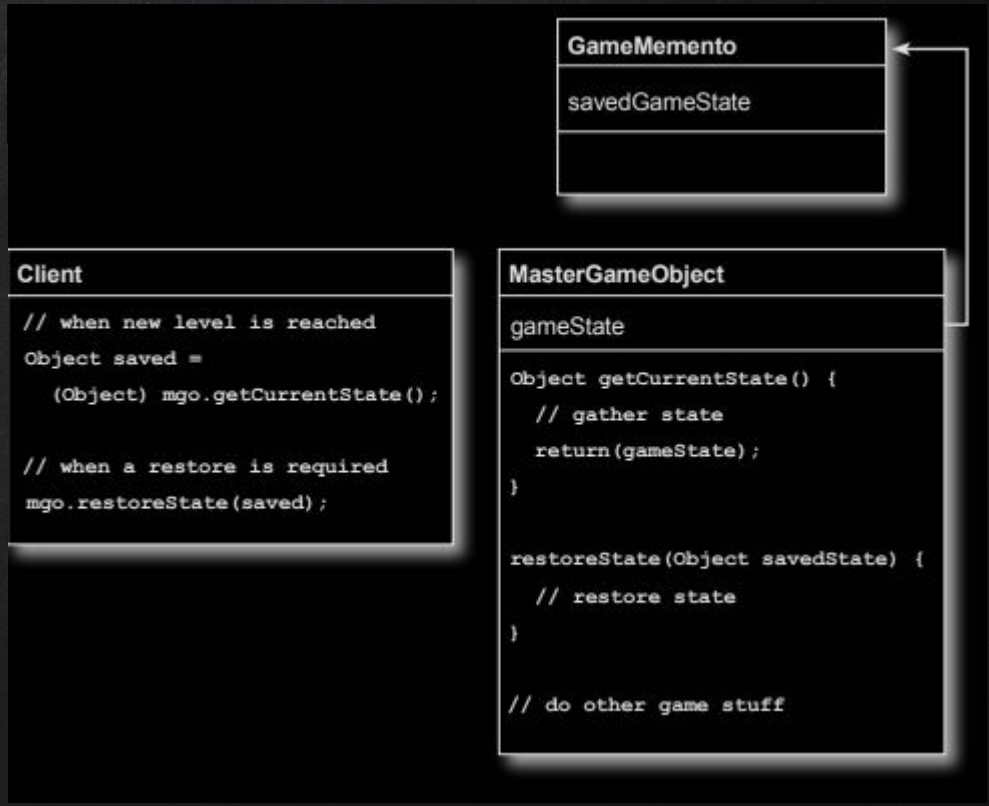
Desvantagens:

- O *Memento* é utilizado para guardar estado.
  - Guardar e restaurar o estado pode ser demorado.
  - Em Java, considere usar serialização para guardar o estado de um sistema.

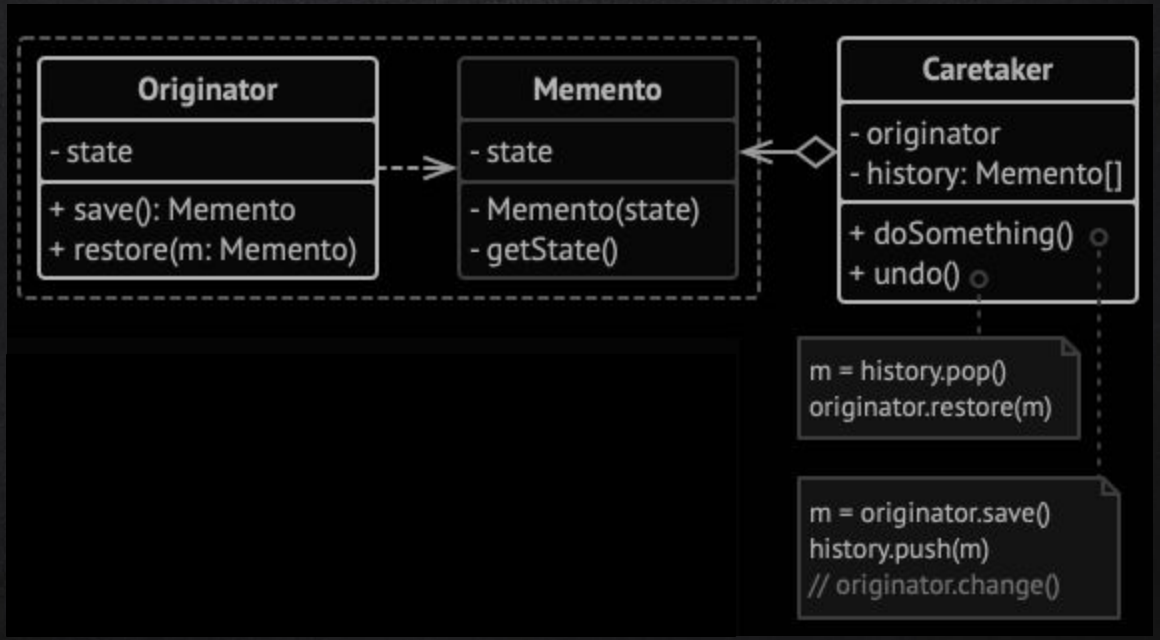
# Exemplo: RPG (Role-Playing Game)

Neste exemplo, existe o comando "guardar progresso", permite aos jogadores guardar o progresso do jogo e, pelo menos, recuperar a maior parte dos seus esforços quando a sua personagem é injustamente eliminada.

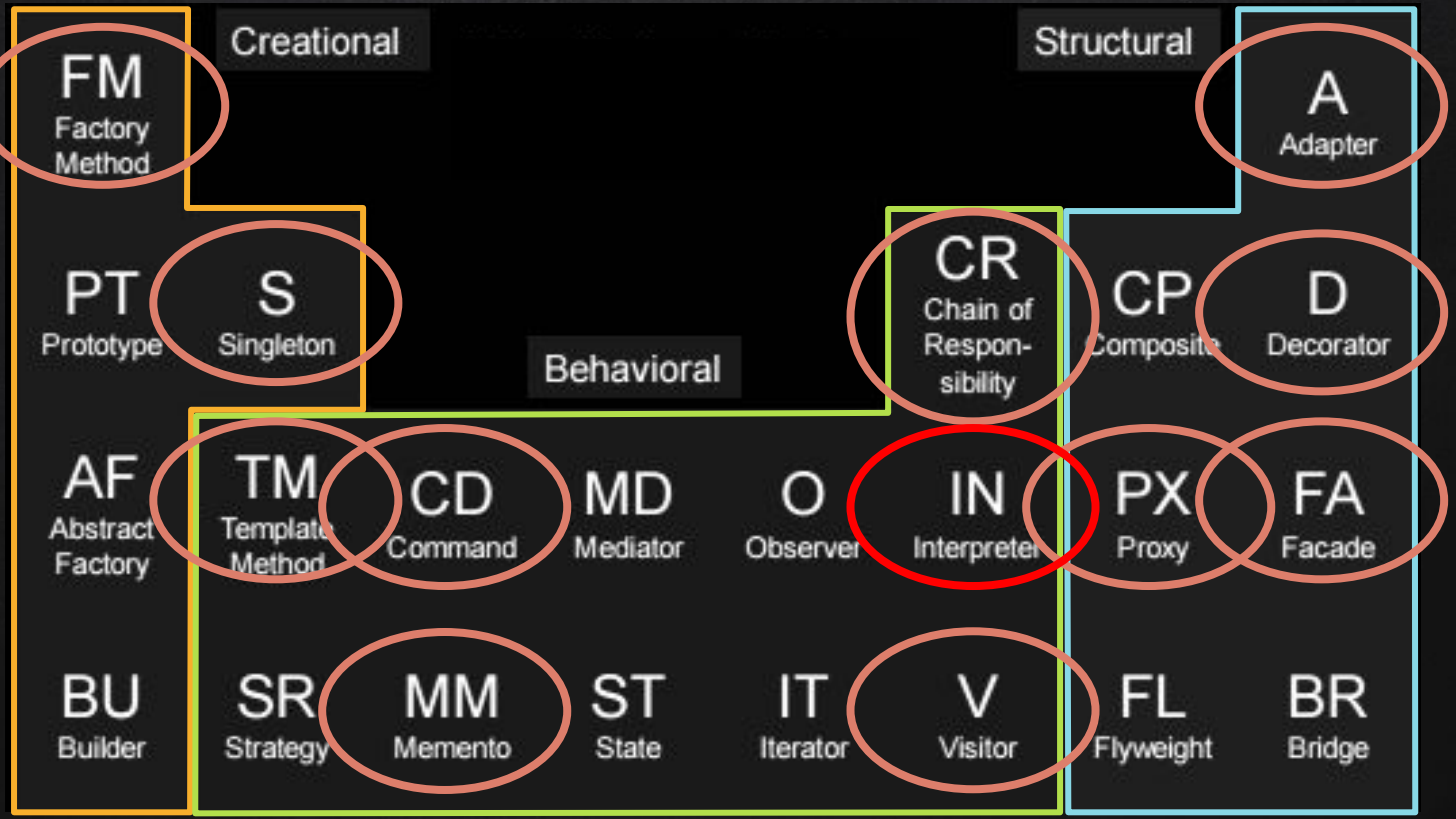
A função "guardar progresso" deve ser desenvolvida de forma a devolver o jogador ao último nível que completou com sucesso.



# Memento


Originator
- state
+ save(): Memento
+ restore(m: Memento)
Memento
- state
- Memento(state)
- getState()
Caretaker
- originator
- history: Memento[]
+ doSomething()
+ undo()
m = history.pop()
originator.restore(m)
m = originator.save()
history.push(m)
// originator.change()

# Tipos de padrões



# Padrão Interpreter (Behavioural)



Para construir um interpretador para uma linguagem.

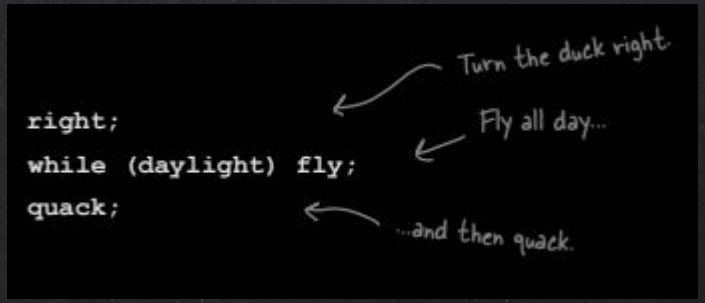
Benefícios:

- Representar cada regra gramatical numa classe facilita a implementação da linguagem. Como a gramática é representada por classes, é possível alterar ou ampliar facilmente uma linguagem.
- Ao adicionar métodos à estrutura da classe, é possível adicionar novos comportamentos para além da interpretação, como *pretty printing* e validação mais sofisticada.

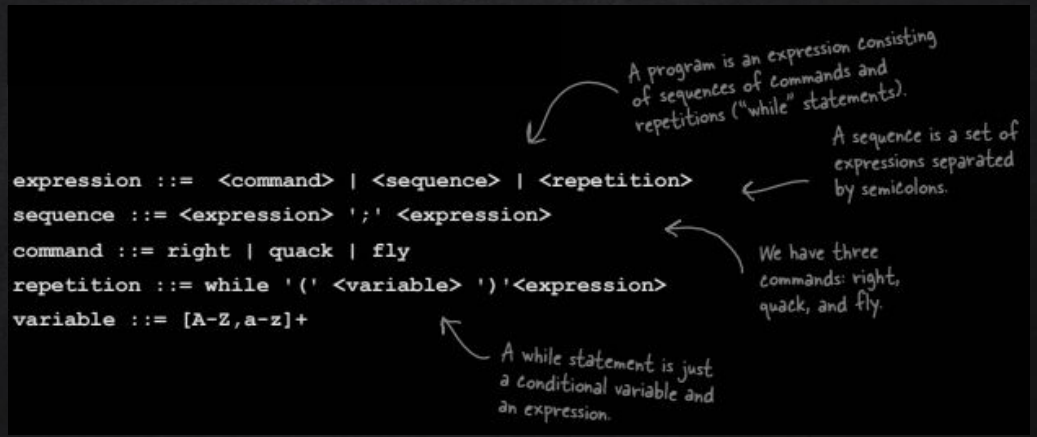
Desvantagens:

- Utiliza-se para implementar uma linguagem simples.
- Adequado quando se tem uma gramática simples e a simplicidade é mais importante do que a eficiência.
- Usado para linguagens de *scripting* e programação.
- Este padrão pode tornar-se complicado quando o número de regras gramaticais é grande. Nestes caso, um gerador de *parsers*/*compilers* pode ser mais apropriado.

# Exemplo da linguagem *Duck Simulator*




Exemplo

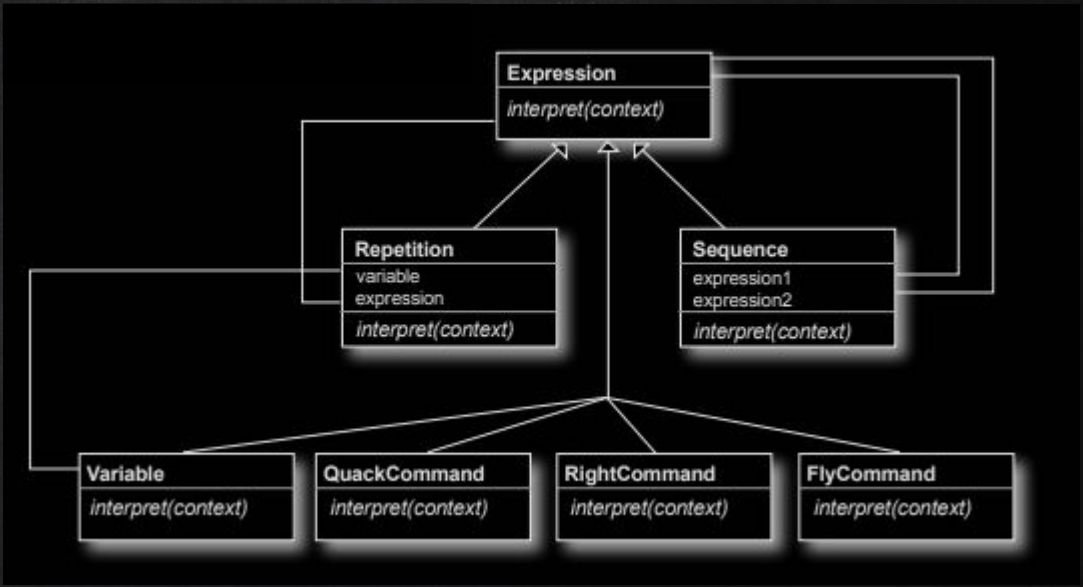



Gramática

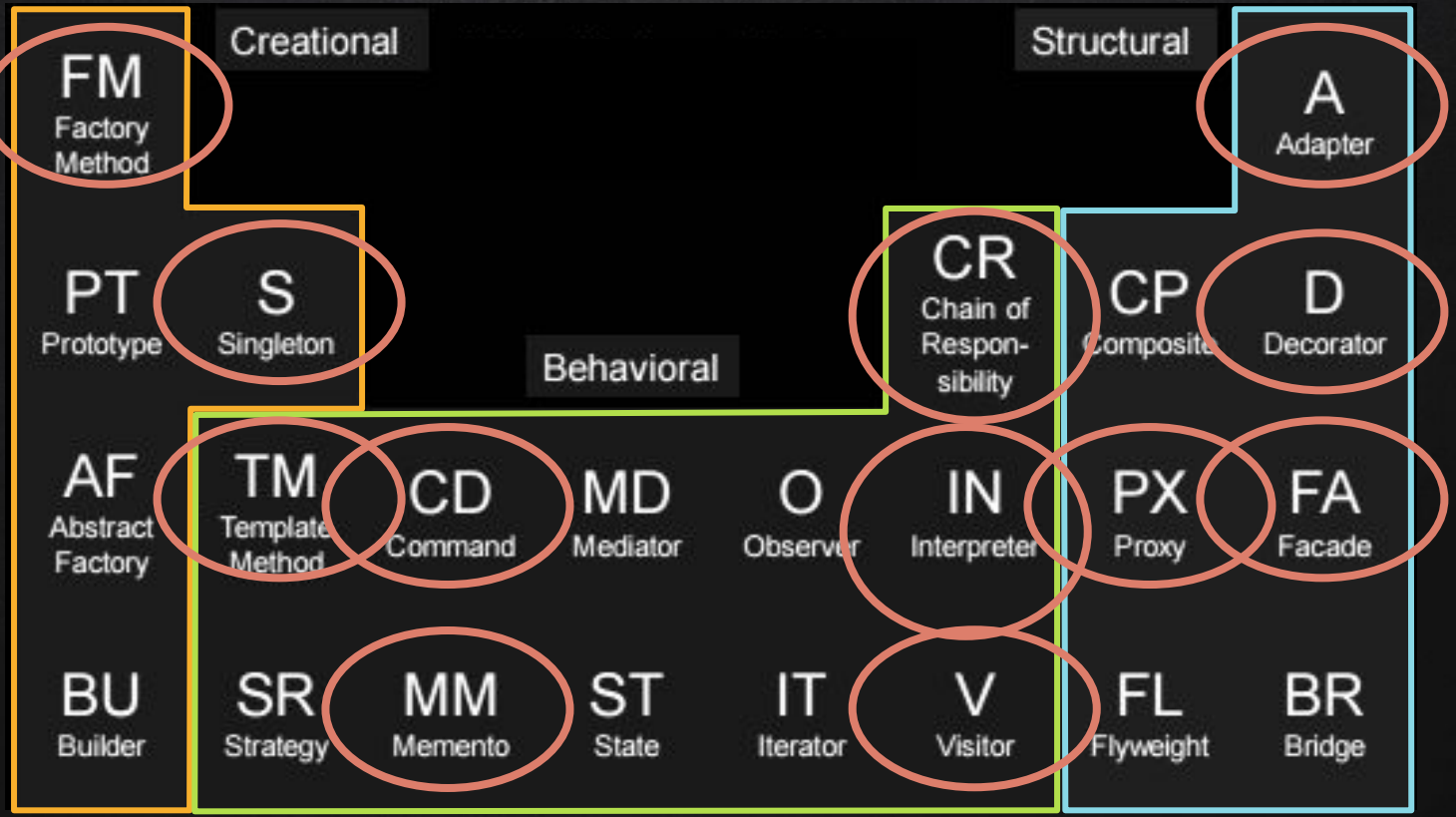
# Exemplo da linguagem *Duck Simulator*

Para interpretar a linguagem, invoca-se o método `interpret()` em cada tipo de expressão.

Este método recebe um contexto – que contém o *input stream* do programa a analisar – faz *match* do *input* e avalia a expressão.



# Tipos de padrões

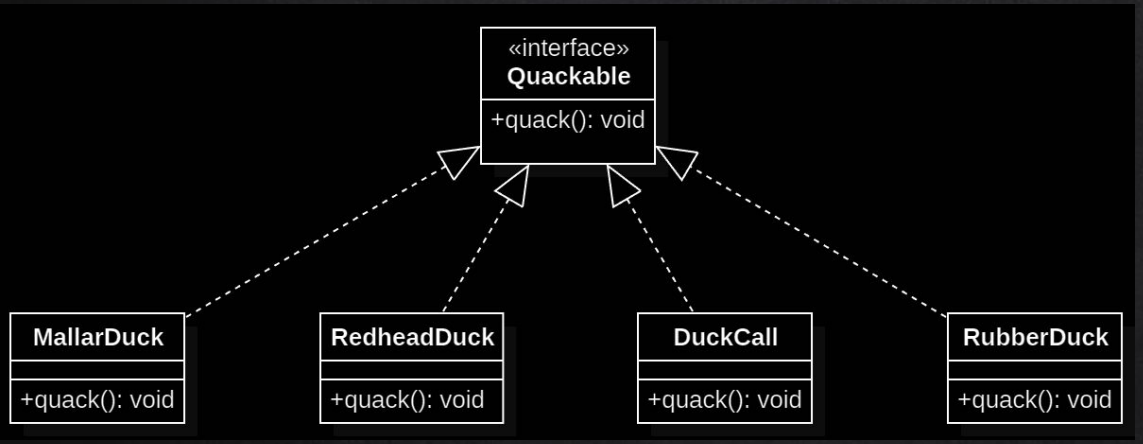


# Agora, alguns exemplos...

# Comecemos pela noção de padrões compostos

Um padrão composto combina dois ou mais padrões numa solução que resolve um problema recorrente.

# Vamos voltar ao exemplo dos patos que podem fazer Quack-Quack!




```
public interface Quackable {
   public void quack();
}

public class MallardDuck implements Quackable {
   public void quack() { System.out.println("Quack");}
}

public class RedheadDuck implements Quackable {
   public void quack() { System.out.println("Quack");}
}

public class DuckCall implements Quackable {
   public void quack() { System.out.println("Kwak");}
}

public class RubberDuck implements Quackable {
   public void quack() { System.out.println("Squeak");}
}
```

# Vamos construir um simulador de Patos

```
public class DuckSimulator {
	public static void main(String[] args) {
		DuckSimulator simulator = new DuckSimulator();
		simulator.simulate();
	 }

	 void simulate() {
		Quackable mallardDuck = new MallardDuck();
		Quackable redheadDuck = new RedheadDuck();
		Quackable duckCall = new DuckCall();
		Quackable rubberDuck = new RubberDuck();

		System.out.println("\nDuck Simulator");

		simulate(mallardDuck);
		simulate(redheadDuck);
		simulate(duckCall);
		simulate(rubberDuck);
	}

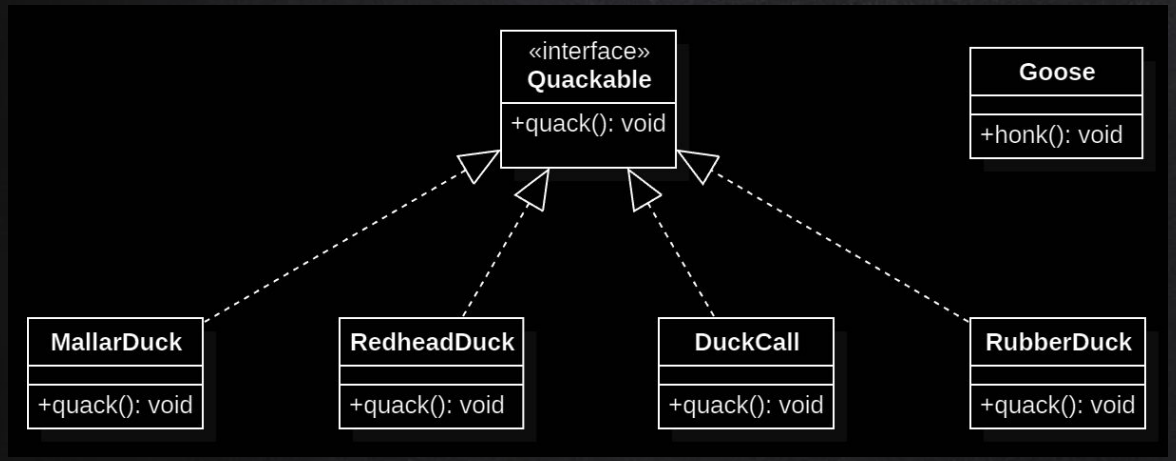
	 void simulate(Quackable duck) {
		duck.quack();
	}
}
```

Polimorfismo



```
File  Edit  Window  Help  ItBetterGetBetterThanThis
% java DuckSimulator
Duck Simulator
Quack
Quack
Kwak
Squeak
```

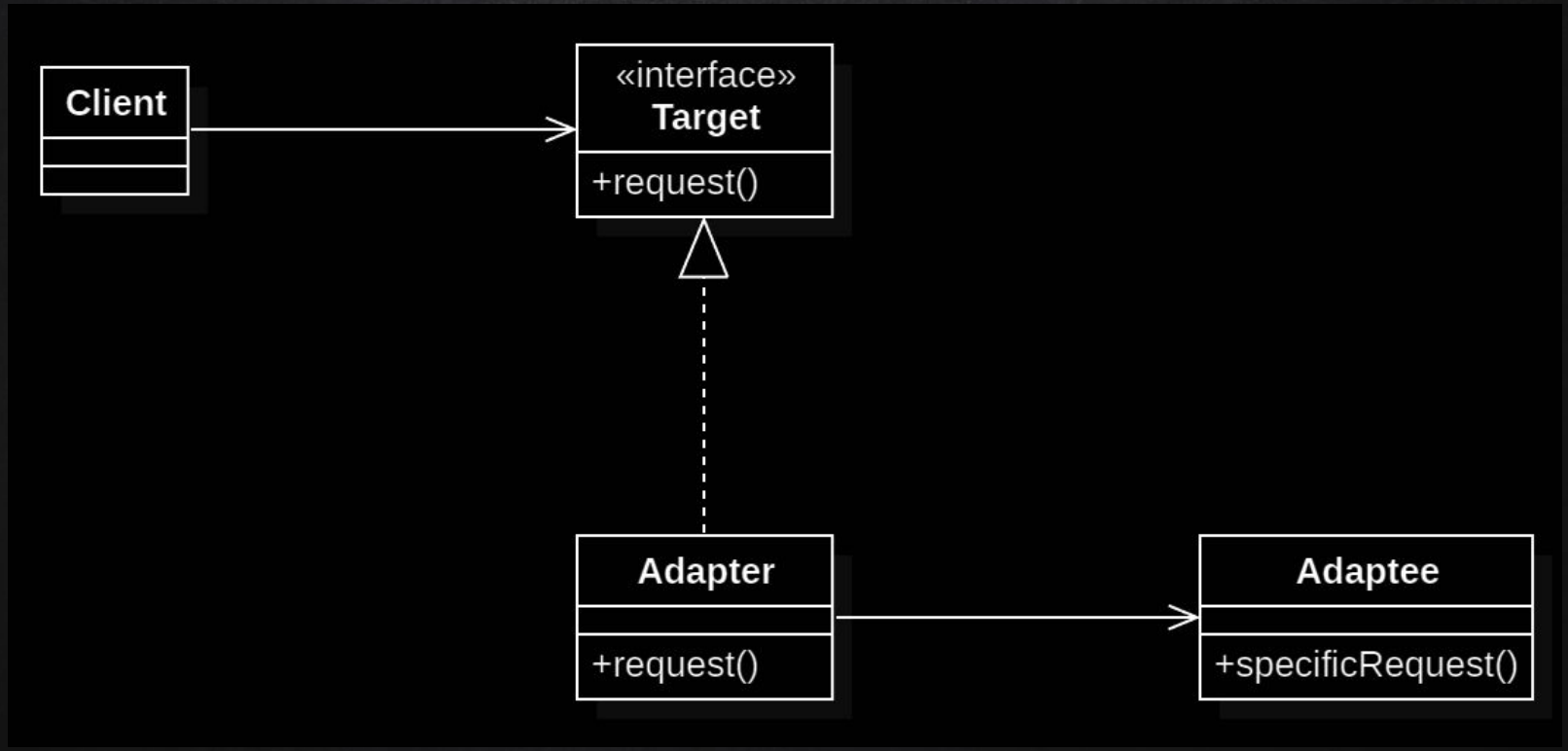
# E Gansos?



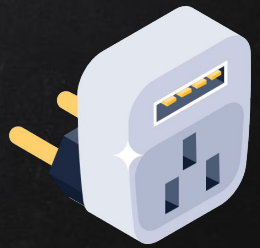

```
public class Goose {
   public void honk() {
      System.out.println("Honk");
   }
}
```

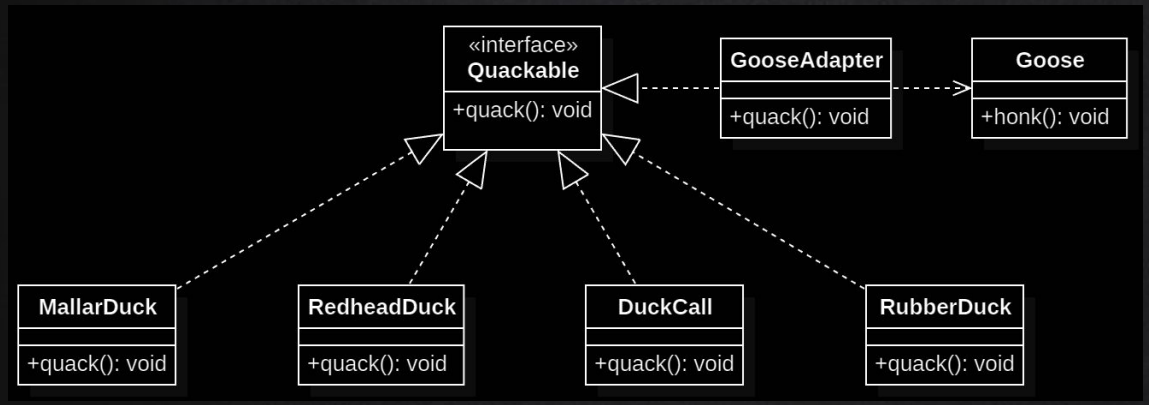
Como se adicionam Gansos ao simulador?

# Padrão Adapter



# Vamos adicionar um adaptador
# Padrão Adapter






```
public class GooseAdapter implements Quackable {
   Goose goose;

   public GooseAdapter(Goose goose) {
      this.goose = goose;
   }

   public void quack() {
      goose.honk();
   }
}
```

# Reconstrução do simulador de Patos

```
public class DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulator = new DuckSimulator();
      simulator.simulate();
   }

   void simulate() {
      Quackable mallardDuck = new MallardDuck();
      Quackable redheadDuck = new RedheadDuck();
      Quackable duckCall = new DuckCall();
      Quackable rubberDuck = new RubberDuck();
      Quackable gooseDuck = new GooseAdapter(new Goose())

      System.out.println("\nDuck Simulator");

      simulate(mallardDuck);
      simulate(redheadDuck);
      simulate(duckCall);
      simulate(rubberDuck);
      simulate(gooseDuck);
   }

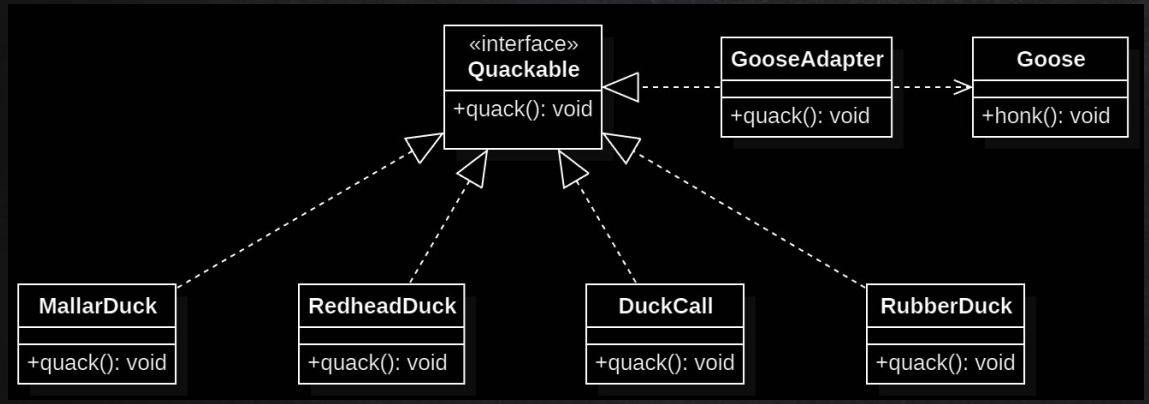
   void simulate(Quackable duck) {
      duck.quack();
   }
}
```



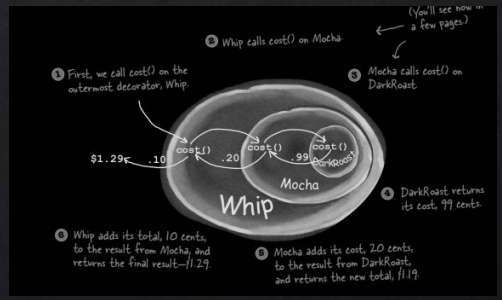
Alterações mínimas!

```
File Edit Window Help GoldenEggs
% java DuckSimulator
Duck Simulator: With Goose Adapter
Quack
Quack
Kwak
Squeak
Honk
```

# Adicionar contagens de Quack-Quacks sem alterar as classes - Padrão Decorator



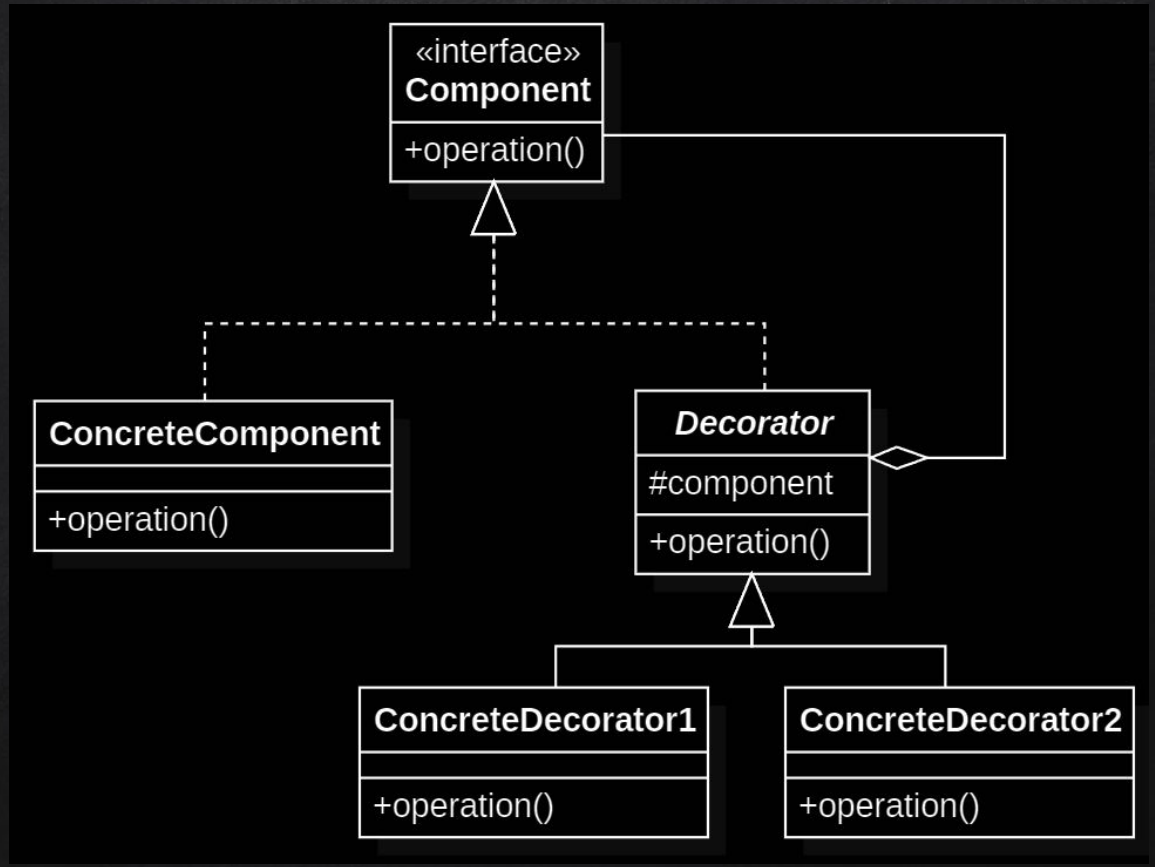



Lembram-se?

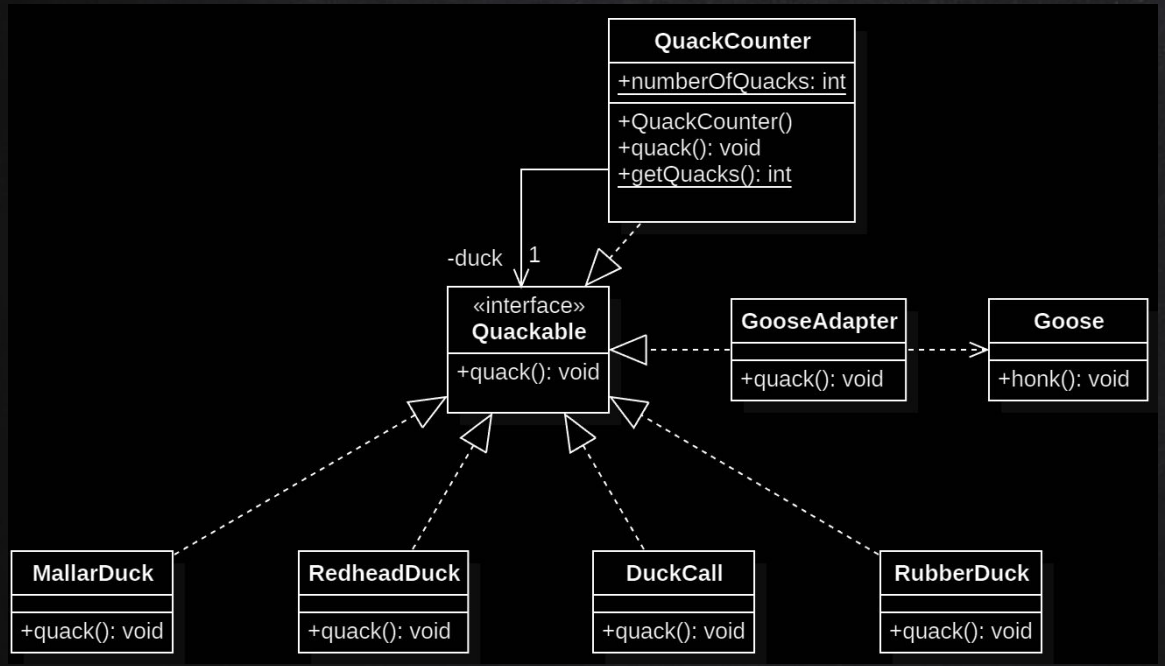


# Padrão Decorator



# Adicionar contagens de Quack-Quacks sem alterar as classes - Padrão Decorator






```
public class QuackCounter implements Quackable {

   private Quackable duck;
   public static int numberOfQuacks;

   public QuackCounter (Quackable duck) {
      this.duck = duck;
   }

   public void quack() {
      duck.quack();
      numberOfQuacks++;
   }

   public static int getQuacks() {
      return numberOfQuacks;
   }
}
```

# Reconstrução do simulador de patos

```
public class DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulator = new DuckSimulator();
      simulator.simulate();
   }
   void simulate() {
      Quackable mallardDuck = new QuackCounter(new MallardDuck());
      Quackable redheadDuck = new QuackCounter(new RedheadDuck());
      Quackable duckCall = new QuackCounter(new DuckCall());
      Quackable rubberDuck = new QuackCounter(new RubberDuck());
      Quackable gooseDuck = new GooseAdapter(new Goose());
      System.out.println("\nDuck Simulator");
      simulate(mallardDuck);
      simulate(redheadDuck);
      simulate(duckCall);
      simulate(rubberDuck);
      simulate(gooseDuck);
      System.out.println("The ducks quacked "
                         + QuackCounter.getQuacks()
                         + " times");
   }
   void simulate(Quackable duck) {
      duck.quack();
   }
}
```



```
File Edit Window Help DecoratedEggs
% java DuckSimulator
Duck Simulator: With Decorator
Quack
Quack
Kwak
Squeak
Honk
The ducks quacked 4 times
%
```

# Reconstrução do simulador de patos

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }
    void simulate() {
        Quackable mallardDuck = new QuackCounter(new MallardDuck());
        Quackab
        Quackab
        Quackab
        Quackab
        System.
        simulat
        simulat
        simulat
        simulat
        simulat
        System.
                            + QuackCounter.getQuacks()
                            + " times");
    }
    void simulate(Quackable duck) {
        duck.quack();
    }
}
```



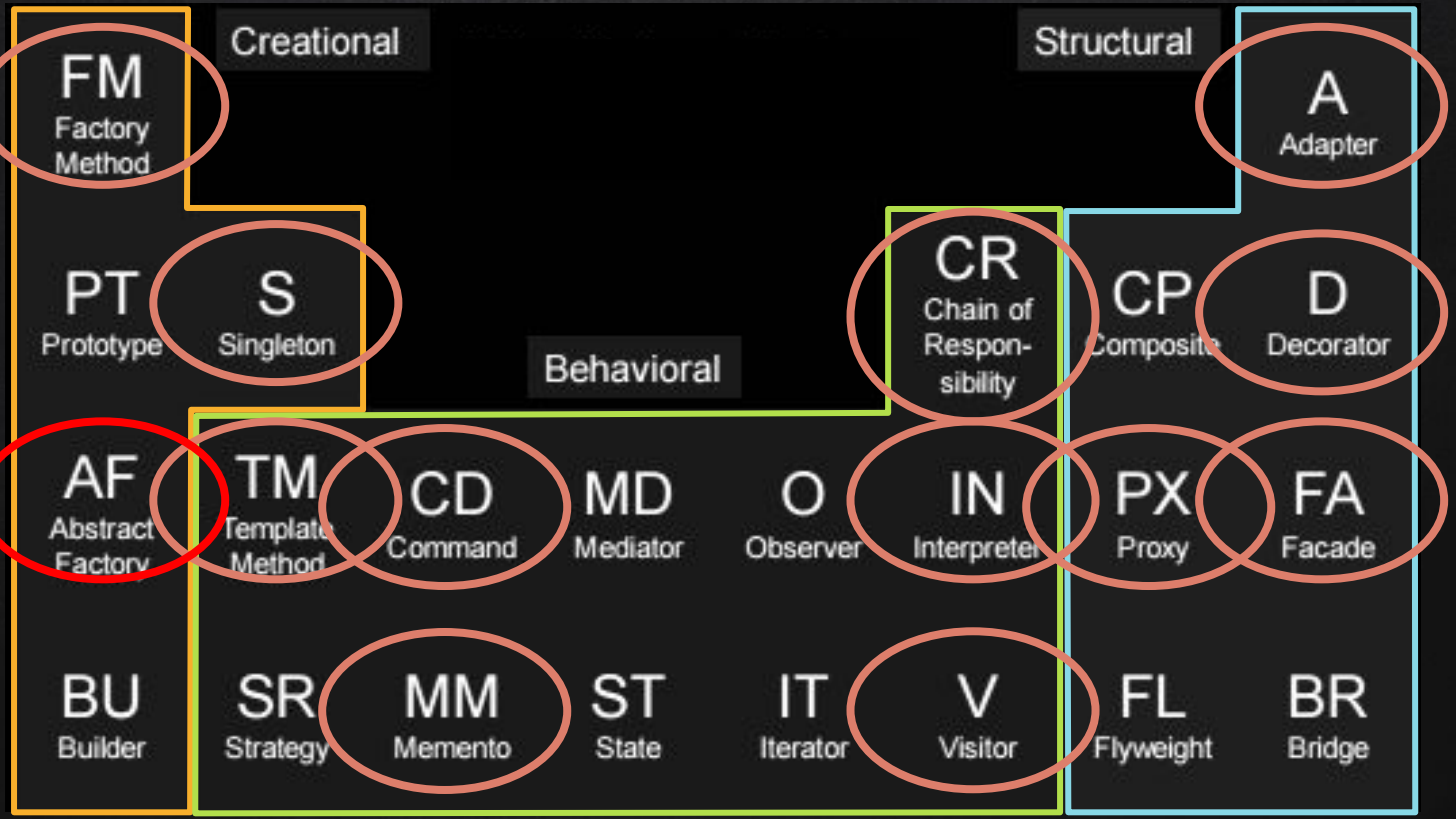
Óptima solução, mas...

Temos de ter cuidado e garantir que todos os objectos são envolvidos, caso contrário, podemos perder alguns *quacks* na contagem!

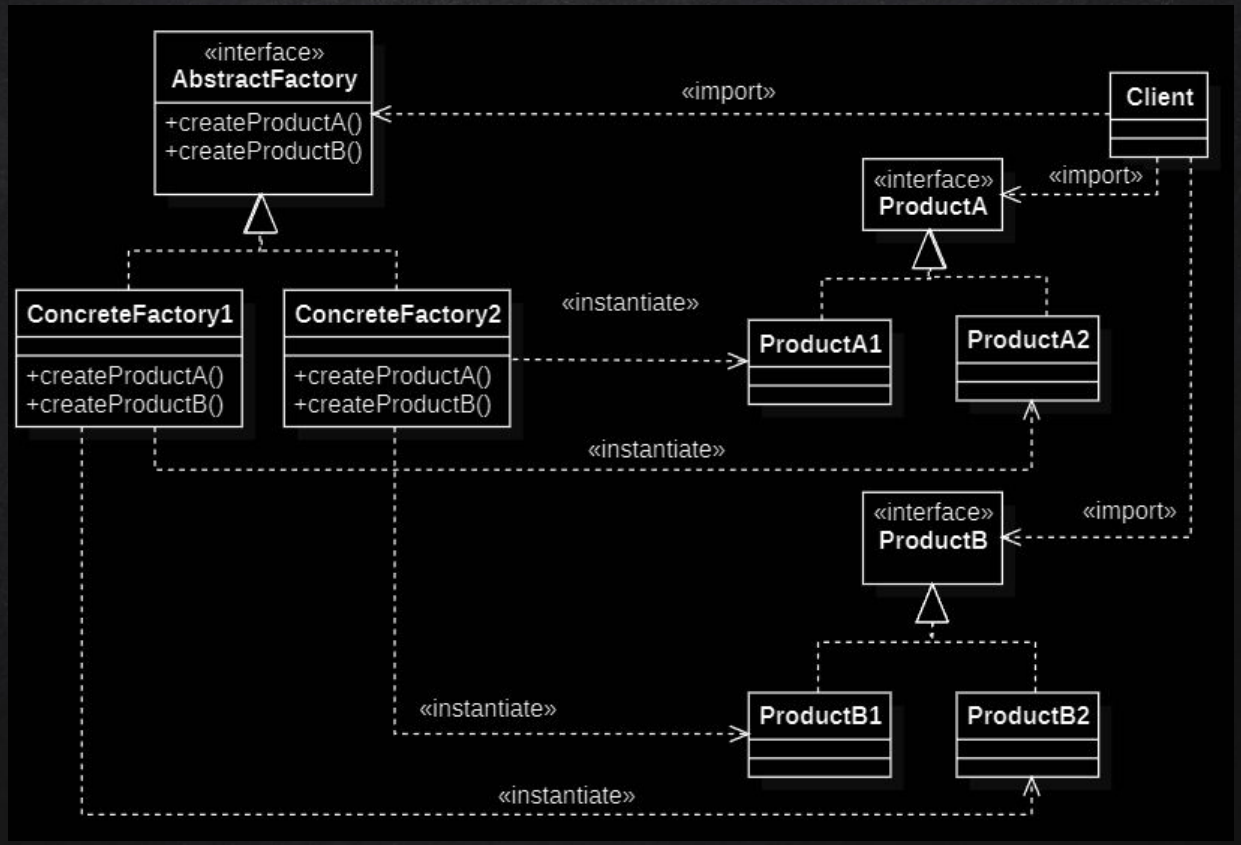
Precisamos de uma fábrica para produzir diferentes tipos de patos. Por exemplo, patos contadores e patos comuns.

```
The ducks quacked 4 times
%
```

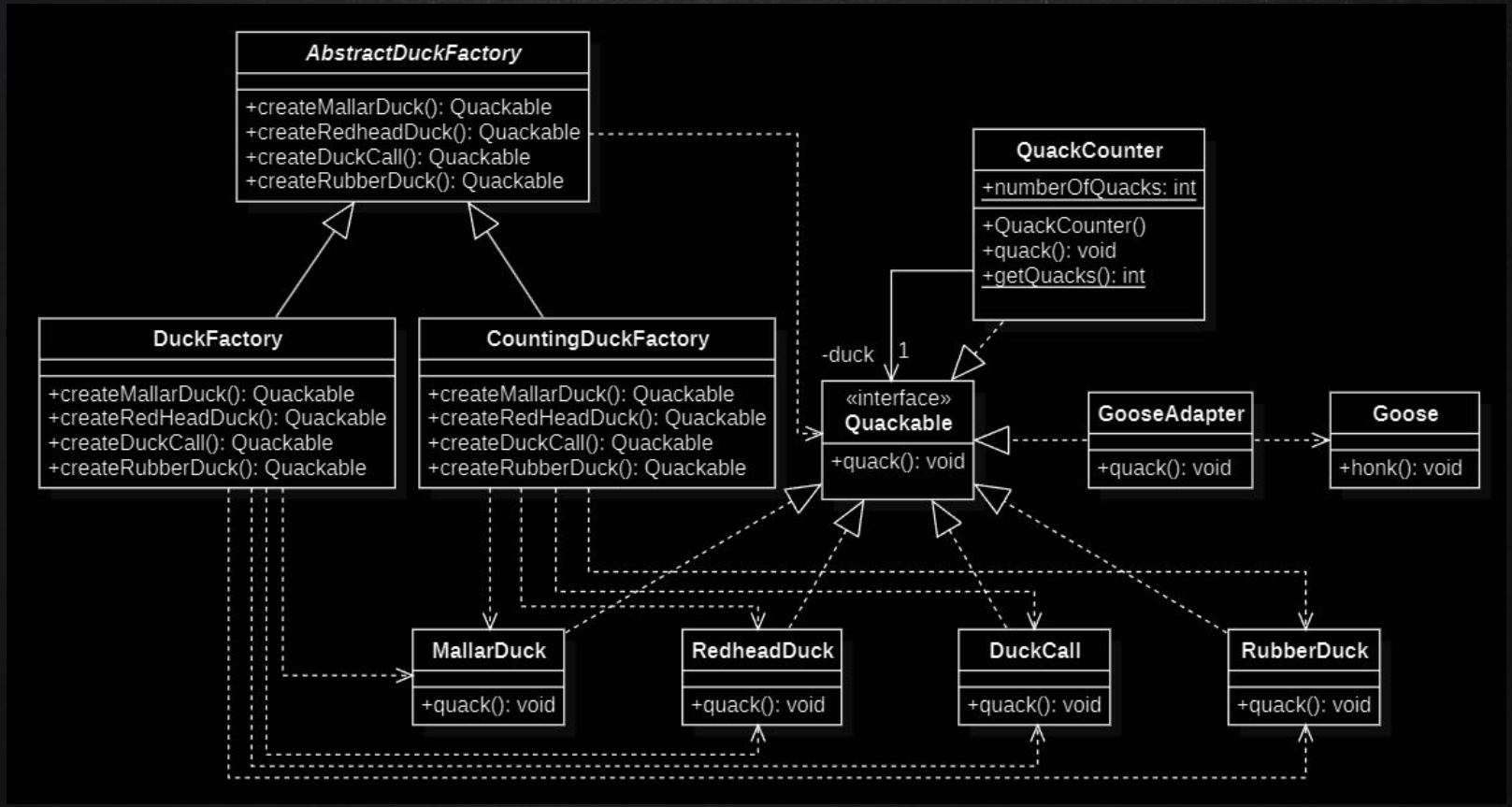
# Tipos de padrões



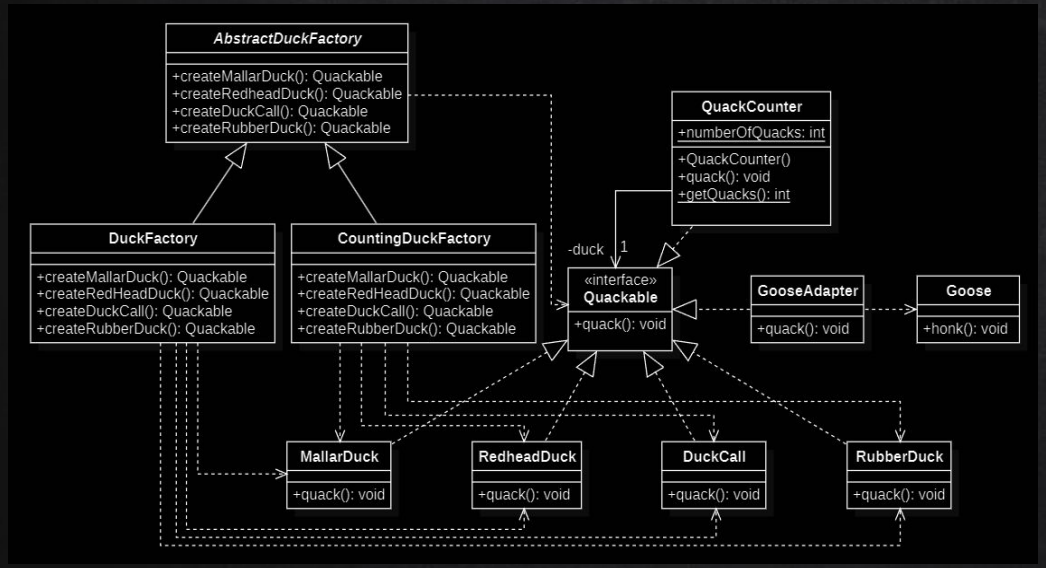
# Padrão Abstract Factory (Creational)



# Agora com patos e gansos...



# Adicionar uma fábrica de patos abstracta e duas fábricas concretas




```
public abstract class AbstractDuckFactory {
   public abstract Quackable createMallardDuck();
   public abstract Quackable createRedheadDuck();
   public abstract Quackable createDuckCall();
   public abstract Quackable createRubberDuck();
}
public class DuckFactory extends AbstractDuckFactory {
   public Quackable createMallardDuck() {
      return new MallardDuck(); }
   public Quackable createRedheadDuck() {
      return new RedheadDuck(); }
   public Quackable createDuckCall() {
      return new DuckCall(); }
   public Quackable createRubberDuck() {
         return new RubberDuck();
   }
}
public class CountingDuckFactory extends AbstractDuckFactory{
   public Quackable createMallardDuck() {
      return new QuackCounter(new MallardDuck());}
   public Quackable createRedheadDuck() {
      return new QuackCounter(new RedheadDuck());}
   public Quackable createDuckCall() {
      return new QuackCounter(new DuckCall());}
   public Quackable createRubberDuck() {
      return new QuackCounter(new RubberDuck());}
}
```

# E NOVAMENTE O SIMULADOR DE PATOS

```
public class DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulator = new DuckSimulator();
      AbstractDuckFactory duckFactory = new CountingDuckFactory();
      simulator.simulate(duckFactory);
   }

   void simulate(AbstractDuckFactory duckFactory) {
      Quackable mallardDuck =  duckFactory.createMallardDuck();
      Quackable redheadDuck = duckFactory.createRedheadDuck();
      Quackable duckCall = duckFactory.createDuckCall();
      Quackable rubberDuck = duckFactory.createRubberDuck();
      Quackable gooseDuck = new GooseAdapter(new Goose());

      System.out.println("\nDuck Simulator");

      simulate(mallardDuck);
      simulate(redheadDuck);
      simulate(duckCall);
      simulate(rubberDuck);
      simulate(gooseDuck);

      System.out.println("The ducks quacked " + QuackCounter.getQuacks()
                         + " times");
   }
   void simulate(Quackable duck) {
      duck.quack();
   }
}
```



```
File Edit Window Help EggFactory
% java DuckSimulator
Duck Simulator: With Abstract Factory
Quack
Quack
Kwak
Squeak
Honk
4 quacks were counted
%
```

# E NOVAMENTE O SIMULADOR DE PATOS

```
classe pública DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulador = novo DuckSimulator();
      AbstractDuckFactory duckFactory = new CountingDuckFactory();
      simulator.simulate(duckFactory);
   }

   void simulate(AbstractDuckFactory duckFactory) {
      Quackable mallardDuck =  duckFactory.createMallardDuck();
      Quackable redheadDuck = duckFactory.createRedheadDuck();
      Quackable duckCall = duckFactory.createDuckCall();
      Quackable rubberDuck = duckFactory.createRubberDuck();
      Quackable gooseDuck = new GooseAdapter(new Goose());

      System.out.println("\nSimulador de pato");

      simulate(mallardDuck);
      simulate(pato-de-cabeça-vermelha);
      simular(chamadaDePato);
      simular(pato de borracha);
      simulate(gooseDuck);

      System.out.println("Os patos grasnaram " + QuackCounter.getQuacks()
                         + " vezes");
   }
   void simular(pato que pode grasnar) {
      pato.quack();
   }
}
```



Em vez de gerir os patos individualmente, podemos usar coleções de patos?
Existe uma forma de aplicar operações a um conjunto de patos?

```
Quack
Kwak
Squeak
Honk
4 quacks were counted
%
```

# E NOVAMENTE O SIMULADOR DE PATOS

```
classe pública DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulador = novo DuckSimulator();
      AbstractDuckFactory duckFactory = new CountingDuckFactory();
      simulator.simulate(duckFactory);
   }

   void simulate(AbstractDuckFactory duckFactory) {
      Quackable mallardDuck =  duckFactory.createMallardDuck();
      Quackable redheadDuck = duckFactory.createRedheadDuck();
      Quackable duckCall = duckFactory.createDuckCall();
      Quackable rubberDuck = duckFactory.createRubberDuck();
      Quackable gooseDuck = new GooseAdapter(new Goose());

      System.out.println("\nSimulador de pato");

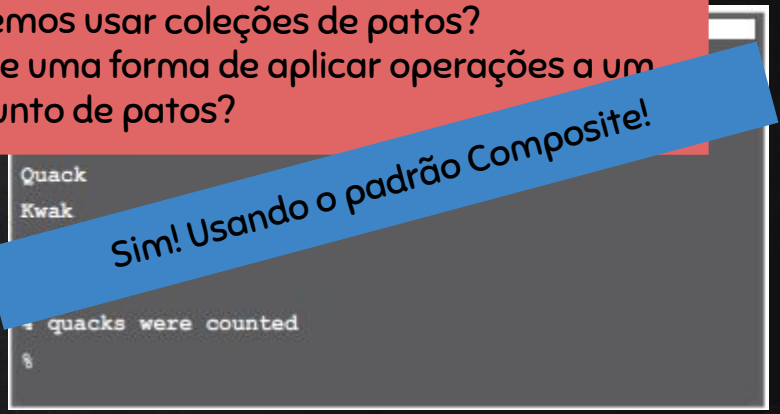
      simulate(mallardDuck);
      simulate(pato-de-cabeça-vermelha);
      simular(chamadaDePato);
      simular(pato de borracha);
      simulate(gooseDuck);

      System.out.println("Os patos grasnaram " + QuackCounter.getQuacks()
                         + " vezes");
   }
   void simular(pato que pode grasnar) {
      pato.quack();
   }
}
```

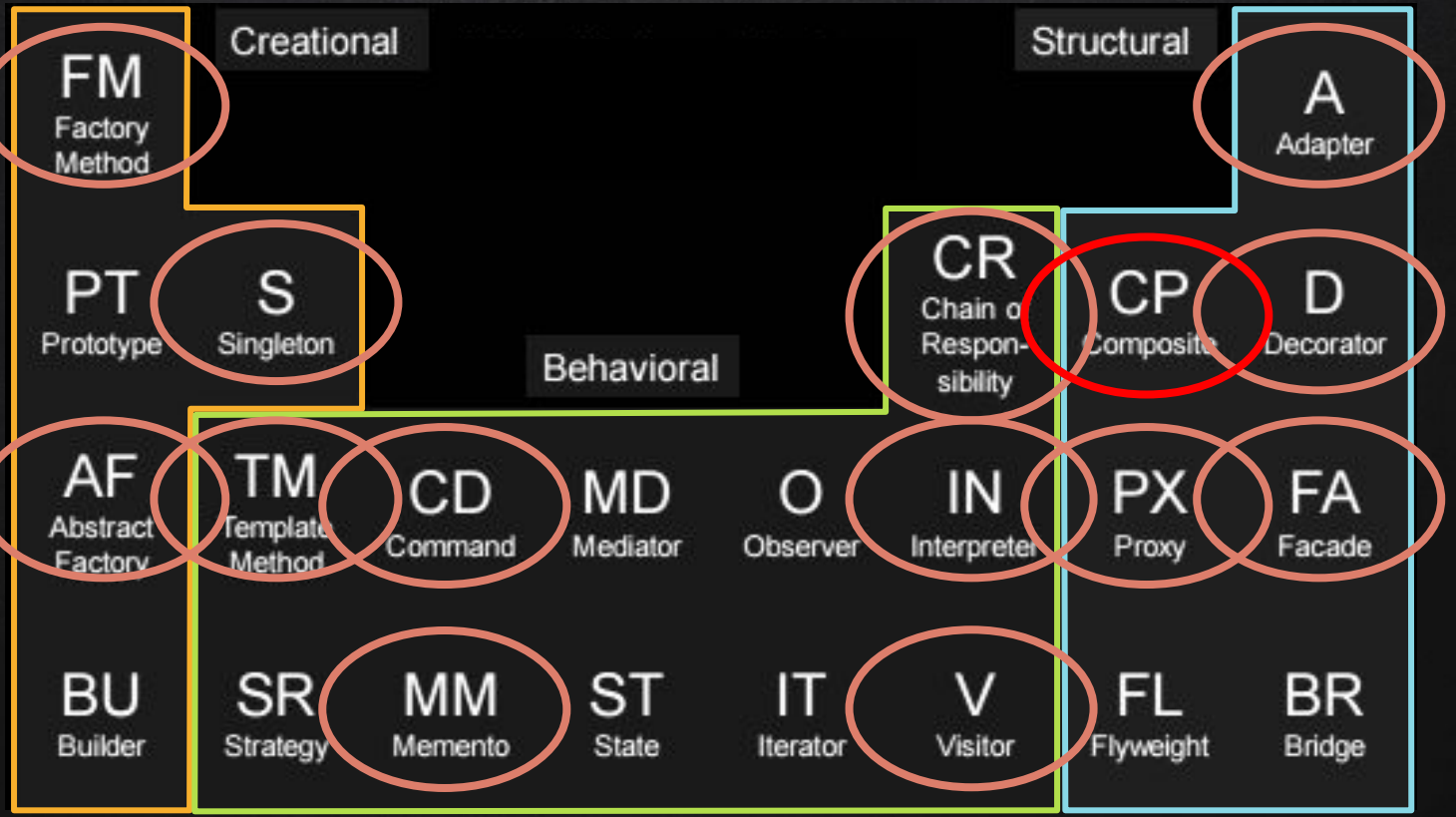


Em vez de gerir os patos individualmente, podemos usar coleções de patos?
Existe uma forma de aplicar operações a um conjunto de patos?

Sim! Usando o padrão Composite!

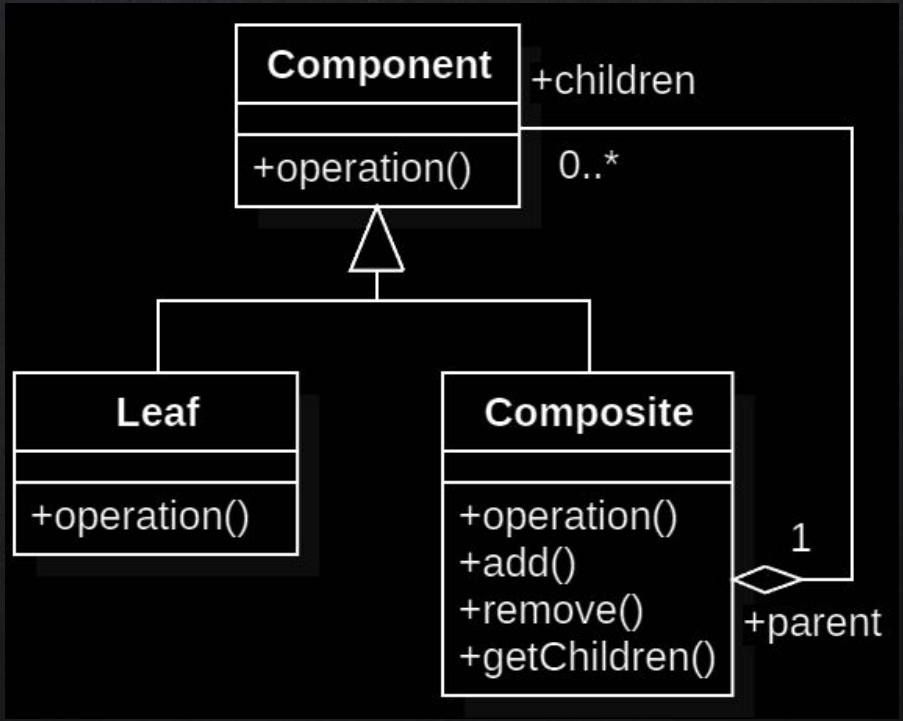


# Tipos de padrões



# Padrão Composite (Structural)

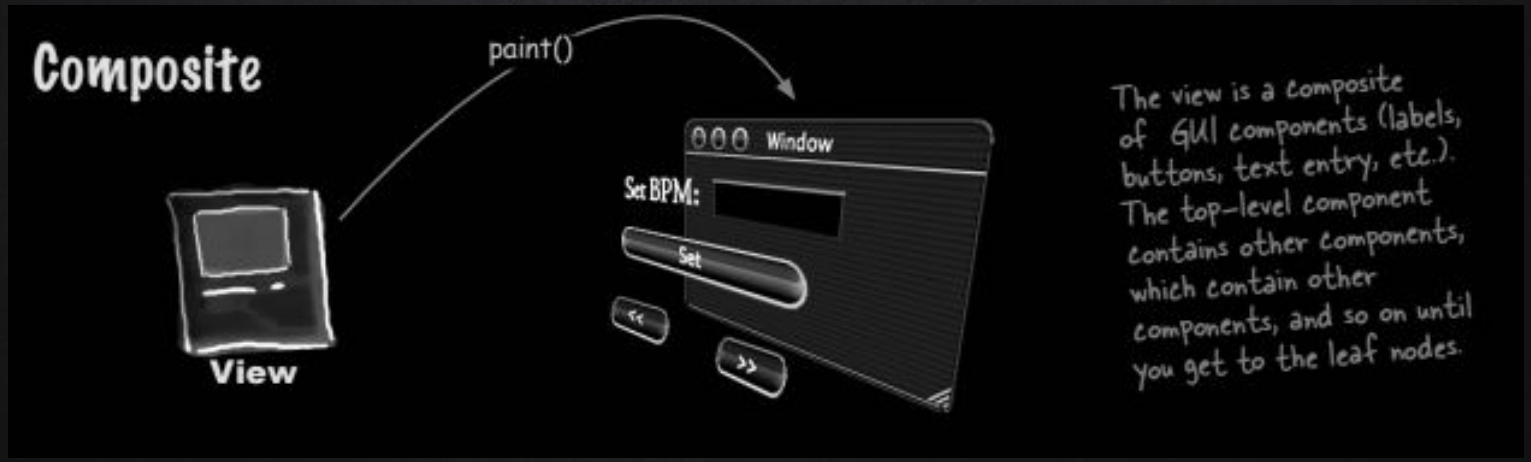
Usado quando os clientes ignoram a diferença entre composições de objetos e objetos individuais.



Um *display* consiste num conjunto aninhado de janelas, painéis, botões, *labels* de texto e assim por diante.

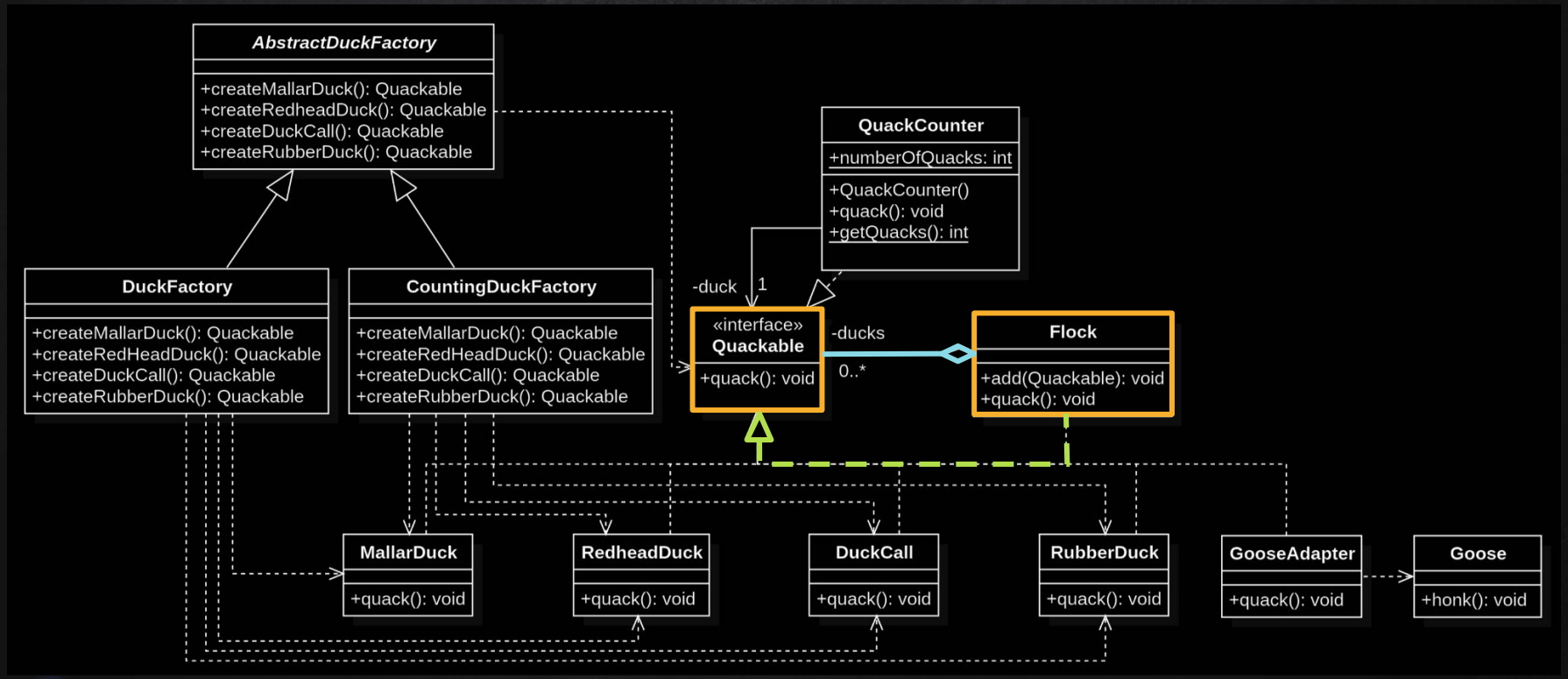
Cada componente do *display* é um *Composite* (como uma janela) ou uma folha (como um botão).

Quando o controlador diz ao *display* para actualizar, é só preciso informar o componente superior da visualização, e o *Composite* trata do resto.



# Adicionar o padrão Composite

## Flock é um Quackable e é composto por Quackables



# Implementação do padrão Composite

```
public class Flock implements Quackable {
   List<Quackable> quackers = new ArrayList<Quackable>();

   public void add(Quackable quacker) {
      quackers.add(quacker);
   }

   public void quack() {
      Iterator<Quackable> iterator = quackers.iterator();
      while (iterator.hasNext()) {
         Quackable quacker = iterator.next();
         quacker.quack();
      }
   }
}
```

Algumas notas:

- O *composite* precisa de implementar a mesma interface que os elementos folha (`Quackable`)
- Dentro de cada `Flock` existe uma `ArrayList` para guardar os `Quackables` pertencentes ao `Flock`
- O método `quack()` precisa de funcionar em todos os `Flocks`
  - Itera-se o `ArrayList` e invoca-se `quack()` em cada elemento.

# Implementação do padrão Composite

```
public class Flock implements Quackable {
   List<Quackable> quackers = new ArrayList<Quackable>();

   public void add(Quackable quacker) {
      quackers.add(quacker);
   }

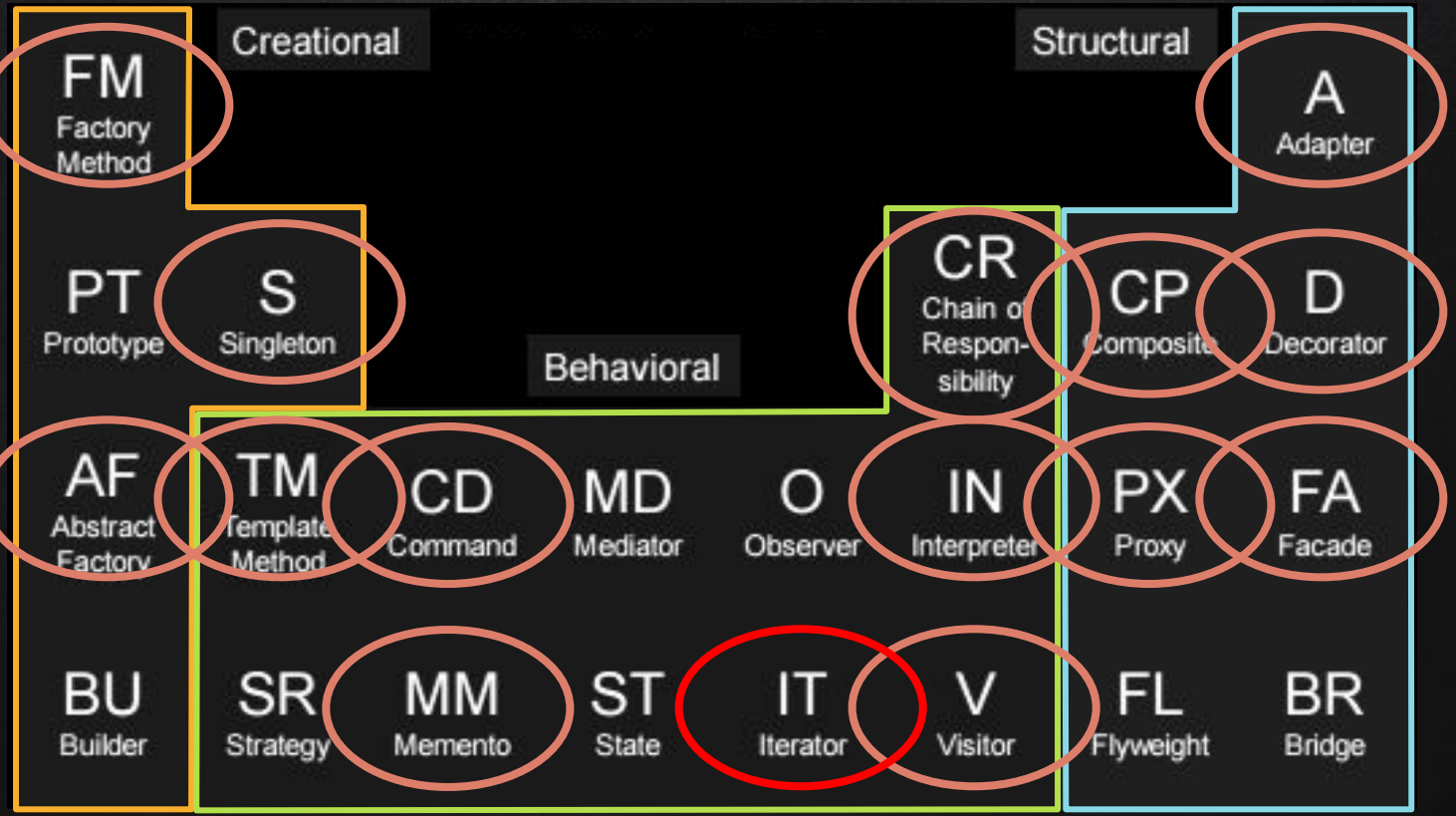
   public void quack() {
      Iterator<Quackable> iterator = quackers.iterator();
      while (iterator.hasNext()) {
         Quackable quacker = iterator.next();
         quacker.quack();
      }
   }
}
```

Algumas notas:

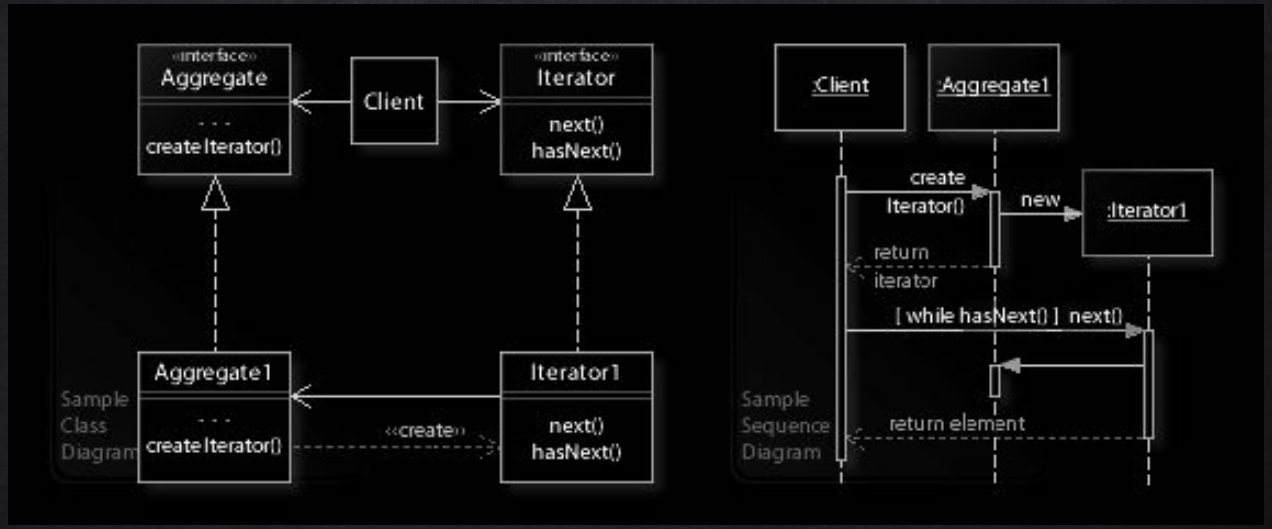
- O *composite* precisa de implementar a mesma interface que os elementos folha (`Quackables`)
- Dentro de cada `Flock` existe uma `ArrayList` para guardar os `Quackables` pertencentes ao `Flock`
- O método `quack()` precisa de funcionar em todos os `Flocks`
  - Itera-se o `ArrayList` e invoca-se `quack()` em cada elemento.

Também incluímos o padrão Iterator! :-)

# Tipos de padrões



# Padrão Iterator (Behavioural)




Oferece uma forma de percorrer uma colecção de objectos sem expôr a sua implementação.

# ... E NOVAMENTE O SIMULADOR DE PATOS

```
public class DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulator = new DuckSimulator();
      AbstractDuckFactory duckFactory = new CountingDuckFactory();
      simulator.simulate(duckFactory);
   }

   void simulate(AbstractDuckFactory duckFactory) {
      Quackable redheadDuck = duckFactory.createRedheadDuck();
      Quackable duckCall = duckFactory.createDuckCall();
      Quackable rubberDuck = duckFactory.createRubberDuck();
      Quackable gooseDuck = new GooseAdapter(new Goose());

      System.out.println("\nDuck Simulator");

      Flock flockOfDucks = new Flock();
      flockOfDucks.add(redheadDuck);
      flockOfDucks.add(duckCall);
      flockOfDucks.add(rubberDuck);
      flockOfDucks.add(gooseDuck);

      Flock flockOfMallards = new Flock();
      Quackable mallardDuck =  duckFactory.createMallardDuck();
      Quackable mallardTwo = duckFactory.createMallardDuck();
      Quackable mallardThree = duckFactory.createMallardDuck();
      Quackable mallardFour = duckFactory.createMallardDuck();

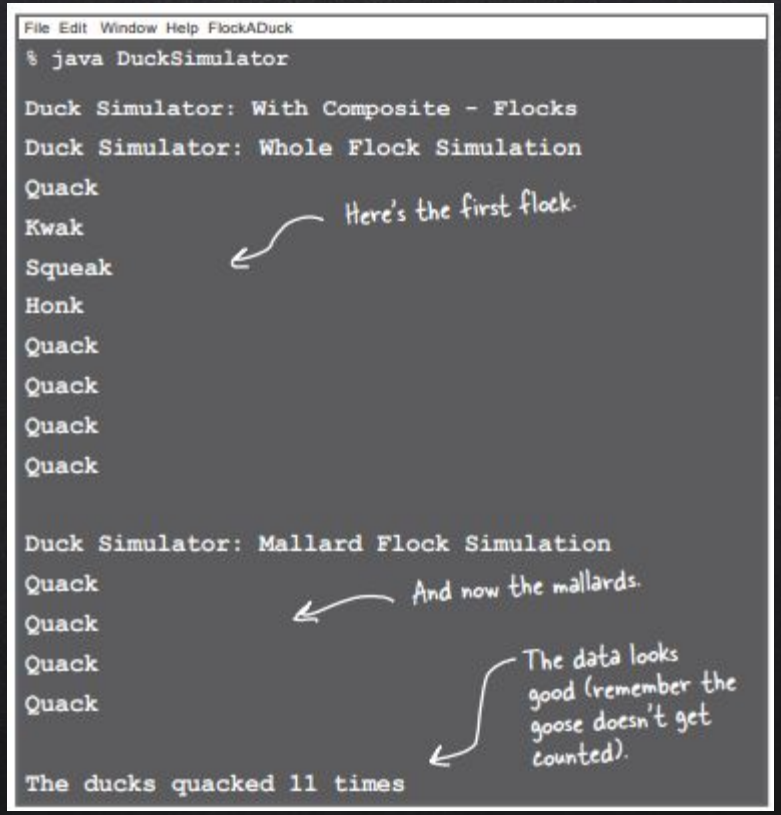
      flockOfDucks.add(flockOfMallards);
              ...
      System.out.println("\nDuck Simulator: Whole Flock Simulation");
      simulate(flockOfDucks);
      System.out.println("\nDuck Simulator: Mallard Flock Simulation");
      simulate(flockOfMallards);

      System.out.println("The ducks quacked " +
                         QuackCounter.getQuacks() + " times");
   }

   void simulate(Quackable duck) {
      duck.quack();
   }
}
```



# ... E NOVAMENTE O SIMULADOR DE PATOS






# ... E NOVAMENTE O SIMULADOR DE PATOS



```
File Edit Window Help FlockADuck
% java DuckSimulator

Duck Simulator: With Composite - Flocks
Duck Simulator: Whole Flock Simulation
Quack
Kwak
Squeak
Honk
```

Here's the first flock.

Está quase perfeito, mas agora queremos ser capazes de observar o comportamento individual dos patos.

```
Quack

Duck Simulator: Mallard Flock Simulation
Quack
Quack
Quack
Quack

The ducks quacked 11 times
```

And now the mallards.

The data looks good (remember the goose doesn't get counted).

# ... E NOVAMENTE O SIMULADOR DE PATOS



```
File Edit Window Help FlockADuck
% java DuckSimulator

Duck Simulator: With Composite - Flocks
Duck Simulator: Whole Flock Simulation
Quack
Kwak
Squeak
Honk

Duck Simulator: Mallard Flock Simulation
Quack
Quack
Quack
Quack

The ducks quacked 11 times
```

Here's the first flock.

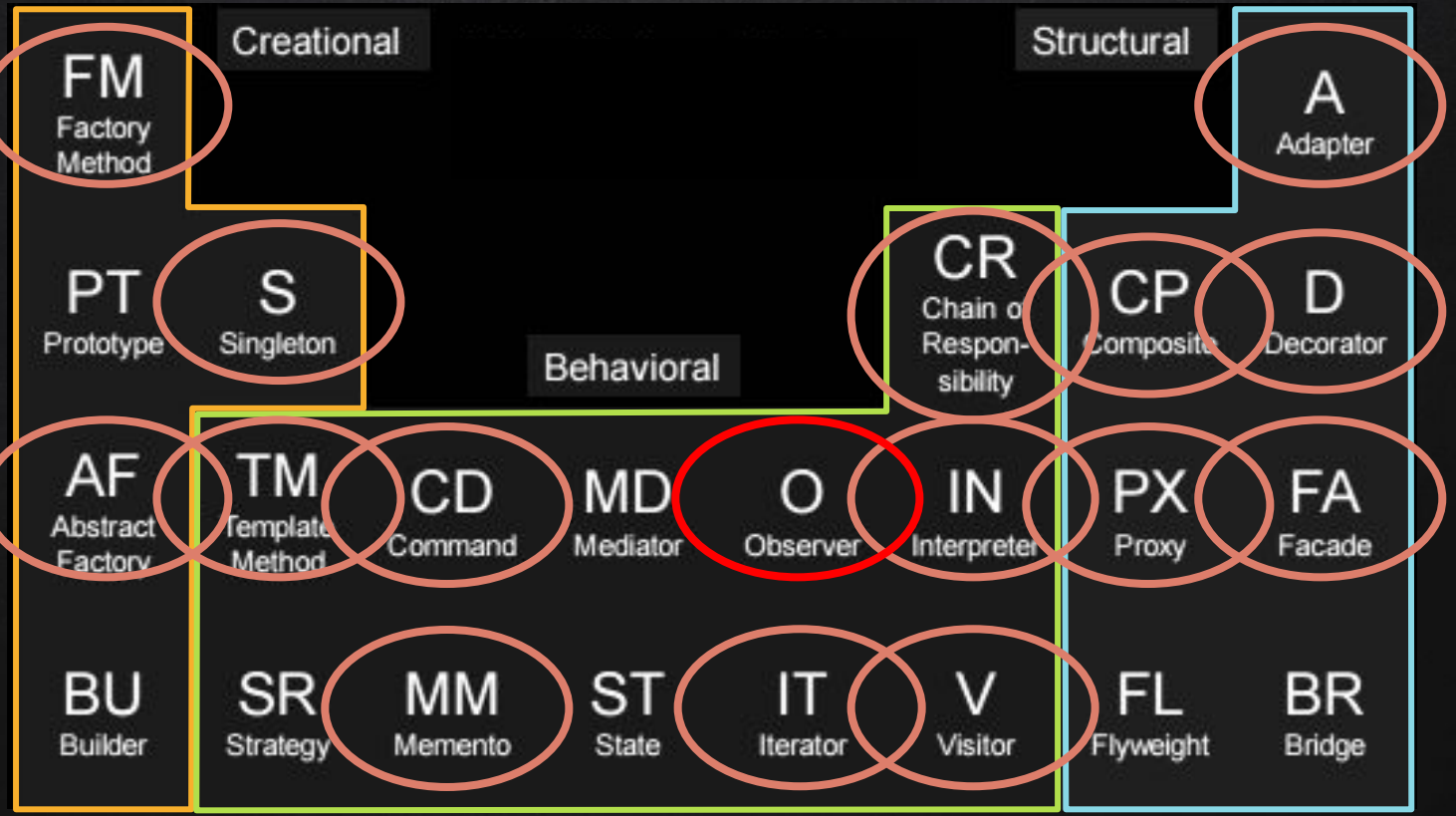
And now the mallards.

The data looks good (remember the goose doesn't get counted).

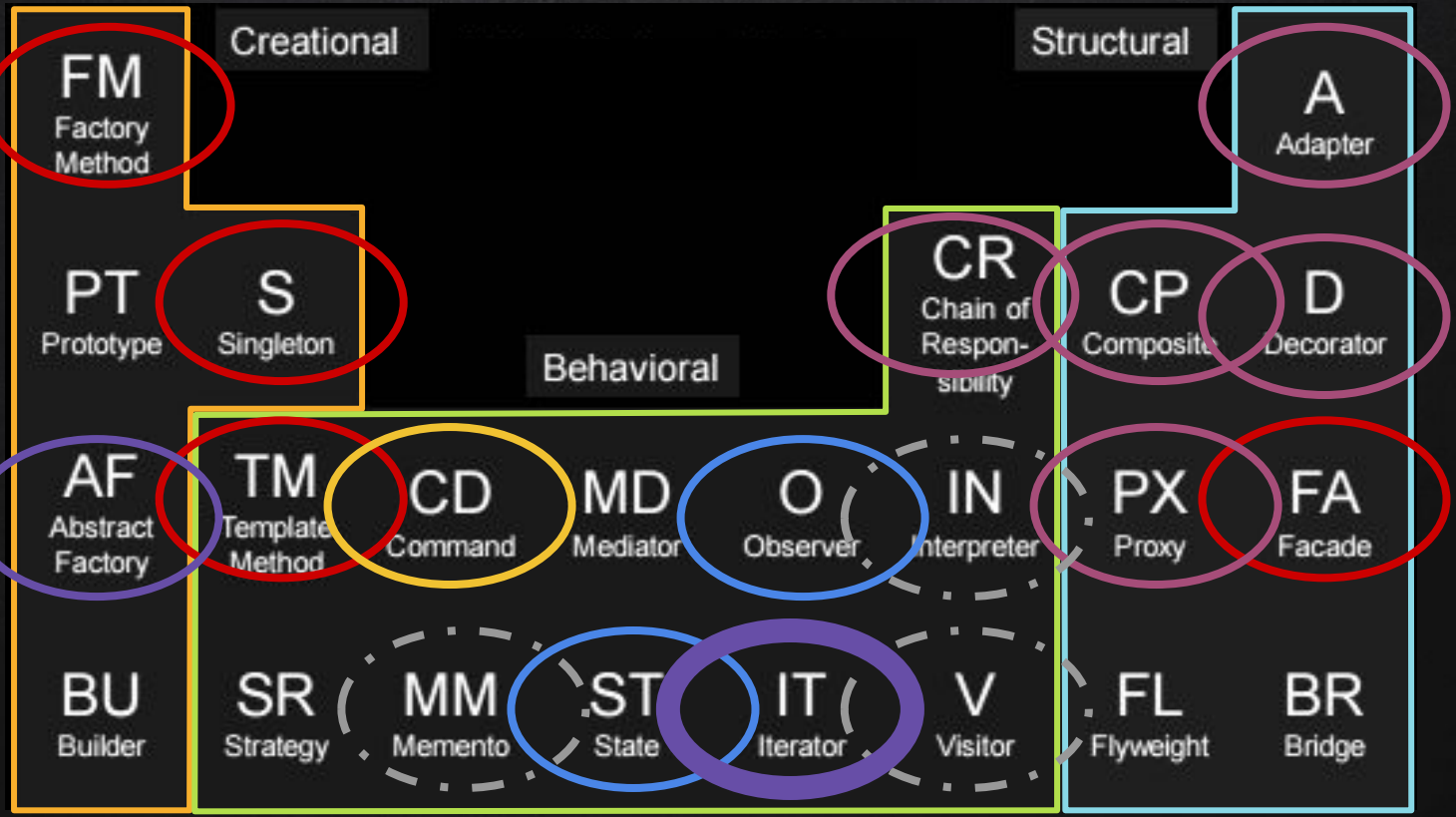
Está quase perfeito, mas ... de observar o ... individual dos patos.

Definitivamente, o padrão Observer!

# Tipos de padrões



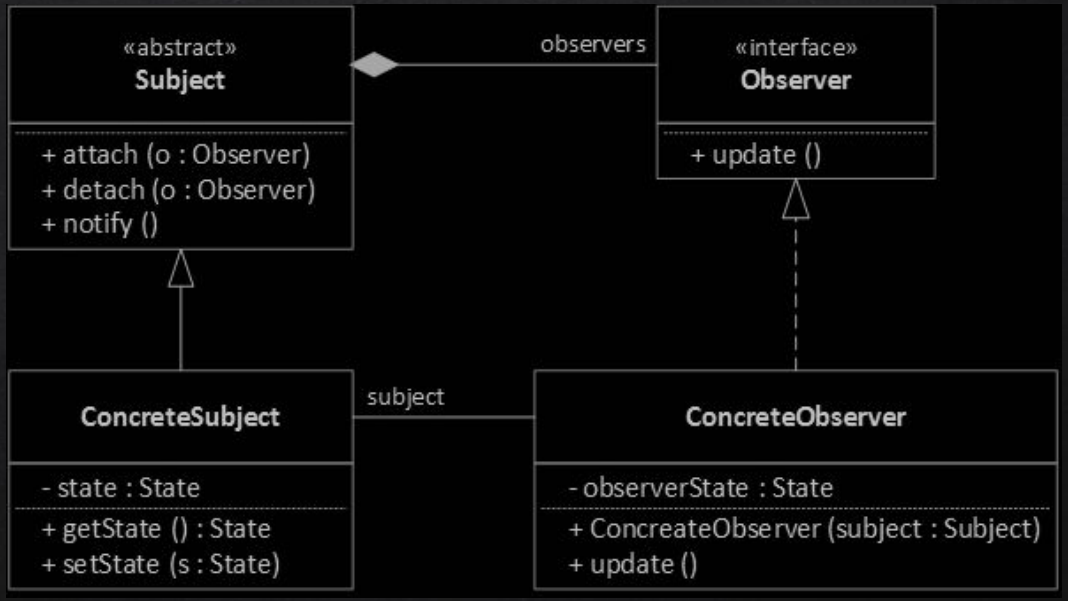
# Tipos de padrões



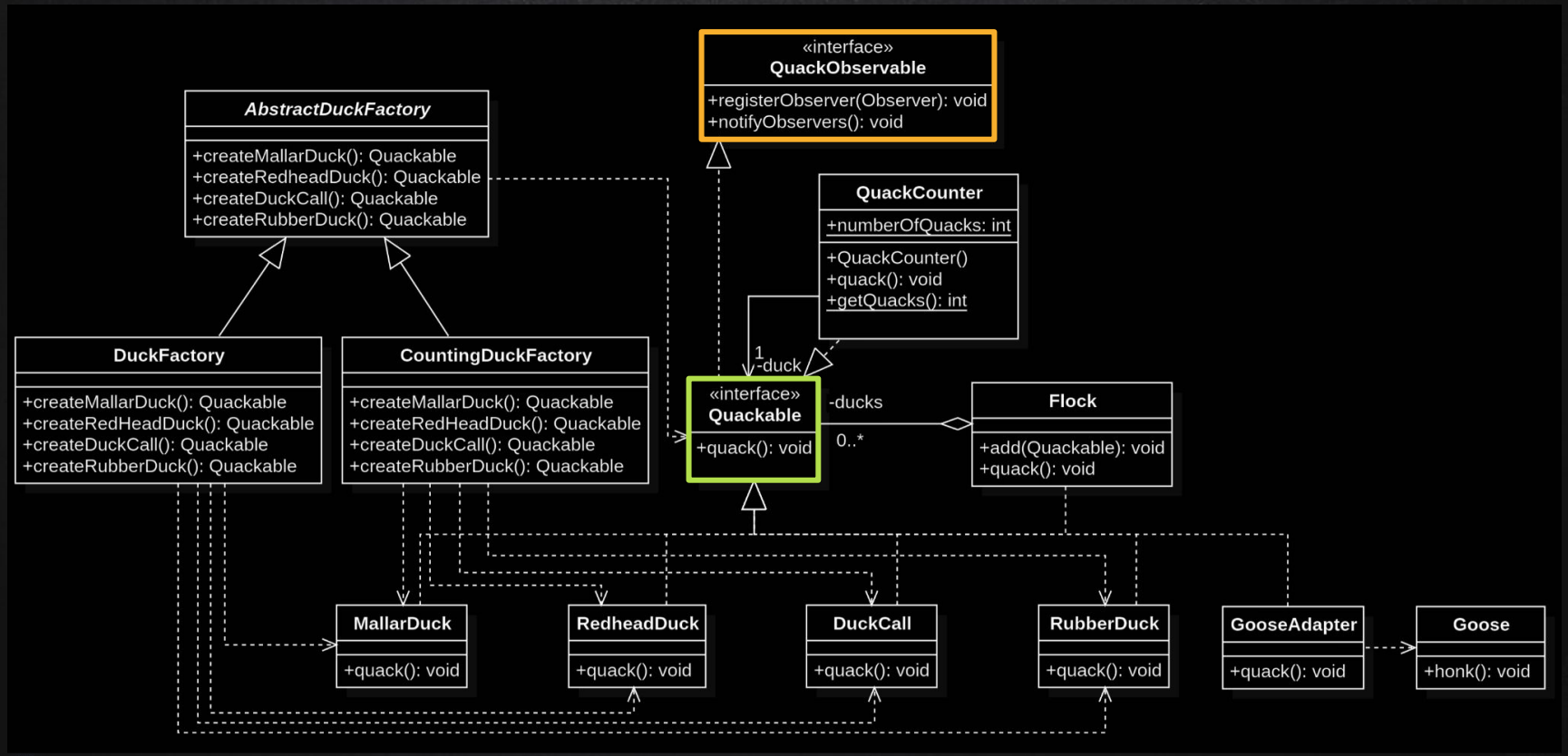
# Padrão Observer



Permite que um grupo de objectos (os observadores) seja notificado quando existe uma alteração ao estado do objecto observado (o sujeito).



# É preciso que Quackable se torne em QuackObservable

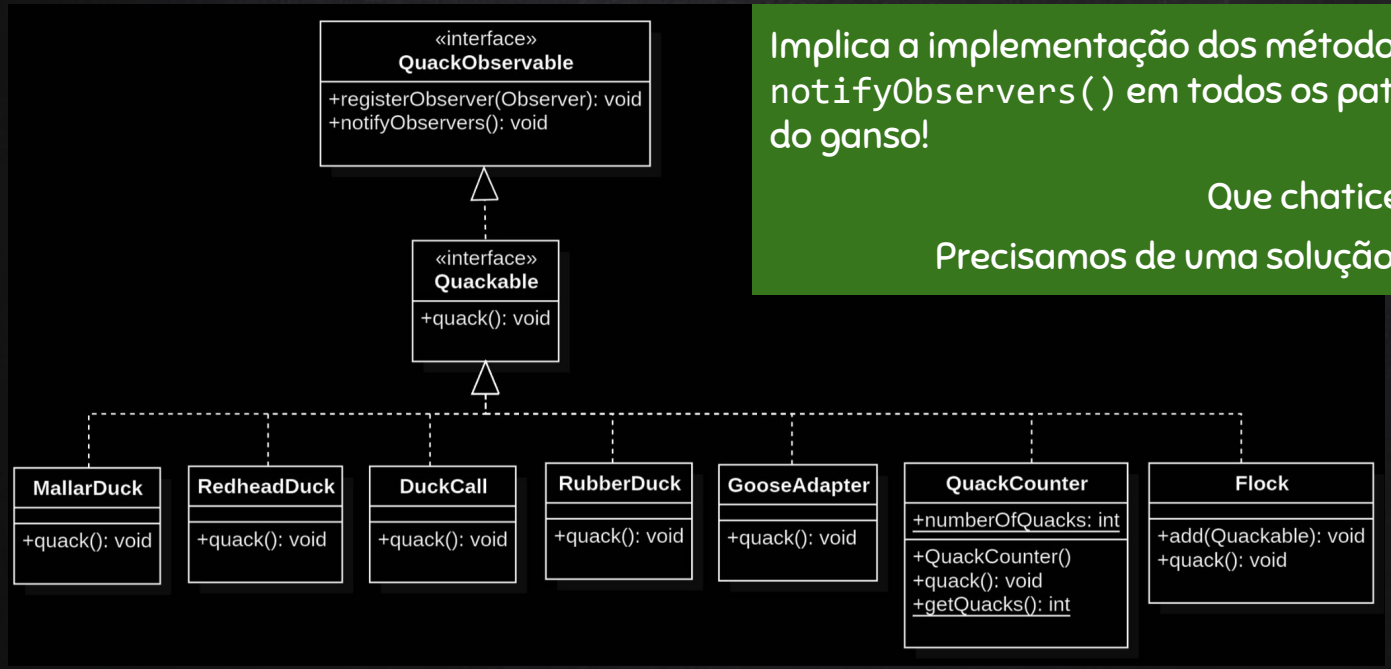


# É PRECISO QUE Quackable SE TORNE EM QuackObservable

```
public interface QuackObservable {
    public void registerObserver(Observer observer);
    public void notifyObservers();
}

public interface Quackable extends QuackObservable {
    public void quack();
}
```

# Ótimo, agora temos que fazer com que todas as instâncias `Quackable` implementem a interface `QuackObservable`?




Implica a implementação dos métodos `registerObserver()` e `notifyObservers()` em todos os patos... até mesmo no chato do ganso!

Que chatice!

Precisamos de uma solução mais inteligente...



# Implementação da classe Observable

```
public class Observable implements QuackObservable {
   private ArrayList<Observer> observers = new ArrayList<>();
   QuackObservable duck;
   public Observable(QuackObservable duck) {
      this.duck = duck;
   }
   public void registerObserver(Observer observer) {
      observers.add(observer);
   }
   public void notifyObservers() {
      Iterator iterator = observers.iterator();
      while (iterator.hasNext()) {
          Observer observer = (Observer)iterator.next();
          observer.update(duck);
      }
   }
}
```

# Implementação das classes concretas dos Patos

```
public class MallardDuck implements Quackable {
   Observable observable;
   public MallardDuck() {
      observable = new Observable(this);
   }
   public void quack() {
      System.out.println(“Quack”);
      notifyObservers();
   }
   public void registerObserver(Observer observer) {
      observable.registerObserver(observer);
   }
   public void notifyObservers() {
      observable.notifyObservers();
   }
}
```

# Definir a interface Observer

```
public interface Observer {
   public void update(QuackObservable duck);
}
```

# Definir a interface Observer e uma classe que a implemena: Quackologist

```
public interface Observer {
   public void update(QuackObservable duck);
}

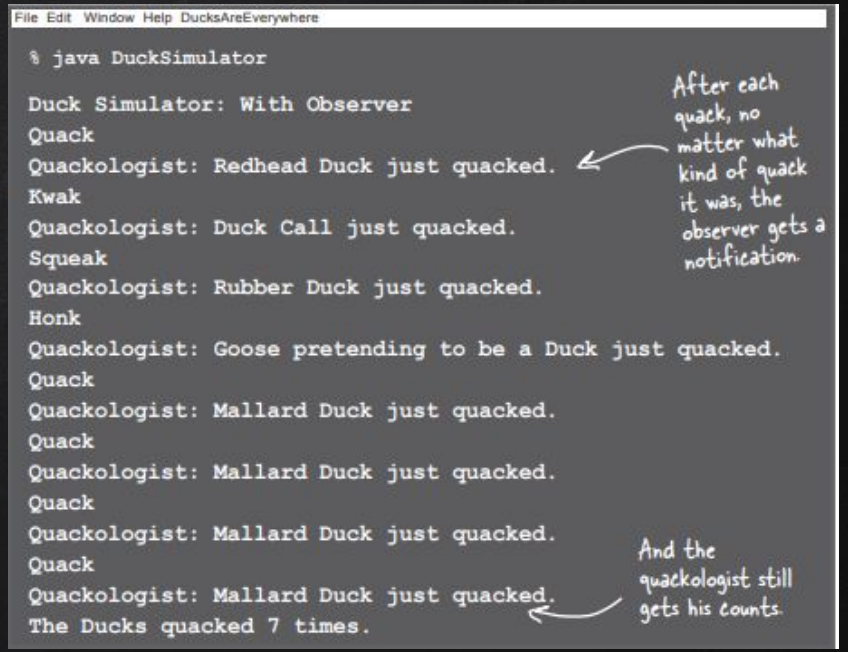
public class Quackologist implements Observer {
   public void update(QuackObservable duck) {
      System.out.println(“Quackologist: “ + duck + “ just quacked.”);
   }
}
```

# Implementação do simulador de Patos

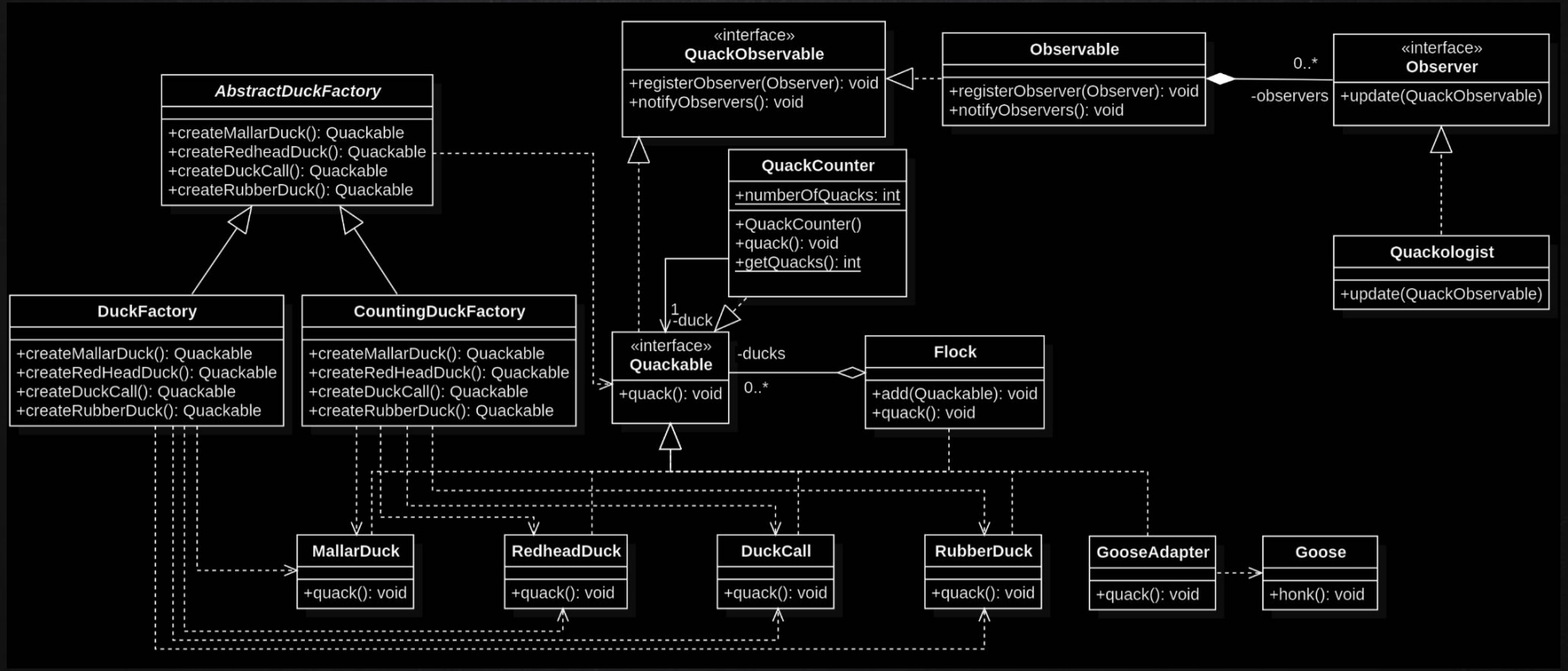
```
public class DuckSimulator {
   public static void main(String[] args) {
      DuckSimulator simulator = new DuckSimulator();
      AbstractDuckFactory duckFactory = new CountingDuckFactory();
      simulator.simulate(duckFactory);
   }
   void simulate(AbstractDuckFactory duckFactory) {
      // create duck factories and ducks here
      // create flocks here
      System.out.println("\nDuck Simulator: With Observer");
      Quackologist quackologist = new Quackologist();
      flockOfDucks.registerObserver(quackologist);
      simulate(flockOfDucks);
      System.out.println("\nThe ducks quacked " + QuackCounter.getQuacks() + " times");
   }
   void simulate(Quackable duck) {
      duck.quack();
   }
}
```

# Finalmente, implementar o simulador de patos

```
public class DuckSimulator {
   ...
   void simulate(Quackable duck) {
      duck.quack();
   }
}
```

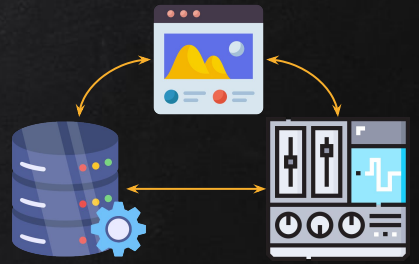


# Juntando tudo



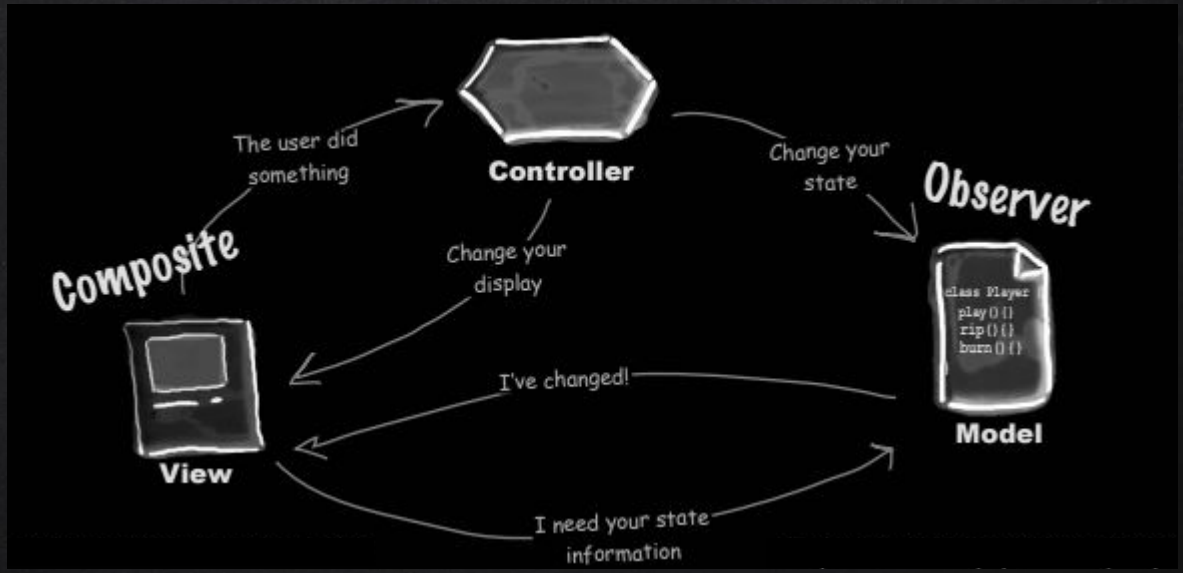
# Padrão MVC (Model, View, Controller)

O MVC usa o princípio de desenho de separação de preocupações para dividir as principais responsabilidades de um sistema interactivo em três partes: modelo, vista e controlador.



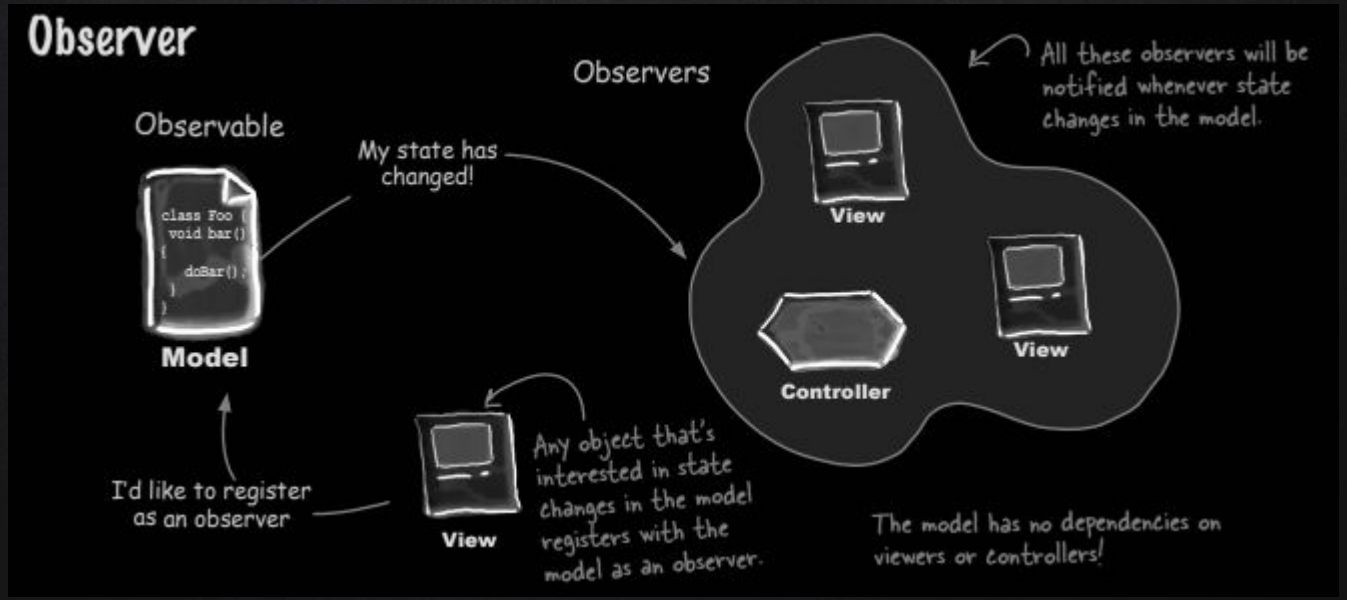
- O modelo contém os dados, o estado e a lógica subjacente que os utilizadores desejam ver e manipular através de uma interface.
  - O "back-end", ou seja, as informações do sistema ou do software subjacente, é independente, pois contém todo o estado, métodos e outros dados necessários para existir por si só.
- A vista é a forma de ver o modelo, permitindo a interacção, ou pelo menos parte dela.
  - É o "front-end" ou a camada de apresentação do software. Um modelo pode ter várias vistas que apresentam os seus diferentes components.
- O controlador interpreta os pedidos, interage com os elementos da visualização e altera o modelo.

# Esquemas MVC

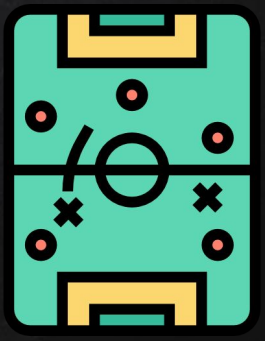


O modelo implementa o padrão Observer para manter os objetos interessados actualizados quando existem alterações de estado.

A utilização deste padrão mantém o modelo completamente independente das vistas e dos controladores, permitindo utilizar diferentes vistas para o mesmo modelo ou várias vistas ao mesmo tempo.

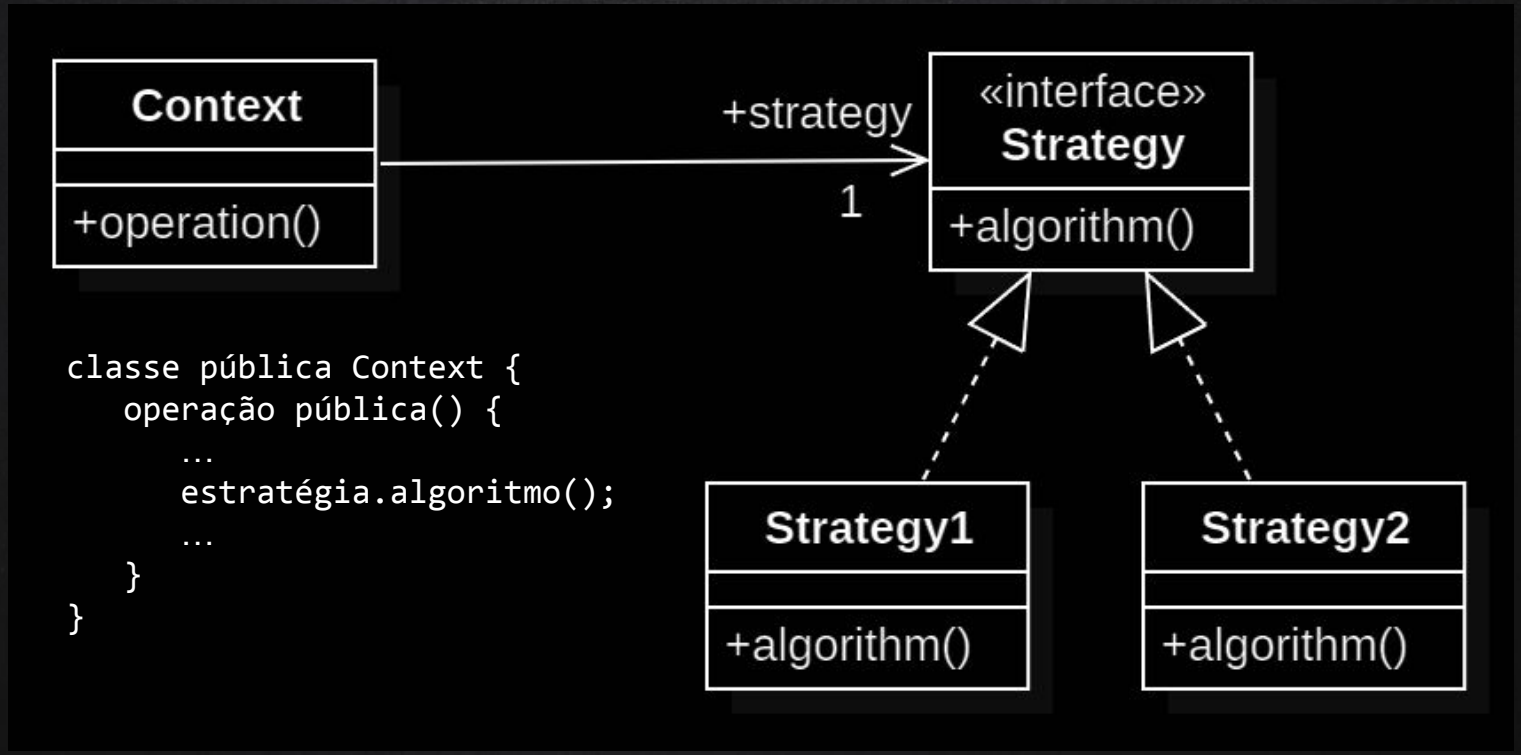


# Padrão Strategy (Behavioural)



- Utilizado quando precisamos fazer com que um algoritmo varie enquanto o resto do sistema permanece inalterado.
  - Algoritmos diferentes serão apropriados em momentos diferentes.
- Não queremos que a classe suporte vários algoritmos.
  - Isso tornaria a classe desnecessariamente complexa (ou, pelo menos, o método).
- Evitamos estes problemas definindo classes que encapsulam os vários algoritmos
  - Cada classe que representa um algoritmo é uma estratégia

# Padrão Strategy



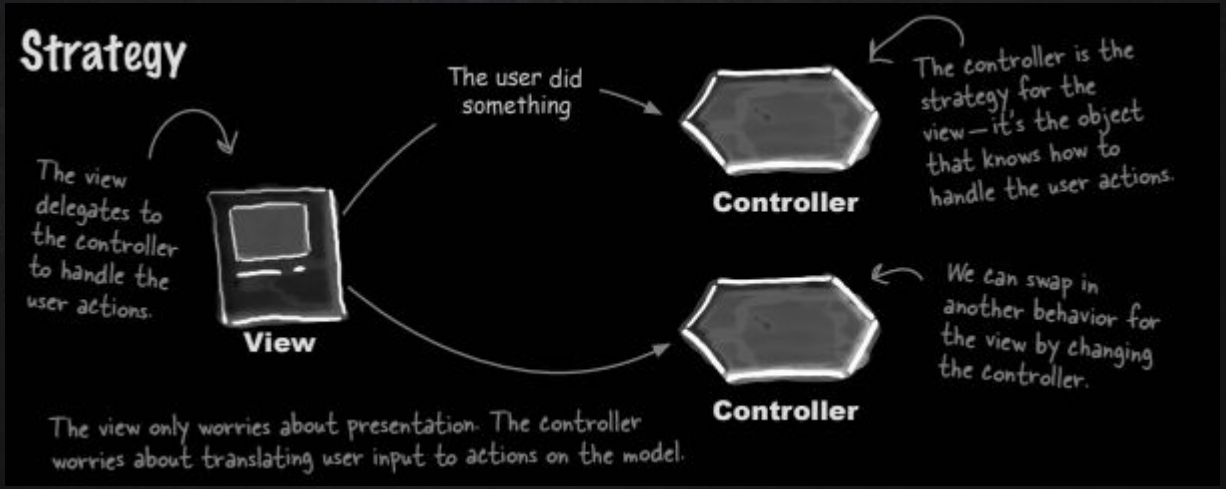

```
classe pública Context {
   operação pública() {
      …
      estratégia.algoritmo();
      …
   }
}
```

A vista e o controlador implementam o padrão Strategy: a vista é um objeto configurado com uma estratégia. O controlador oferece a estratégia.

A vista preocupa-se apenas com os aspectos visuais da aplicação e delega ao controlador todas as decisões sobre o comportamento da interface.

A utilização do Strategy também mantém a vista desacoplada do modelo, visto que o controlador é responsável por interagir com o modelo para executar os pedidos do utilizador.

A vista não sabe nada sobre esta interação.



# Referências


Design Patterns: Elements of Reusable Object-Oriented Software

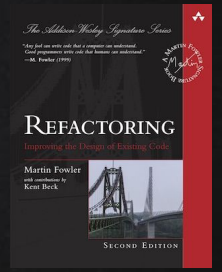
Erich Gamma, John Vlissides, Richard Helm, Ralph Johnson, Pearson Education, Addison-Wesley Professional






Refactoring, Improving the Design of Existing Code

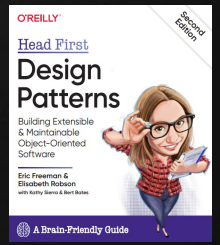
Martin Fowler, with Kent Beck, Addison-Wesley, 2018



Head First Design Patterns: Building Extensible & Maintainable Object-Oriented Software

Eric Freeman & Elisabeth Robson, O'Reilly


Some content of the previous slides was taken on 2018 University of Alberta course on design patterns



# Referências

Exemplos de implementação de padrões:

https://refactoring.guru/design-patterns



Porque é que os programadores confundem o Natal com o Halloween?

Porque 31 de outubro == 25 de dezembro